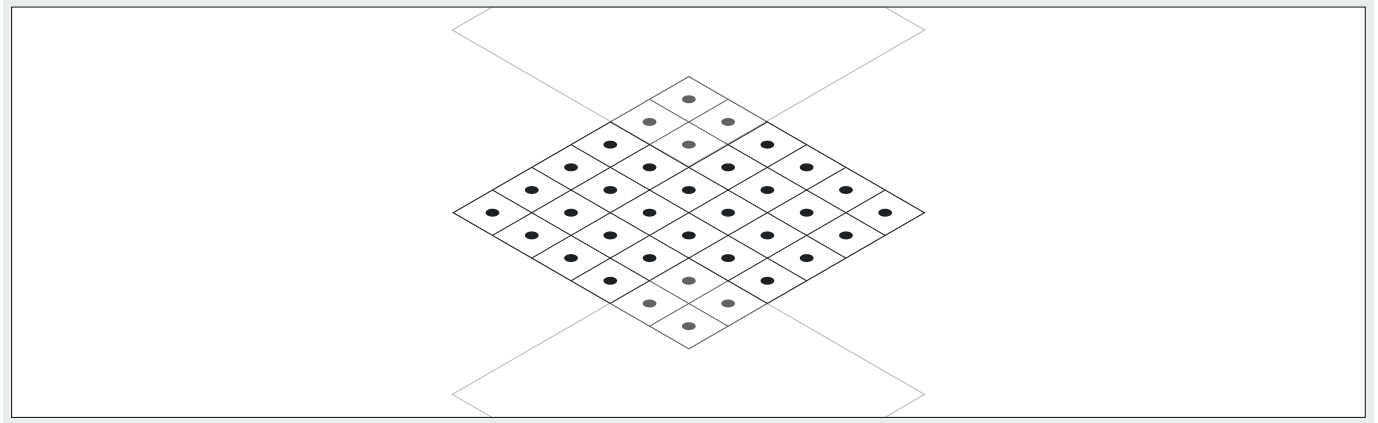


Table of Contents

1. Overview • Data integration • Palantir
2. Overview • Data integration • Palantir
3. Connecting to data • Palantir
4. What is a data pipeline? • Palantir
5. Application reference • Palantir
6. Core concepts • Datasets • Palantir
7. Core concepts • Streams • Palantir
8. Core concepts • Media sets (unstructured data) • Palantir
9. Core concepts • Branching • Palantir
10. Core concepts • Builds • Palantir
11. Core concepts • Schedules • Palantir
12. Core concepts • Health checks • Palantir
13. Core concepts • Iceberg tables • Palantir
14. Core concepts • Virtual tables • Palantir
15. Core concepts • Change data capture (CDC) • Palantir
16. Core concepts • Views • Palantir
17. Overview • Data integration • Palantir
18. Palantir-provided drivers for JDBC sources • Palantir
19. Resource guides • Streaming • Palantir
20. Resource guides • Flink fundamentals • Palantir
21. Resource guides • Reset stream • Palantir
22. Resource guides • Stream monitoring • Palantir
23. Resource guides • Streaming profiles • Palantir
24. S3-compatible API for Foundry datasets • Palantir

Data connectivity & integration / Overview



Data connectivity and integration

Foundry provides a highly configurable set of data connectivity and integration tools that extend far beyond typical extract-transform-load (ETL) or extract-load-transform (ELT) solutions. Foundry is designed to reduce the cost of data integration over time through a rich suite of capabilities that act as a force multiplier for data teams. While commodity cloud services provide storage and compute for basic pipelines and experimentation, many additional layers of capability are required to manage, deliver, and validate datasets for critical operations. Foundry is designed to serve as the data integration backbone for the most complex environments in the world.

Connecting data

This begins with an extensible data connection framework that establishes connections with all types of source systems – structured, unstructured, or semi-structured – and with all key data transfer approaches, such as batch, micro-batch, or streaming. This functionality is integrated with the platform's data transformation and data management functionality, with a lineage of data versions, granular security for collaborative management of data, and branching of data sync configurations.

[API Reference ↗](#)



[Send feedback](#)

[Learn more about connecting to data in Foundry.](#)

Data transformation

For **data transformation**, Foundry provides an extensible, scalable *build system for data* which leverages multimodal compute to produce output datasets. Foundry's compute-agnostic "Build" framework provides fully integrated security and data lineage, and enables mixing-and-matching of third-party compute runtimes. Foundry also includes an integrated suite of data transformation authoring, change management, data quality, pipeline scheduling, and metadata introspection capabilities that work cohesively to provide a "mission control" for data engineers.

[Learn more about transforming data with Foundry.](#)

Pipeline management

Foundry's **pipeline management** capabilities combine change management, data quality, and data loading features.

The Pipeline Builder application enables fast, flexible, and scalable delivery of data pipelines while providing robustness and security. [Learn more about Pipeline Builder.](#)

Data engineers can define a rigorous release process for production pipelines, including health checks that guarantee only fully compliant data will be deployed to production. Where issues are found, the platform provides diagnostics on the discrepancies detected.

Diagnostics are available both in Foundry's integrated analysis and modeling tools, as well as to any third-party tools accessing the outputs via REST APIs or other interfaces.

[Learn more about maintaining and managing pipelines in Foundry.](#)

NEXT

Connecting to data →

[API Reference ↗](#)

© 2026 Parantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Send feedback](#)

[Privacy Statement ↗](#)

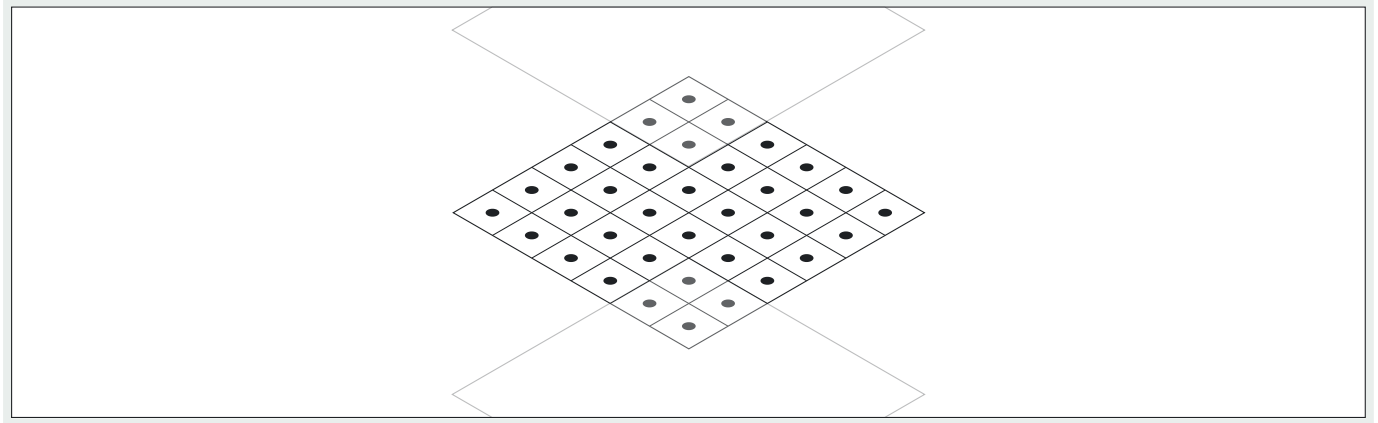
[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Data connectivity & integration / Overview



Data connectivity and integration

Foundry provides a highly configurable set of data connectivity and integration tools that extend far beyond typical extract-transform-load (ETL) or extract-load-transform (ELT) solutions. Foundry is designed to reduce the cost of data integration over time through a rich suite of capabilities that act as a force multiplier for data teams. While commodity cloud services provide storage and compute for basic pipelines and experimentation, many additional layers of capability are required to manage, deliver, and validate datasets for critical operations. Foundry is designed to serve as the data integration backbone for the most complex environments in the world.

Connecting data

This begins with an extensible data connection framework that establishes connections with all types of source systems – structured, unstructured, or semi-structured – and with all key data transfer approaches, such as batch, micro-batch, or streaming. This functionality is integrated with the platform's data transformation and data management functionality, with a lineage of data versions, granular security for collaborative management of data, and branching of data sync configurations.

[API Reference ↗](#)[Send feedback](#)[Learn more about connecting to data in Foundry.](#)

Data transformation

For **data transformation**, Foundry provides an extensible, scalable *build system for data* which leverages multimodal compute to produce output datasets. Foundry's compute-agnostic "Build" framework provides fully integrated security and data lineage, and enables mixing-and-matching of third-party compute runtimes. Foundry also includes an integrated suite of data transformation authoring, change management, data quality, pipeline scheduling, and metadata introspection capabilities that work cohesively to provide a "mission control" for data engineers.

[Learn more about transforming data with Foundry.](#)

Pipeline management

Foundry's **pipeline management** capabilities combine change management, data quality, and data loading features.

The Pipeline Builder application enables fast, flexible, and scalable delivery of data pipelines while providing robustness and security. [Learn more about Pipeline Builder.](#)

Data engineers can define a rigorous release process for production pipelines, including health checks that guarantee only fully compliant data will be deployed to production. Where issues are found, the platform provides diagnostics on the discrepancies detected.

Diagnostics are available both in Foundry's integrated analysis and modeling tools, as well as to any third-party tools accessing the outputs via REST APIs or other interfaces.

[Learn more about maintaining and managing pipelines in Foundry.](#)

NEXT

Connecting to data →

[API Reference ↗](#)

© 2026 Parantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Send feedback](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Data connectivity & integration / Connecting to data

Connecting to data

The first step to getting value from Foundry is to connect it to your Organization's sources of data. Foundry's tools for connecting to data support the full range of standard enterprise data sources, ranging from cloud-based object stores, file systems, and databases and data warehouses.

You can connect to data in a variety of ways with different Foundry applications, depending on the type of data you need to access.

Data Connection

Connect to sources to run batch, streaming, media, and CDC syncs and to use virtual tables.

The [Data Connection](#) framework is designed to manage data over time, through discrete versions that are managed using dataset [transactions](#). This framework enables full lineage of data versions across time, providing you with an understanding of which sync tasks produced which versions of a given dataset. It also enables syncing of only the data required, in cases where full data loading on each sync is not possible.

Granular security in Data Connection allows federated management of data syncs across different teams. Collections of syncs, or even individual data syncs, can be made visible or editable to only specific teams (defined through role- or classification-based access controls). Learn more about [securing a data foundation](#).

You can manage sync metadata independently of the actual sync definitions. This allows for full configurations, where the new sync is sandboxed and tested in a branch before it affects any downstream transformation jobs.

API Reference ↗



Send feedback

HyperAuto

To evolve beyond simple data syncing solutions, [Palantir HyperAuto](#) implements support for Software-Defined Data Integration (SDDI). This toolset allows organizations to not only connect to common ERP and CRM systems, but also to programmatically generate data pipelines that clean, normalize, and harmonize datasets into a cohesive data asset at unprecedented speed. This data asset can then feed into the [Ontology](#) to translate data into operational value.

External transforms

Perform scheduled syncs and exports to external systems using REST APIs.

If you want to connect to external sources to create syncs and export data, we recommend using Code Repositories to write [external Python transforms](#) using the REST API. You can also add dataset inputs and media set outputs to your transforms.

PREVIOUS

← [Overview](#)

NEXT

[What is a data pipeline?](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

[Terms of Use](#) ↗

— [Cookies Settings](#)

[API Reference](#) ↗

[Send feedback](#)

Data connectivity & integration / What is a data pipeline?

What is a data pipeline?

The overall goal of data integration in Foundry is to provide a digital view of the objective reality within your Organization. Achieving this goal typically requires syncing data from many source systems, imposing a common schema, combining datasets together, and enabling teams to build use cases off a common data foundation.

Within this context, the term "data pipeline" is widely used to refer to the flow of data from a source system through intermediate datasets to ultimately produce high-quality, curated datasets that can be structured into the [Ontology](#) or serve as the foundation of [machine learning](#) and [analytical](#) workflows.

Although any two datasets in Foundry that are connected together via transformation logic could be considered a pipeline, in practice what we refer to as a "data pipeline" is more constrained. Typically, a pipeline has a notion of *ownership*—a person or group of people who oversees the pipeline to ensure that data flows through it regularly and reliably to power business processes.

Beyond a notion of ownership, there are several other characteristics associated with a high-quality, production-ready data pipeline. We explore these ideas throughout the rest of this document and provide links to additional resources to learn more:

- [What is a data pipeline?](#)
 - [Pipeline setup](#)
 - [Build scheduling](#)
 - [Data quality](#)
 - [Security and governance](#)

API Reference ↗ [Processes and documentation](#)

Ir   o features common to all pipelines, consider which type [Send feedback](#)
created for your data foundation based on factors such as data scale, latency requirements,

and maintenance complexity. There are three primary types of pipelines available in Foundry: batch, incremental, and streaming. [Learn more about the types of pipelines.](#)

Pipeline setup

Foundry's Pipeline Builder enables users to quickly and easily set up a pipeline with a streamlined point-and-click interface. With Pipeline Builder, users gain the benefits of Git-style change management, data health checks, multi-modal security, and fine-grain data auditing.

Technical users can build and maintain pipelines more rapidly than ever before, focusing on declarative descriptions of their end-to-end pipelines and desired outputs. Additionally, Pipeline Builder's point-and-click, form-based interface enables less technical users to create pipelines through a simplified approach.

- Learn to [build a pipeline using Pipeline Builder.](#)

Build scheduling

Simply put, a series of data transformations must run regularly in order to be considered a data pipeline. Defining build [schedules](#) in Foundry is a basic step in building a pipeline, as downstream data consumers expect data to update regularly. The frequency with which data flows through a pipeline is subject to organizational requirements: some pipelines may run only weekly or daily, while others run on an hourly or even more frequent basis.

The following resources help you get started with scheduling builds in Foundry:

- Learn to [create a schedule.](#)
- Learn about [scheduling best practices.](#)
- Learn to [troubleshoot existing schedules.](#)

Data quality

API Reference ↗

In the initial stages of defining a pipeline, we recommend frequently checking inputs and outputs at every step. Data synced from source systems often includes

Send feedback

undefined values and poorly formatted or inconsistent data. Cleaning and normalizing data is a core part of the pipeline building process.

Tools for checking assumptions about datasets are available throughout Foundry:

- [Dataset Previews](#) support computing statistics on any column of a dataset, as well as filtering to a subset of rows to quickly check expectations.
- Code Repositories' support for [debugging transforms](#) can be used to check that input datasets are structured as expected while authoring transformation logic.
- Applications in Foundry's analytical suite, especially [Contour](#), can be very helpful for validating assumptions about datasets in a point-and-click fashion.

After your pipeline has been established, [health checks](#) are the recommended way to validate that data remains high-quality over time. Here are some resources to get started with health checks:

- Learn about [recommended health checks](#).
- Read the [checks reference](#) to learn about the range of available checks.

Security and governance

Foundry's platform security primitives provide best-in-class capabilities for securing a data foundation and ensuring that sensitive data is handled appropriately. The cross-cutting concepts of [Projects](#) and [Markings](#) provide support for discretionary and mandatory controls, respectively, which can be used to comply with the full range of governance requirements.

To learn more about how to handle data securely in your pipeline, refer to these sections:

- The [recommended Project structure](#) provides a simple framework that is suitable for most pipelines.
- [Securing a data foundation](#) walks through an end-to-end example of best practices.
- [Protecting sensitive data](#) describes how to handle highly sensitive data such as personally identifiable information, or PII.

API Reference ↗

Support processes and documentation

Send feedback

Once a pipeline has been published to production by following the above guidelines, it is important to think through the longevity of the pipeline from an organizational perspective. Support processes for pipeline maintenance should be fleshed out, expectations should be clearly defined, and documentation should be available so that pipelines remain high-quality even as they are handed off from one team to another.

Learn more about these best practices:

- [Pipeline definition and expectations](#)
- [Set up support processes](#)

Data connectivity & integration / Application reference

Application reference

This page provides a reference to the Foundry applications you may encounter while performing data integration workflows.

Pipeline Builder

Pipeline Builder is Foundry's primary application for data integration. With Pipeline Builder, you can create end-to-end pipeline workflows, from data sources to final outputs. Users of Pipeline Builder can describe their workflow, transform data, edit schemas, and build outputs in a single easy-to-use application.

Pipeline Builder features an intuitive point-and-click interface and robust backend model that allows technical and less-technical users to define and deploy pipelines faster than in code-heavy applications. The streamlined builder interface allows users to apply data transforms alongside schema checks, saving time and costs typically spent on computation and checks at build time. Additional features like full version control and extensibility make Pipeline Builder an ideal application for safe collaboration.

API Reference ↗



Send feedback

The screenshot displays the Foundry Pipeline Builder interface. At the top, there's a menu bar with options like 'File', 'Settings', 'Help', and 'Batch'. Below this is a toolbar with icons for 'Tools', 'Select', 'Remove', 'Layout', 'Add datasets', 'Parameters', 'Transform', and 'Edit'. The main workspace shows a data pipeline with several steps: 'Facility dataset' (2 columns), 'Clean Facility Data' (21 columns), 'Data column data' (21 columns), 'Facility' (5 columns), 'Vendor Out 1 - demo data' (17 columns), 'Clean out errors' (17 columns), 'Union' (17 columns), 'Extra vendor attributes' (17 columns), and 'Vendor' (6 columns). A legend on the right identifies 'Facility' as a green box and 'Vendor' as an orange box. Below the pipeline, a 'Data preview' section shows a table with columns: id, address, city, zip, latitude, and longitude. The table contains 11 rows of data. On the far right, a 'Datasets' panel lists 'Vendor' and 'Facility' datasets with their respective column counts and a warning that one column will be dropped.

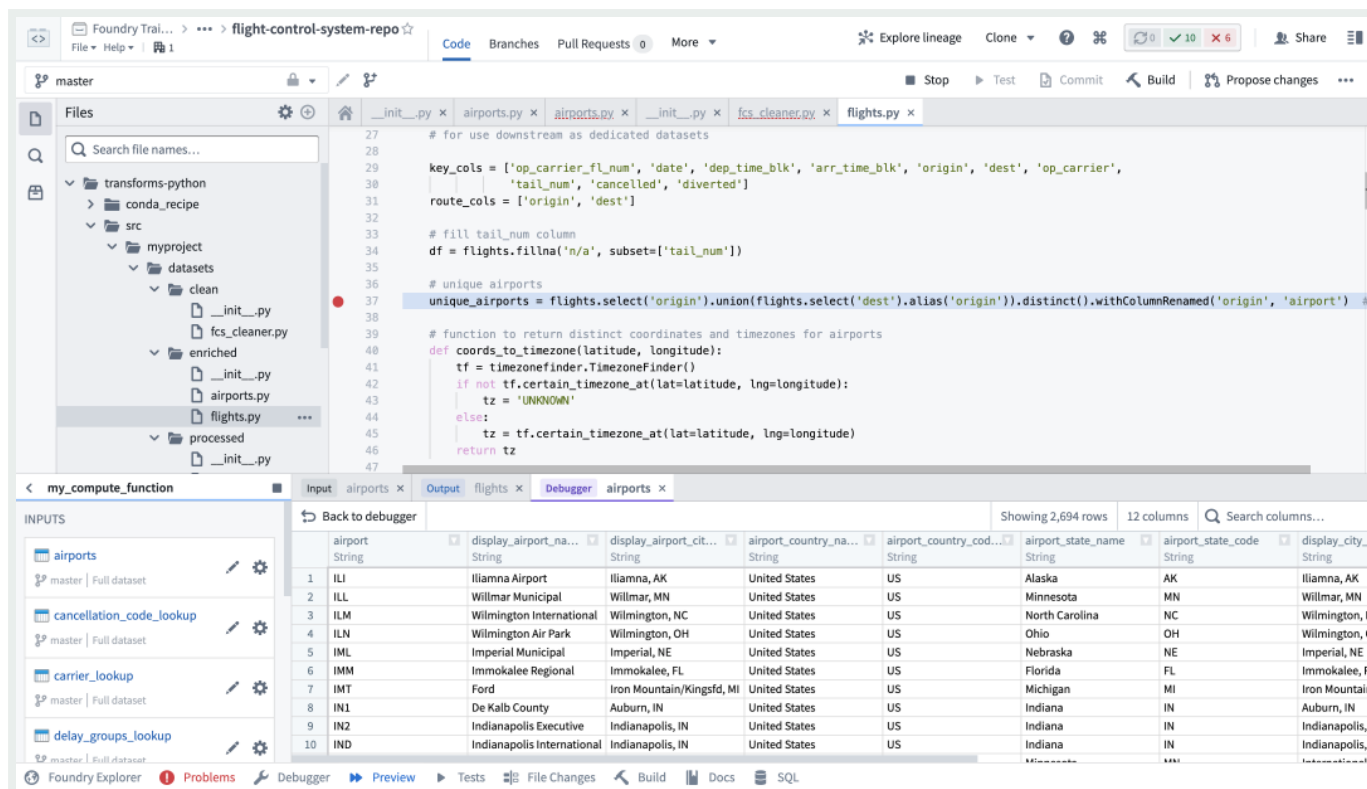
	id	address	city	zip	latitude	longitude
1	0022093277	1100 SO. AKERS STREET	VISALIA	93277	36.320843946	
2	0013177954	1100 EAST LOOP 304	CROCKETT	77954	31.32360364	
3	0015371373	209 FRONT ST.	VIDALIA	71373	31.5611748630001	
4	0072590723	16453 SOUTH COLORADO	PARAMOUNT	90723	33.8646318610001	
5	0196500984	CALLE FERNANDEZ JUNCO	CAROLINA	00984	18.3804430020001	
6	0196600956	AVE LAUREL 100, URB. SAN	BAYAMON	00956	18.3676556610001	
7	0015954560	500 E VETERANS ST	TOMAH	54660	44.00308068	
8	0005854452	601 S CENTER AVE	MERRILL	54452	45.1756325370001	
9	0021891761	2295 S. VINEYARD AVENUE	ONTARIO	91761	34.0320795620001	
10	0019090027	4867 SUNSET BOULEVARD	LOS ANGELES	90027	34.0985736490001	
11	0042990020	3160 GENEVA STREET	LOS ANGELES	90020	34.067204777	

Code Repositories

Code Repositories is Foundry's primary interface for authoring code, most commonly used for creating data pipelines in Python, Java, and SQL. Code Repositories provides an integrated development environment (IDE) on top of a `git` server, enabling collaboration and governance of pipeline logic, as well as native support for writing, testing, and previewing data transformation logic. Code Repositories can also be used for authoring [machine learning models](#) and [Ontology Functions](#).

API Reference ↗

Send feedback



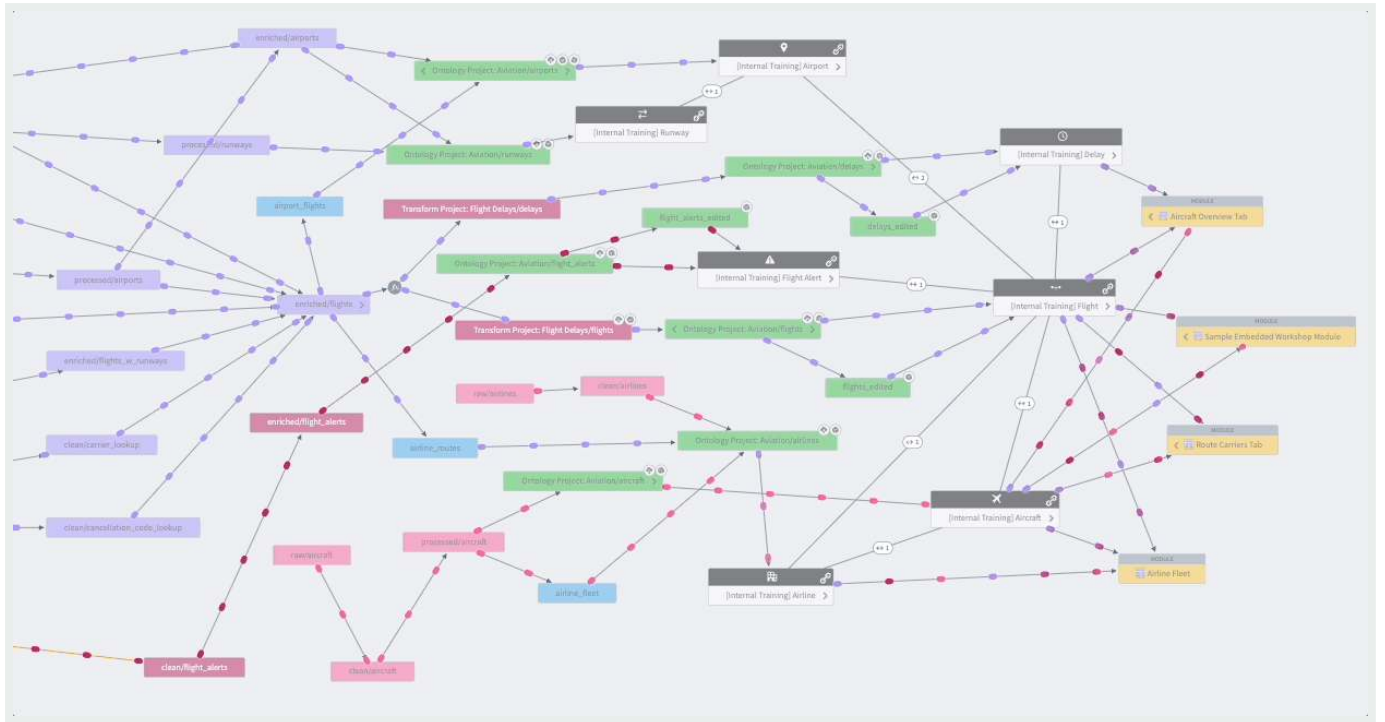
If you are interested in data science and code-based analysis, [Code Workbook](#) may be a better fit for your use case. [Learn more about the differences between Code Workbook, Code Workspaces, and Code Repositories.](#)

Data Lineage

Data Lineage is an application that shows how data flows through Foundry. You can use it to explore how any resource in Foundry is connected to other resources, across the boundaries of individual Projects or use cases. This includes support for data sources, datasets, analyses, Ontology object and link types, and user-facing applications. In addition to exploring connections, you can use Data Lineage to view previews of data, see the logic used to derive any piece of data, and manage scheduled pipelines.

[API Reference ↗](#)

[Send feedback](#)



Data Connection

Data Connection is the application used to sync data into Foundry and manage associated resources including source credentials. After initial setup, Data Connection makes it simple to explore data sources and sync new data for use case development, while complying with the full range of governance controls required for managing source systems and use cases at scale.

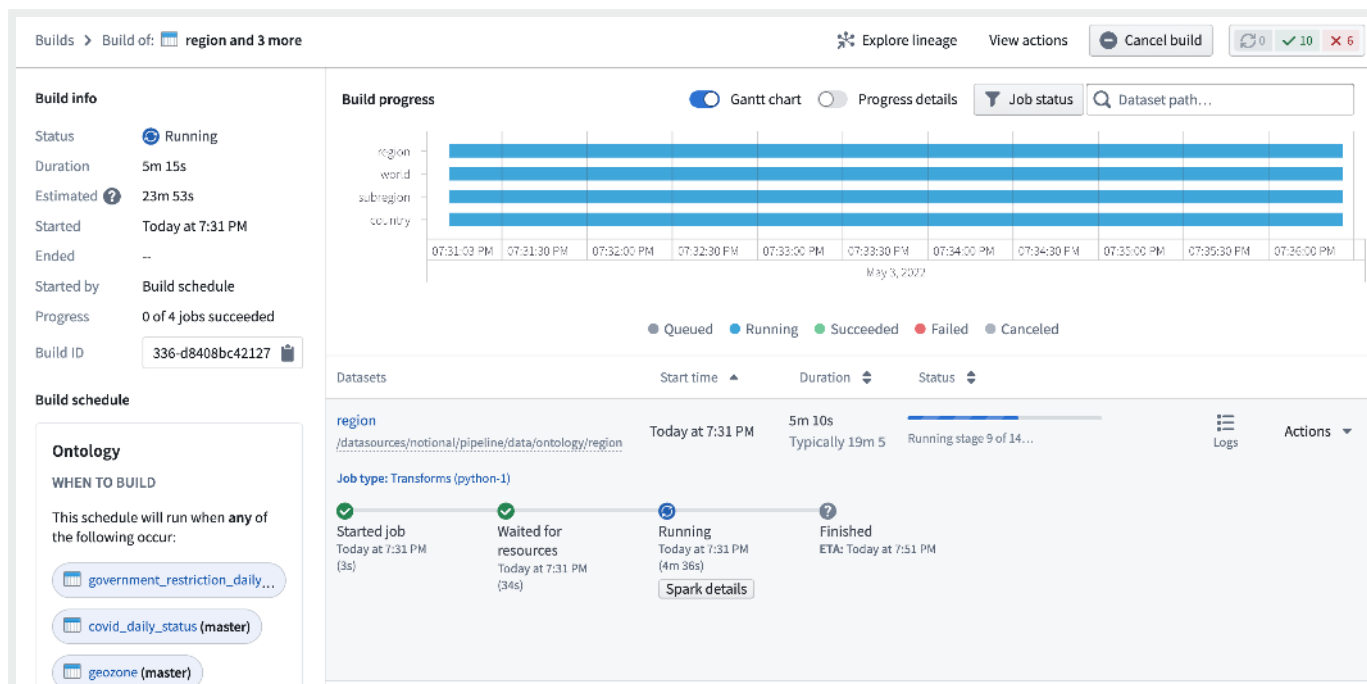
Dataset Preview

Dataset Preview is an application used to view and understand [datasets](#). Opening a dataset from any other application shows you the contents of the dataset, along with a range of contextual information. This includes information about dataset ownership, how the dataset has changed over time, any applicable health checks, and further details.

API Reference ↗

Send feedback

Send feedback



PREVIOUS
← What is a data pipeline?

CORE CONCEPTS / DATASETS
NEXT →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Datasets

A **dataset** is the most essential representation of data from when it lands in Foundry through when it is mapped into the [Ontology](#). Fundamentally, a dataset is a wrapper around a collection of **files** which are stored in a [backing file system](#). The benefit of using Foundry datasets is that they provide integrated support for permission management, schema management, version control, and updates over time. We explore the concepts underlying this functionality throughout the rest of this document.

In Foundry's data integration layer, datasets can store and represent all types of data, whether structured, unstructured, or semi-structured data:

- *Structured* (tabular) data consists of files that contain tabular data in an open-source format such as [Parquet](#) [↗](#), along with metadata about the columns in the dataset. This metadata is stored alongside the dataset as a [schema](#).
- *Unstructured* data consists of files that do not have an associated schema; examples include non-tabular files like customer support emails, as well as media files like images, video, or PDF documents. Foundry provides two ways to store unstructured data: as a schema-less dataset or as a [media set](#), a separate resource type optimized for media workflows. If you are [manually uploading unstructured data](#) in Foundry, you can choose to either **Upload as media set** or **Upload to a dataset without a schema**.
- *Semi-structured* data is non-tabular, but can still contain tags or metadata that provide some amount of organization; examples include file formats such as XML or JSON. There is no dedicated upload option for semi-structured data; instead, you can upload these files to a dataset without a schema, then [infer a tabular schema](#) in a downstream data transformation for performance and ease of use.

Datasets are designed to change over time. When you open a dataset in Foundry and see rows and columns, what you are seeing is actually the latest [dataset view](#).

As an end user, you will typically modify datasets using [builds](#), which allow you to update the contents of a dataset according to logic that you specify. Behind the scenes, however, datasets are updated over time using **transactions**, which represent modifications to the files within a dataset. A transaction has a simple lifecycle:

- After a transaction is *started*, it is in an **OPEN** state. In this state, files can be opened and written in the dataset's backing file system.
- A transaction can be *committed*, which puts it into a **COMMITTED** state and any written files are now in the latest dataset view.
- A transaction can be *aborted*, which puts it into an **ABORTED** state. Any files that were written during the transaction are ignored.

If you are a software engineer, you may be familiar with Git. Dataset transactions are the basis of Foundry's support for data versioning, sometimes referred to as "Git for data." A transaction is analogous to a *commit* in Git: an atomic change to the contents of a dataset.

Transaction types

The way dataset files are modified in a transaction depends on the transaction type. There are four possible transaction types: **SNAPSHOT**, **APPEND**, **UPDATE**, and **DELETE**.

SNAPSHOT

A **SNAPSHOT** transaction replaces the current view of the dataset with a completely new set of files.

SNAPSHOT transactions are the simplest transaction type, and are the basis of [batch pipelines](#).

 [API Reference ↗](#)

An **APPEND** transaction adds new files to the current dataset view.

[Send feedback](#)

An **APPEND** transaction cannot modify existing files in the current dataset view. If an **APPEND** transaction is opened and existing files are overwritten, then attempting to commit the transaction will fail.

APPEND transactions are the basis of [incremental pipelines](#). By only syncing new data into Foundry and only processing this new data throughout the pipeline, changes to large datasets can be processed end-to-end in a performant way. However, building and maintaining incremental pipelines comes with additional complexity. [Learn more about incremental pipelines.](#)

UPDATE

An **UPDATE** transaction, like an **APPEND**, adds new files to a dataset view, but may also overwrite the contents of existing files.

 An **UPDATE** transaction that overwrites the contents of existing files will break the append-only requirement for [incremental pipelines](#). If a dataset receives such an **UPDATE** transaction, downstream pipelines cannot process data incrementally and must fall back to **SNAPSHOT** (batch) processing.

An **UPDATE** transaction that only adds new files, without overwriting any existing ones, does not affect incrementality. Do not overwrite existing files in an **UPDATE** transaction unless there is no other option. For more details, see [incremental transforms and append-only inputs](#).

When configuring [file-based syncs](#) to ingest data from an external system, the transaction type you select determines how data flows through your pipeline. The three common ingestion patterns are:

- **Batch mirror** (**SNAPSHOT**): Each sync run ingests all files and commits a **SNAPSHOT** transaction. This is the simplest pattern and is suitable when the full set of files is small enough to reload each run.
- **Incremental mirror** (**APPEND**): Each sync run ingests only new files and commits an **APPEND** transaction. This pattern supports end-to-end incremental pipelines and is the [API Reference ↗](#) approach when files are only added, not modified.
- **Incremental mirror** (**UPDATE**): Each sync run ingests new and modified files and commits an **UPDATE** transaction. Use this pattern only when the external system

Send feedback

modifies existing files, as it prevents downstream incremental processing.

For detailed configuration guidance on these patterns, including filter settings and contradictory options, see [file-based syncs](#).

DELETE

A **DELETE** transaction removes files that are in the current dataset view.

Note that committing a **DELETE** transaction does not delete the underlying file from the backing file system—it simply removes the file reference from the dataset view.

In practice, **DELETE** transactions are mostly used to enable data retention workflows. By deleting files on a dataset based on a retention policy—typically based on the age of the file—data can be removed from Foundry, both to minimize storage costs and to comply with data governance requirements.

Example of transaction types

Imagine the following transaction history for a branch on a dataset, starting from the oldest:

1. **SNAPSHOT** contains files **A** and **B**
2. **APPEND** adds file **C**
3. **UPDATE** modifies file **A** to have different contents, **A'**
4. **DELETE** removes file **B**

At this point, the current dataset view would contain **A'** and **C**. If we added a fifth **SNAPSHOT** transaction containing file **D**, the current dataset view would then only contain **D** (since **SNAPSHOT** transactions begin new views) and the first four transactions would be in an old view.

Retention

API Reference ↗

Since a **DELETE** transaction does not actually remove older data from the [backing filesystem](#), you can use [Retention policies](#) to remove data in transactions no longer needed.

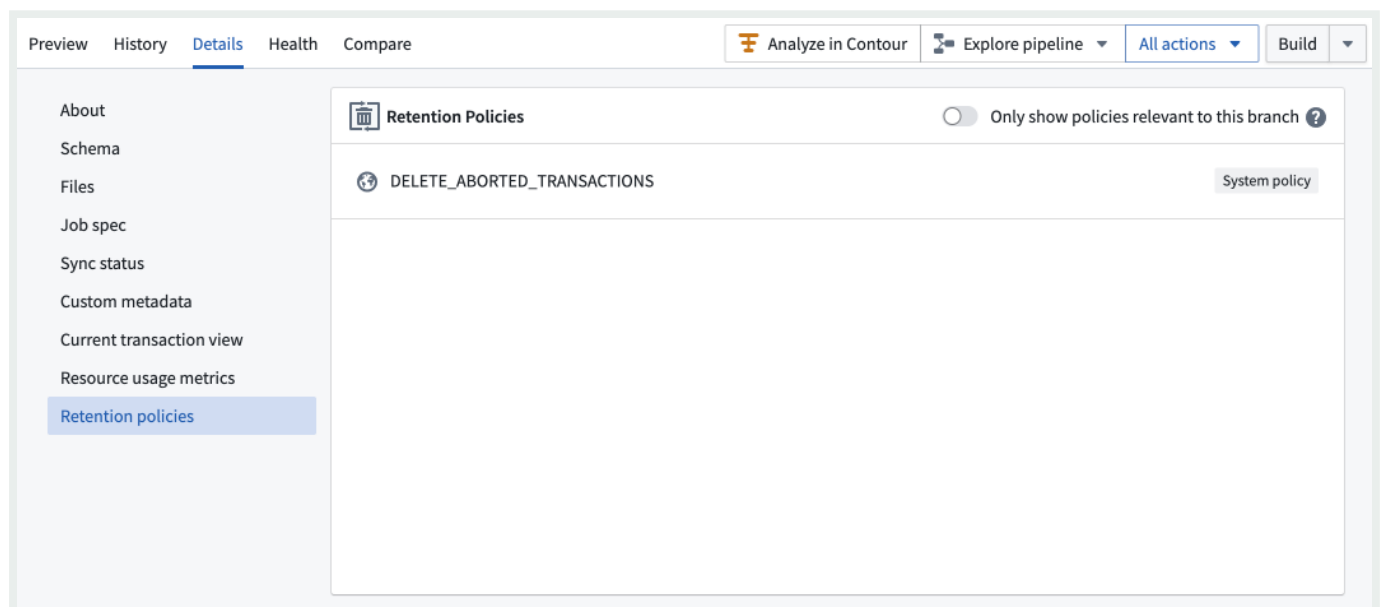
Send feedback

View retention policies for a dataset [Beta]

Beta

Viewing dataset retention policies is in the [beta](#) phase of development and may not be available on your enrollment. Functionality may change during active development. Contact Palantir Support to request access to dataset retention policies.

To view the retention policies that currently apply to a given dataset, navigate to the [dataset details page](#).



Branches

While dataset transactions are designed to enable a dataset's contents to change over time, additional functionality is needed to enable *collaboration* —having multiple users experiment with changes to a dataset simultaneously. **Branches** in a dataset are designed to enable these workflows.

To learn about branching in Foundry, both on individual datasets and across entire pipelines, see the [API Reference](#) or the [Branching](#) concept page.

Send feedback

Dataset views

A dataset view represents the effective file contents of a dataset for a branch at a point in time. Historical views are analogous to historical versions of a dataset. To calculate which files are in a view:

1. Start with an empty set of files.
2. The view at a given time begins at the latest `SNAPSHOT` transaction before that point in time. If there is no `SNAPSHOT` transaction present, then take the earliest transaction for the dataset instead.
3. For the first transaction in the view, and for each subsequent transaction, do the following:
 - For a `SNAPSHOT` (which would only be the first transaction) or `APPEND` transaction, add all the transaction's files to the set.
 - For an `UPDATE` transaction, add all the transaction's files to the set and replace existing files.
 - For a `DELETE` transaction, remove all files in the transaction from the set.

The resulting set of files constitutes a dataset view.

Because a new view only begins at a `SNAPSHOT` transaction, the number of views in a dataset's history is equal to the number of `SNAPSHOT` transactions it contains. If a dataset exclusively contains `SNAPSHOT` transactions, this means the number of views is equal to the number of transactions. In the case of an incremental dataset, `APPEND`, `UPDATE`, and `DELETE` transactions extend the current view rather than beginning a new one, so the number of views remains equal to the number of `SNAPSHOT` transactions.

A view may constitute transactions from multiple branches. For example, given an incremental dataset starting with a `SNAPSHOT` and some `APPEND` transactions on the `master` branch, these transactions will form the start of the dataset view on the branch if subsequent transactions on the branch are also `APPEND` (or, strictly, *not* `SNAPSHOT`) transactions.

 [API Reference ↗](#)

A schema is metadata on a *dataset view* that defines how the files will be interpreted. This includes how files in the view should be parsed, and how the columns or

[Send feedback](#)

fields in the files are named or typed. The most common schemas in Foundry are *tabular*—they describe the columns in a dataset, including their names and [field types](#).

Note that there is no guarantee that the files in a dataset actually conform to the specified schema. For example, it is possible to apply a Parquet schema to a dataset that contains CSV files. In this case, client applications attempting to read the contents of the dataset would encounter errors caused by the fact that some files do not conform to the schema.

Because schemas are stored on a *dataset view*, schemas can change over time. This is useful because the contents of a dataset may structurally change over time. For example, a new transaction may introduce a new column to a tabular dataset or change the type of a field.

In Foundry, you can view the schema on any dataset in the [Dataset Preview](#) application by navigating to the **Details** tab and selecting **Schema**.

Supported field types

Below is a list of field types available for datasets:

- `BOOLEAN`
- `BYTE`
- `SHORT`
- `INTEGER`
- `LONG`
- `FLOAT`
- `DOUBLE`
- `DECIMAL`
- `STRING`
- `MAP`
- `ARRAY`
- `STRUCT`
- `BINARY`

API Reference ↗

Send feedback

Some field types require additional parameters:

- `DECIMAL` requires `precision` and `scale`. If you are unsure of what to set for these parameters, a good default is `precision: 38` and `scale: 18`. `38` is the highest possible precision value.
- `MAP` requires `mapKeyType` and `mapValueType`, which are both field types.
- `ARRAY` requires `arraySubType`, a field type.
- `STRUCT` requires `subSchemas`, a list of field types.

For more information on these field types indicated above, including descriptions and examples, see [Spark data types documentation ↗](#).

Schema options

In the **Schema** section of the **Details** tab in Dataset Preview, you can add optional parsing configurations for CSV files in the `options` block at the bottom of the schema metadata. See the [CSV parsing](#) documentation for more information.

File formats

Schemas include information about the underlying storage format of the files in the dataset. The three most widely-used formats are:

- Parquet
- Avro
- Text

The Text file format can be used to represent a wide range of file types, including a variety of CSV formats or JSON files. Additional information about how Text should be parsed is stored in a schema field called `customMetadata`.

In practice, for non-tabular formats such as JSON or XML, we recommend storing files in an unstructured (schema-less) dataset and applying a schema in a downstream data transformation as described in the [schema inference documentation](#).

API Reference ↗

Backing timesystem

Send feedback

The files tracked within a dataset are not stored in Foundry itself. Instead, a mapping is maintained between a file's *logical path* in Foundry and its *physical path* in a backing file system. The backing filesystem for Foundry is specified by a base directory in a [Hadoop FileSystem ↗](#). This can be a self-hosted HDFS cluster, but is more commonly configured using a cloud storage provider such as Amazon S3. All dataset files are stored in a folder hierarchy below the backing file system's base directory.

PREVIOUS

← [Application reference](#)

NEXT

[Streams](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Streams

Similar to a [dataset](#), a stream is a representation of data from when it lands in Foundry through when it is processed by a downstream system. A stream is a wrapper around a collection of rows which are stored by both a persistent “hot buffer” and “cold storage” backed by a file system. The benefit of using Foundry streams is they provide the same primitives as a Foundry dataset (branching, version control, permission management, schema management, etc) while also providing a low latency view of the data.

Streams are inherently tabular and, therefore, inherently structured. They are stored in open source formats such as [Avro](#), along with metadata about the columns themselves. This metadata is stored alongside the stream as a schema.

Stream storage

Hot buffer

As records flow into a Foundry stream, they are stored in a hot buffer that is available in low latency for all downstream applications that support reading streams. This hot buffer is critical for enabling low latency transforms and availability. It provides at-least-once semantics for data ingestion and optional exactly-once semantics for data processing in platform.

Cold buffer

[API Reference](#)

Any data from within a Foundry stream is transferred from the hot buffer to the cold storage every 15 minutes. We call this process "archiving", and it makes the stream a standard Foundry dataset. This means that any Foundry application can operate on

Send feedback

streaming data, even if it doesn't process data in real-time from the hot buffer. The [dataset view](#) of a Foundry stream behaves exactly as a standard Foundry dataset in the platform.

Stream processing

Read data from a stream

Foundry products with low latency enabled are able to read a hybrid view of the data. By reading data from both the hot and cold storage layers, products can provide a complete view of the data. This view gives products access to the low latency records still in hot storage and older data that has been transferred to cold storage. In this way, a Foundry stream can have the benefits of both the low latency of hot storage and the lower storage costs of cold storage.

Transactions

Unlike standard Foundry [datasets](#), streams do not have transaction boundaries inherent in the stream themselves. Instead, each row is treated as its own transaction, and state is tracked on a per row basis. This allows a stream to be read at a granular level so Foundry can support push-based transformations without requiring batching or polling.

Stream types

You can configure stream types for each of your streams based on its throughput needs. These stream type settings apply to how streams write data to the [hot buffer](#) storage mentioned above. You should only need to modify the stream type settings if stream metrics indicate the stream is being bottlenecked when writing to the hot buffer storage. Latency and throughput are tradeoffs, so only set high throughput/compressed stream types after inspecting stream metrics. We support the following stream configurations:

- 1 **High Throughput:** This is best for streams that send large amounts of data every second.

[API Reference ↗](#) Stream type might introduce some non-zero latency at the expense of a higher throughput. Therefore, before you enable it you should inspect the average batch size is equal to the max batch size or if the job fails due to producer batches expiring, you might need to enable the high throughput setting.

Send feedback

2. **Compressed:** Enabling this configuration compresses message batches when producing data to the hot storage buffer. Compression helps reduce the size of the data being sent, resulting in lower network usage and storage, at the cost of some additional CPU usage for compression and decompression. We only recommend enabling this stream type if your stream contains a high volume of repetitive strings and is experiencing poor network bandwidth symptoms like non-zero lag, lower than expected throughput, or dropped records.

You can set stream types when creating a new stream on the **Define** page. You can also update stream types in stream settings for an existing stream. For this, navigate to your streaming dataset in Foundry and select **Details** in the toolbar. Then, go to **Stream Settings**. You can change the stream type and enable/disable compression here.

Partitions

To maintain high throughput, Foundry breaks the input stream into multiple partitions for parallel processing. When creating a stream, you can control the number of partitions we create through the throughput slider. Note that although the data is partitioned, all reads and writes to the stream operate as if there is a single partition. This behavior provides design transparency to consumers and producers of Foundry streams.

Each additional partition for a given stream increases the max throughput the stream can process. A good heuristic is that each partition increases the throughput by approximately 5mb/s.

Supported field types

Foundry streams support the same data types as a Foundry dataset, including:

- **BOOLEAN**
- **BYTE**
- **SHORT**
- **INTEGER**

API Reference ↗

- **FLOAT**
- **DOUBLE**

Send feedback

- DECIMAL
- STRING
- MAP
- ARRAY
- STRUCT
- BINARY
- DATE
- TIMESTAMP

Streaming Jobs

All streaming jobs are represented internally as [job graphs](#), which provide a visual representation of your streaming pipeline. As data is processed, it flows through the job graph according to the directed edges until reaching a data sink.

Checkpointing

Foundry streaming provides fault tolerance while processing data by storing both the active state and current processing location in a checkpoint.

Checkpoints are periodically produced by data sources and flow through the job graph alongside the data from the source. Once a checkpoint has reached all data sinks at the end of the job graph, all rows emitted before that checkpoint by the source must have also reached the sink.

Checkpoints allow a streaming job to be restarted from the point of the latest checkpoint, rather than reprocessing already-seen data. Checkpoints store the state of each operator in your job graph, plus the last-processed data point in the stream. On your streaming job's **Job Details** page, you can see the status, size, and duration of your stream's last few checkpoints in real-time.

Streaming consistency guarantees

[API Reference ↗](#)

Streaming in Foundry operates with two consistency guarantees: `AT_MOST_ONCE` and `EXACTLY_ONCE`.

[Send feedback](#)

AT_LEAST_ONCE semantics

AT_LEAST_ONCE semantics guarantee that a message will be delivered downstream at least once, but a message may be delivered multiple times in case of checkpointing failures or retries. This means that duplicates may occur, and the consuming application should be designed to handle or tolerate duplicate messages.

Benefits of **AT_LEAST_ONCE** semantics

- Ensures that no messages are lost, providing a high level of message durability.
- Generally offers lower latency compared to **EXACTLY_ONCE** semantics, as messages can be delivered without blocking on the bookkeeping of records.

Drawbacks of **AT_LEAST_ONCE** semantics

- Requires the downstream consuming application to be able to handle duplicate messages.

EXACTLY_ONCE semantics

EXACTLY_ONCE semantics guarantee that each message will be delivered and processed exactly once, ensuring that there are no duplicates or missing messages. This is the strongest level of message delivery guarantee and can greatly simplify the design of the consuming application.

Benefits of **EXACTLY_ONCE** semantics

- Ensures that no messages are lost, providing a high level of message durability.
- Eliminates the need for the consuming application to handle duplicate messages or implement idempotent processing.
- Ensures consistency in processing results, as each message is processed exactly once.

Drawbacks of **EXACTLY_ONCE** semantics

[API Reference ↗](#)

- Typically introduces higher latency compared to **AT_LEAST_ONCE** semantics due to the additional coordination and tracking required to ensure that messages are processed exactly once.

[Send feedback](#)

duplicated. Foundry streaming solves this problem through checkpointing.

Latency trade-offs

Choosing between `AT_LEAST_ONCE` and `EXACTLY_ONCE` semantics often involves a trade-off between latency and processing complexity. `AT_LEAST_ONCE` semantics generally provide lower latency because they do not require complex coordination or tracking mechanisms, but they place more responsibility on the consuming application to handle duplicates and maintain consistency. When `EXACTLY_ONCE` is enabled, records are only visible downstream after each checkpoint has completed (default is two seconds). Notably, the records are still being processed at streaming speeds but only show up downstream when "finalized".

On the other hand, `EXACTLY_ONCE` semantics provide stronger guarantees and can simplify the design of the consuming application by ensuring that each message is processed exactly once. However, this guarantee comes at the cost of higher latency due to the additional overhead required.

Streaming sources in Foundry currently only support `AT_LEAST_ONCE` semantics for extracts and exports. Streaming pipelines *do* support both `AT_LEAST_ONCE` and `EXACTLY_ONCE` semantics, and this is configurable under the **Build settings** section of Pipeline Builder.

[API Reference ↗](#)

[Send feedback](#)

Legend (Job grouping)

Output Stream

Not running

Build settings

Apply

Custom job grouping

0

Default job grouping

1

Single jobs

☐

Output Stream

Advanced configuration

Data consistency guarantee

Select a value

At Least Once

Exactly Once

Number of Output Partitions

PREVIOUS

← Datasets

Media sets (unstructured data)

NEXT →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

—Cookies Settings

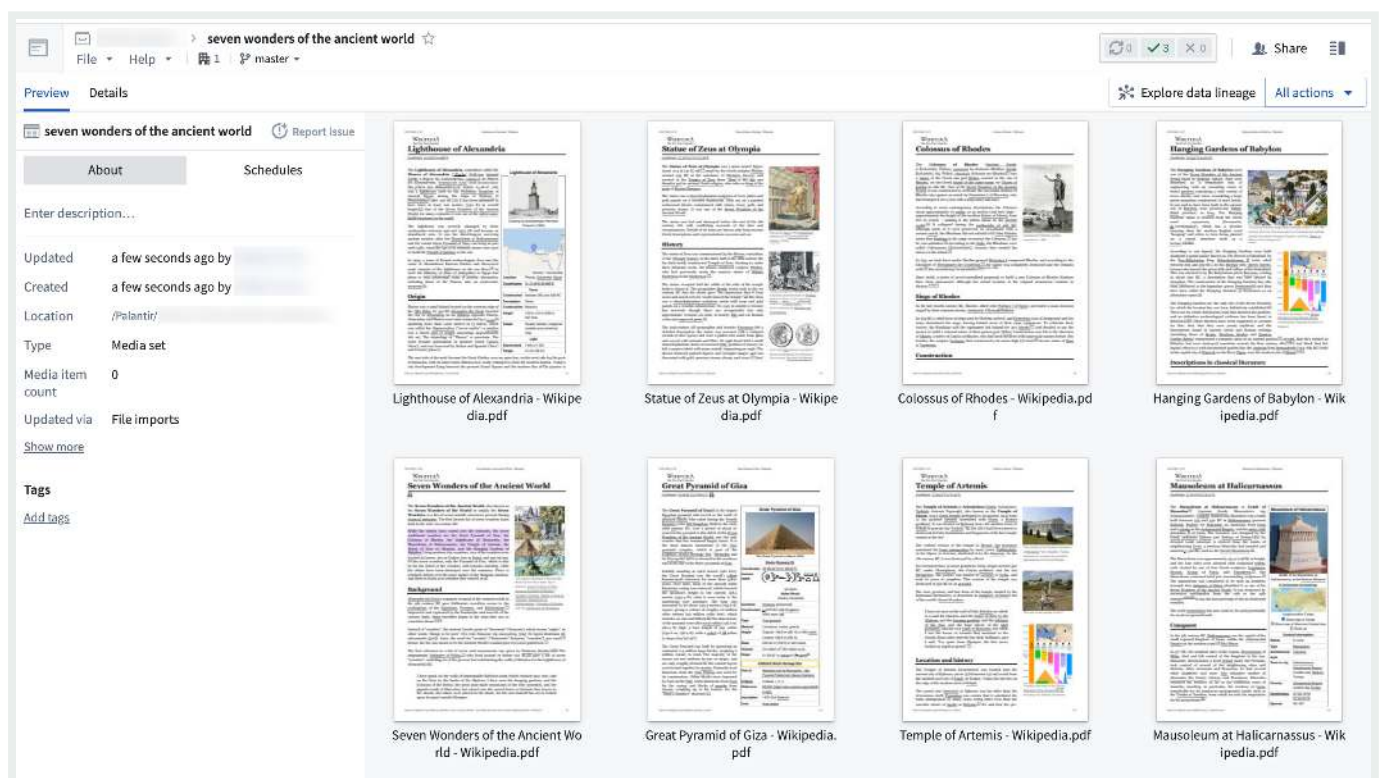
API Reference ↗

Send feedback

Data connectivity & integration / Core concepts / Media sets (unstructured data)

Media sets (unstructured data)

A **media set** is a collection of media files with a common schema, for example, files of the same format. Media sets are designed to work with high-scale, unstructured data and enable the processing of media items such as audio, imagery, video, and documents. Media sets enable access to flexible storage, compute optimizations, and schema-specific transformations to enhance media workflows and pipelines.



Example media set workflows include:

- Enabling content analysis by extracting text from PDFs with Pipeline Builder
- Enabling spatial analysis with raster tiling (TIFF, NITF) in the Map application
- Enabling processing of medical imaging files (DICOM format) with Pipeline Builder

API Reference ↗

Send feedback

To find out more about media sets and how to work with media in Foundry, **visit the [media set documentation](#)**.

PREVIOUS
← Streams

NEXT
Branching →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

API Reference ↗

Send feedback

Branching

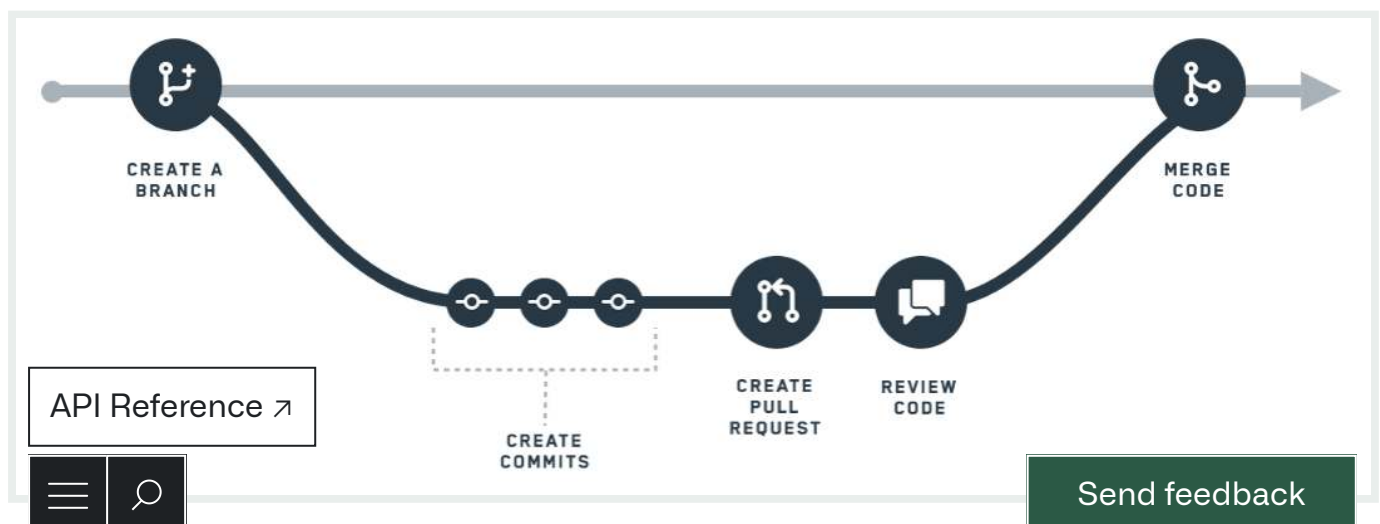
Software developers typically use version control systems to coordinate work in a codebase. This enables multiple engineers to contribute to the same code safely.

Within Foundry, we think about data the way software developers think about code: you need a way to allow many people to make changes and interact with the same data without interfering with someone else's work. We took best practices from software development and applied them to writing data pipelines, harnessing a common feature of version control systems called **branching**.

At a high level, **branching** allows you to take a fork in the road and work on data in your own branch. Then, once you're happy with your changes, you can push your changes out of your branch back to the main road.

Branching workflow

How to use a branching workflow to make changes to data pipeline code in Foundry:



1. **Create a branch.** In Foundry, the **master branch** refers to the primary data pipeline. When you want to work on your own changes, you create your own branch, which creates an environment for you to experiment and test out ideas without worrying about affecting the **master** branch.
2. **Create commits.** Within your branch, you can make changes to data transformations. Changes, which include additions and removals, are called **commits**. Your commits are tracked so that there's a clear history of all the changes you've done on your branch.
3. **Create a pull request.** Once you're done working in isolation, you'll want to push your changes back to the master branch. To start this process, you create a **pull request**. A pull request signals to your team that you've made changes you'd like them to review and validate for the **master** data pipeline.
4. **Review code.** After you create a pull request, your team will have the chance to review your commits. Every organization will have a different process or team for reviewing pull requests.

Foundry branching implements an industry-standard Git-like version control paradigm. As such, the Code Repositories application was designed to have one active developer for each individual branch on a file. If other users are working on your **personal branch**, your changes may be overwritten. To avoid this, we strongly recommend having each person work in a separate branch.

The rest of this page describes the technical details of how branches work in Foundry. If you are interested in learning how to use branching in practice, follow the tutorial for creating a simple batch pipeline in [Pipeline Builder](#) or [Code Repositories](#).

Branching technical details

In Foundry, branching for data is implemented using features at two different levels: branching within an [individual dataset](#), and branching when [building datasets](#).

Dataset branches

A [page on datasets](#), datasets have version history in the form of a series of [transactions](#). Actually, dataset branches are similar to branches in Git—*datasets*, *branches*, and *transactions* are analogous to Git *repositories*, *branches*, and *commits*.

API Reference ↗

Send feedback

Similar to branches in Git, dataset branches are simply a pointer to the most recent transaction on that branch. As a result, a branch can be interpreted as a linear sequence of transactions ordered by commit time. When a dataset is changed on a branch by committing a transaction, the transactions and views of all other branches are unaltered. Unlike Git, there is no support for merging dataset branches.

Each dataset branch has exactly one **parent branch**, unless it is a **root branch**. In practice, most datasets in Foundry have a single root branch called `master`, and child branches are created from this single root branch.

Below are all the supported operations related to dataset branches:

- **Creating a new branch.** A root branch can be created with no parent branch and no transactions.
- **Creating a child branch.** A child branch can be created from another branch, or from any transaction. The new branch points to the same transaction as the parent branch. Subsequently, transactions can be started on both the parent and child branches, and their transaction pointers will move independently.
- **Starting a new transaction on a branch.** Starts a transaction and moves the branch pointer to the new transaction.
- **Deleting a branch.** Deletes the branch pointer, but not any of the transactions on the branch. See the note about parent branches in the guarantees below.

Dataset branch guarantees

Foundry maintains the following guarantees for dataset branches:

- **One open transaction per branch.** Every branch can have at most one open (as in, opened and neither committed or aborted) transaction, and this transaction is always the latest transaction on the branch. A dataset with one branch can thus have at most one open transaction. If a branch points to an open transaction, then a new branch can be created off of this branch, but no further transactions can be started until the transaction is closed.
- **On every branch, transactions are ordered by start and commit time.** This guarantee is [API Reference ↗](#) of previous constraint. Note, however, that no guarantee is given regarding the temporal order of transactions on different branches.
- **Every non-root branch has exactly one parent branch.** In case "i [Send feedback](#) s
—branches with child branches—are deleted, the child branches are re-parented under

the deleted branch's parent (or no parent if it was a root branch). Note that no transactions are rearranged in this process; re-parenting merely changes the branch ancestry record.

Branches in builds

Dataset branching provides the foundational semantics for *versioning data*. Foundry combines dataset branching with Git branching to support branching over logic and data simultaneously, to enable data engineers to safely experiment with changes to data pipelines before introducing them to production.

Tying branches of logic in Git to branches of data in datasets is done through Foundry's [build](#) system. Every build runs on a user-specified branch, and the jobs within the build modify datasets on that branch only. As a result, branches in builds provide a way to isolate the changes of different users from each other.

Foundry builds perform two main functions involving branches, as described below:

- First, they [compile the build graph](#) by collecting job specifications, or JobSpecs, from appropriate branches.
- Second, they [resolve job inputs and outputs](#) with respect to the user-defined build branch and a sequence of *fallback branches*.

Job graph compilation

When you author data transformations in Foundry in a branch of a [Code Repository](#), committing your code publishes JobSpecs to the appropriate branch of the build system. When a build is run on that branch, the build traverses the JobSpecs and their dependencies on your branch to determine which logic should be executed.

A build usually specifies *branch fallbacks* that govern from which branch it retrieves JobSpecs in case no JobSpec is set on the build branch. For example, if you run a build on `develop` with a fallback chain of `develop --> master`, if there is no published JobSpec for a given output dataset, the JobSpec will be read from the `master` branch instead.

[API Reference ↗](#)

Dataset icon color provides information about JobSpecs and branches. If the icon is gray, this indicates that no JobSpec exists on the master branch. If the icon is blue, a JobSpec is defined on the master branch.

[Send feedback](#)

Input and output resolution

For all datasets designated as *outputs* by the JobSpecs in a build, a transaction is opened on the build branch. If this branch does not exist on a dataset, then it is created off the first branch in the fallback chain, or as a root branch if no fallback branch exists. Job inputs are read from the build branch if possible, but otherwise use the first existing branch in the fallback chain.

Example: Building on branches

To understand how branches work in builds, let's step through an example workflow:

1. Suppose that datasets A, B, and C exist on the `master` branch.
2. A data engineer creates a branch called `feature` in their Code Repository. This creates a branch in the underlying Git repository.
3. The data engineer modifies the code that produces datasets B and C. When the data engineer commits their code, checks in the repository publish JobSpecs to both datasets on the `feature` branch.
4. The data engineer initiates a build on the `feature` branch. As the `feature` branch was created off the `master` branch in the Git repository, the fallback chain for this build is `feature --> master`. The JobSpecs published in step (3) is read, and two jobs corresponding to the user's code are started. Transactions are opened on both datasets B and C on the `feature` branch. The two jobs execute serially as follows:
 1. *Job 1*: The `master` branch of dataset A is used as input because of branch fallbacks. Computation produces new data, which is written to dataset B in the currently open transaction. The transaction is committed.
 2. *Job 2*: Next, this job begins. The `feature` branch of dataset B is used as input. Computation produces new data, which is written to dataset C in the currently open transaction. The transaction is committed.
5. Finally, because all jobs in the build succeeded, the build is successful.

After modifying code, committing, and running a build, the data engineer has produced new data on the `feature` branch of datasets B and C. Notice that dataset A was completely unaffected by this process, and the `master` branch of datasets B and C is also unaltered.

[API Reference ↗](#)

Build branch guarantees

[Send feedback](#)

Builds in Foundry provide the following guarantees for branches:

- Build resolution only succeeds if the specified branch fallback sequence is compatible with the branch ancestries in the involved datasets.
- A build never modifies any dataset branches other than the build branch.
- A build never creates branches on input datasets.

PREVIOUS
← [Media sets \(unstructured data\)](#)

NEXT
[Builds](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Builds

A **Build** is the mechanism used to compute new versions of [datasets](#) in Foundry. Builds provide orchestration and coordination of computation, ensure the appropriate input data is read, and write output data to the appropriate location.

A build is composed of **jobs**, each of which is a unit of work that is defined by shared logic and computes one or more output datasets. Note that if a job defines multiple output datasets, they will always update together and it is not possible to build only some of the datasets without running the full job. A job specification, or **JobSpec**, is a definition of how a job should be constructed. JobSpecs are published when changes are made to data transformation logic in Foundry, such as when a data engineer commits new transforms code in a [Code Repository](#).

Running a build results in a one-time computation of a set of output datasets. To keep data flowing through the system, [schedules](#) are used to run builds over time.

You can explore builds in Foundry using the [Builds application](#).

Jobs and JobSpecs

A *job* encapsulates the computation of a new version of one or more *output datasets* from the data of a set of input datasets. A *JobSpec* defines how to construct a job by detailing input dataset dependencies and the logic that should be executed as part of the job.

Input dataset dependencies are declared as a set of *InputSpecs*, each of which specifies a p
API Reference ↗ set. InputSpecs specify a subset of data to read from a dataset in terms
of its views.

[Send feedback](#)

There are many types of logic that can be represented as a job in Foundry, including but not limited to:

- A Data Connection [Sync](#) defines how data should be read from an external data source.
- A transform written in a [Code Repository](#) allows you to write code that transforms datasets.
- [Health checks](#) are defined as jobs generated on a dataset to validate characteristics of the dataset.
- [Analytical applications](#) support defining logic that transforms datasets.
- An [export](#) defines how input data should be sent outside of Foundry.

Job states

At any given time, a job is always in one of the following states:

- **WAITING** : The initial state of a job; the job is waiting for its dependent jobs to complete and has not been invoked.
- **RUN_PENDING** : The job is waiting to run, but its execution environment has not yet confirmed the status.
- **RUNNING** : The job has been invoked and is currently being computed.
- **ABORT_PENDING** : The job has been aborted, but its execution environment has yet to confirm the aborted status.
- **ABORTED** : The job was aborted either upon user request, or as a result of a dependent job failing.
- **FAILED** : The job was invoked, but the computation failed.
- **COMPLETED** : The job was invoked, and the computation finished successfully.

Build lifecycle

When a build is run, several steps are performed to validate the submitted build, ensure data consistency, and only run the jobs necessary to produce new outputs.

[API Reference ↗](#)

Send feedback

As a first step, a build:

- Detects cycles in the specified input datasets and fails the build if there are cycles present.
- Validates all input datasets exist and identifies the appropriate schema for each input dataset.
- Opens new [transactions](#) on each output dataset to ensure that only the active build can write to the output dataset. This is known as *build locking*.
- Detects if other builds are in progress that would change the inputs into the build. If so, the build may be *queued* and wait for the other build to complete.

Job execution

Once the above steps have completed, the jobs within a build are executed. Jobs that do not depend on each other are run in parallel. As jobs proceed through the [job states](#), the state of the overall build is updated accordingly:

- If a job in a build fails, all directly-dependent jobs and transactions on output datasets within that build will be terminated. Optionally, a build can be configured to abort all non-dependent jobs at the same time.
- If all jobs in the build are completed, then the build is considered completed.

Note that if a job in a build fails, previously completed jobs may still have written data to their output datasets.

Staleness

An output dataset is considered *fresh* if the [build resolution](#) step determines that input datasets and the logic specified within the JobSpec have not changed since the last time the output dataset was built. If an output dataset is fresh, it will not be recomputed in subsequent builds.

To override the build system's default staleness behavior, you can run a **force build**, which recomputes all datasets as part of the build, regardless of whether they are already up-to-date.

API Reference ↗

Branching

Send feedback

Builds in Foundry implement **branching** to support collaboration workflows on data pipelines. To learn more about branching:

- Refer to the [branching overview](#) for a high-level explanation.
- Refer to the [branching in builds](#) section for details about how branching works in builds.

Live logs

Live logs provide real-time visibility into running jobs, allowing you to monitor how jobs are progressing and inspect long-running tasks such as streams or compute modules.

The screenshot displays the Foundry Builds application interface. On the left, the 'Build info' panel shows the build is 'Running' with a duration of 10m 53s and an estimated time of 13m 28s. The 'Build schedule' section indicates the build runs 'Every 5 minutes (Europe/P...)'. The main area shows a 'Logs' view with a 'Wrap lines' toggle. The log content includes timestamps and task details, such as 'Starting task 160.0 in stage 85.0 (TID 6507)' and 'Reading File path:'. At the bottom, the 'Build progress' section shows a Gantt chart and a legend for job statuses: Queued, Waiting, Running, Succeeded, Failed, and Canceled.

You can access live logs through the Builds application. Select the **View live** button in the top right corner of the log viewer when viewing a job to start generating.

[API Reference ↗](#)

[Send feedback](#)

Builds > Build of: **issues_inbox_view**

Explore lineage Actions Cancel build

Build info

Status Running
Duration 1m 49s
Estimated 13m 47s
Started Today at 6:08 PM
Ended --
Started by Build schedule
Progress 0 of 1 job succeeded
Build ID

Build schedule

Schedule

WHEN TO BUILD

This schedule will run:

Every 5 minutes (Europe/P...

RECENT RUNS

Last modified 6 months ago by

Metrics Schedule

Logs
Wrap lines
issues_inbox_view
Download logs
View live

2024-08-20 18:09:44 2024-09-02 18:19:44 Filter logs Columns

	level	time	message	params	unsafeParams	origin
6	INFO	2024-09-02 18:08:59.992	Getting Spark contex...	{ "jobRid": "ri.foun...	{}	com.palar
7	INFO	2024-09-02 18:08:59.992	Starting transforms _	{ "appName": "python...	{}	com.palar
8	INFO	2024-09-02 18:09:00.069	Creating service	{ "channel": "foundr...	{}	com.palar
9	INFO	2024-09-02 18:09:00.073	Creating Dialogue ch...	{}	{}	com.palar
10	INFO	2024-09-02 18:09:00.076	Reloaded channel '{}...	{ "channel": "foundr...	{ "uris": ["https:/...	com.palar
11	INFO	2024-09-02 18:09:00.088	Creating service	{ "channel": "foundr...	{}	com.palar
12	INFO	2024-09-02 18:09:00.088	Creating Dialogue ch...	{}	{}	com.palar
13	INFO	2024-09-02 18:09:00.090	Reloaded channel '{}...	{ "channel": "foundr...	{ "uris": ["https:/...	com.palar
14	INFO	2024-09-02 18:09:00.102	Creating service	{ "channel": "foundr...	{}	com.palar
15	INFO	2024-09-02 18:09:00.182	Creating Dialogue ch...	{}	{}	com.palar
16	INFO	2024-09-02 18:09:00.183	Reloaded channel '{}...	{ "channel": "foundr...	{ "uris": ["https:/...	com.palar
17	INFO	2024-09-02 18:09:00.125	Creating service	{ "channel": "foundr...	{}	com.palar
18	INFO	2024-09-02 18:09:00.126	Creating Dialogue ch...	{}	{}	com.palar
19	INFO	2024-09-02 18:09:00.127	Reloaded channel '{}...	{ "channel": "foundr...	{ "uris": ["https:/...	com.palar
20	INFO	2024-09-02 18:09:00.144	Self-detected	{ "version": "5.112...	{}	com.palar
21	INFO	2024-09-02 18:09:00.194	Found transforms con...	{}	{}	com.palar
22	INFO	2024-09-02 18:09:00.208	Found transforms lan...	{}	{}	com.palar
23	INFO	2024-09-02 18:09:00.248	Using dataset as des...	{ "datasetRid": "ri...	{}	com.palar

Collapse

Build progress

Gantt chart
Job status
Dataset path...

Queued Waiting Running Succeeded Failed Canceled

Job status Progress details Critical path

A key feature of live logs is built-in color coding by log level, making it easier to identify and prioritize warnings and errors:

API Reference ↗

Send feedback

```
INFO [2024-09-02T17:44:18.930842Z]
INFO [2024-09-02T17:44:18.931099Z]
INFO [2024-09-02T17:44:18.931228Z]

DEBUG [2024-09-02T17:44:18.931310Z]

INFO [2024-09-02T17:44:18.931387Z]
INFO [2024-09-02T17:44:18.931556Z]

INFO [2024-09-02T17:44:18.932781Z]
INFO [2024-09-02T17:44:18.933055Z]

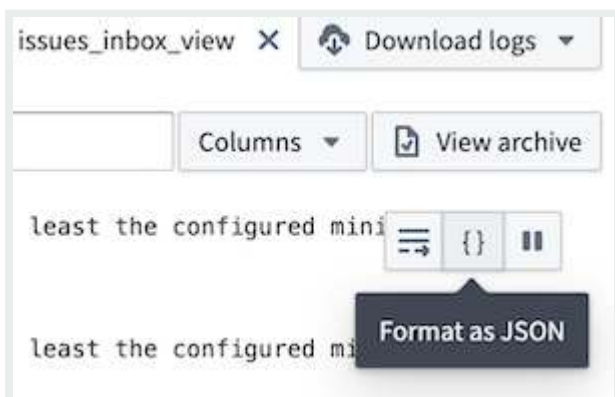
INFO [2024-09-02T17:44:18.934046Z]

WARN [2024-09-02T17:44:18.934390Z]

INFO [2024-09-02T17:44:18.935234Z]
```

- **Info:** Blue
- **Fatal/Error:** Red
- **Warn:** Orange
- **Debug/Trace:** Gray

Additionally, safe parameters and parameters are visible as a JSON block, providing a structured and readable format for your data.



You can pause the log feed at any point by selecting **Pause** from the top right of the interface, and resume from the same location.

[Send feedback](#)



Note that the time range selection does not apply to live logs, since they are streamed in real-time from the job.

i Once enabled, a ten-second delay may occur before the live logs are visible in the interface.

PREVIOUS
← Branching

NEXT
Schedules →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

—[Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Schedules

Schedules are used to run [builds](#) on a recurring basis to keep data flowing through Foundry consistently. In a schedule, the **trigger** defines the condition that must be satisfied for the associated build to be run.

When a trigger is satisfied and a build is performed, we say that the schedule is **run**. If a schedule is triggered while the previous run is still in action, then it will remain triggered and run only after the previous schedule is finished.

The run history provides a record of when a schedule was run and information about the task that was performed on each run. A schedule run can be one of a few types:

- **Succeeded.** The run successfully initiated a build. Note that a successful run only indicates that the build was successfully started. The build itself may still be running, or may have failed.
- **Ignored.** The run was attempted, but a build was not created. An ignored run likely indicates that everything is up-to-date and there was no work to do. See [staleness](#) for more details.
- **Failed.** The schedule failed to run.

To learn more about schedules, refer to the following resources:

- Learn to [create a schedule](#).
- Learn about [scheduling best practices](#).
- Explore the [available trigger types](#).

[API Reference ↗](#)[Find a schedule](#)[Send feedback](#)

Schedules can be edited, managed, and updated in the schedule sidebar of the [Data lineage](#) application. Workflows around finding schedules can be conducted in the **Build schedules** application available from your Apps sidebar. Queries that you can run include but are not limited to the following:

- “Find paused schedules by a given user”
- “Find schedules scoped to a certain project, filter by name, and sort by latest run”
- “Find schedules with 'TESTING_PROJECT_1' in their name”
- “Find paused schedules”

You can use the following search criteria:

- **Files:** Find schedules by the datasets or other assets in Foundry that they build. If no branch is specified, the default branch will be used.
- **User:** Find schedules last updated by the user(s) selected.
- **Projects:** Find project-scoped schedules scoped to a specific project. User-scoped schedules are currently not supported in this parameter.

When arriving at the page you will initially see your own schedules. You can then filter schedules by name, pause or sort them. Search parameters are stored in the page link, allowing you to bookmark a page or share the link with other users.

The list of schedules can be further refined by **Schedule name** (if one is present on the schedule) and **Pause status**. It can also be sorted by **Name**, **Creation date**, **Last run date**, or **Last update date** of the schedule.

Pause a schedule

A schedule can be [paused](#) to temporarily prevent it from running.

When a schedule is paused, its trigger state is reset and all observed events are forgotten. While a schedule is paused, it cannot be triggered and will ignore all observed events.

A paused schedule can be [resumed](#) to allow it to begin running again.

API Reference ↗

Project scope

Send feedback

The datasets a schedule has permission to build is determined by whether a schedule is saved using the user's permissions to datasets or whether it is saved using the set of Projects containing the datasets being built. The former (user mode) is prone to unexpected changes if the permissions of the user change, since the schedule runs as if the user were running the build. The latter (Project-scoped mode) is more consistent, since the schedule is run independently of the user's permissions and only changes if the set of Projects the schedule is scoped to changes.

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

API Reference ↗

Send feedback

Health checks

Once data is synced into Foundry, has been transformed in a pipeline, and is running regularly using [schedules](#), **health checks** can be used to validate data quality throughout the pipeline. This is necessary to ensure that the data flowing through the pipeline remains reliable and maintains expected structure over time. Health checks are commonly used in [pipeline maintenance](#).

Several types of health checks are available in Foundry:

- *Job-level checks* validate that the job corresponding to an output dataset is completing successfully.
- *Build-level checks* validate that builds are completing successfully, and are completing within an expected duration.
- *Freshness checks* validate that data is being kept up-to-date.

To learn more, refer to these resources:

- Explore the [Health checks](#) service to learn how to define health checks.
- Read the [checks reference](#) to learn about the range of available checks.
- Learn about which [health checks are recommended](#).

PREVIOUS
← Schedules

NEXT
Iceberg tables →

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Iceberg tables [Beta]

Beta

Iceberg table support is in the [beta](#) phase of development and may not be available on your enrollment. Functionality may change during active development. Contact Palantir Support to request enabling Iceberg tables.

[Apache Iceberg](#) is a widely adopted open-source table format and is available in Foundry via **Iceberg tables** as an alternative resource type for representing tabular data.

Foundry offers Iceberg both as managed tables and as [virtual tables](#) via an external catalog or storage provider.

What is Iceberg?

[Apache Iceberg](#) is an open table format that has gained significant traction in the data and analytics community due to benefits around scalability, performance, and broad ecosystem support. In particular, the wide adoption of the Iceberg format specification enables a broad array of integrations and interoperability across modern data ecosystems.

The Apache Iceberg project includes the [Apache Iceberg table format specification](#), as well as a set of engine connectors that support the Iceberg specification, such as [Spark](#), [Flink](#) and more.

Besides the Apache Iceberg project, there is a growing ecosystem of connectors and engines that support the Iceberg format:



Send feedback

- [PyIceberg](#) provides a gateway into the Python ecosystem.

- Engines like [Trino ↗](#) and [PrestoDB ↗](#) offer native support.
- Other third-party tools and platforms, such as [Databricks ↗](#) and [Snowflake ↗](#), provide offerings built on top of the Iceberg format.

Foundry Iceberg catalog

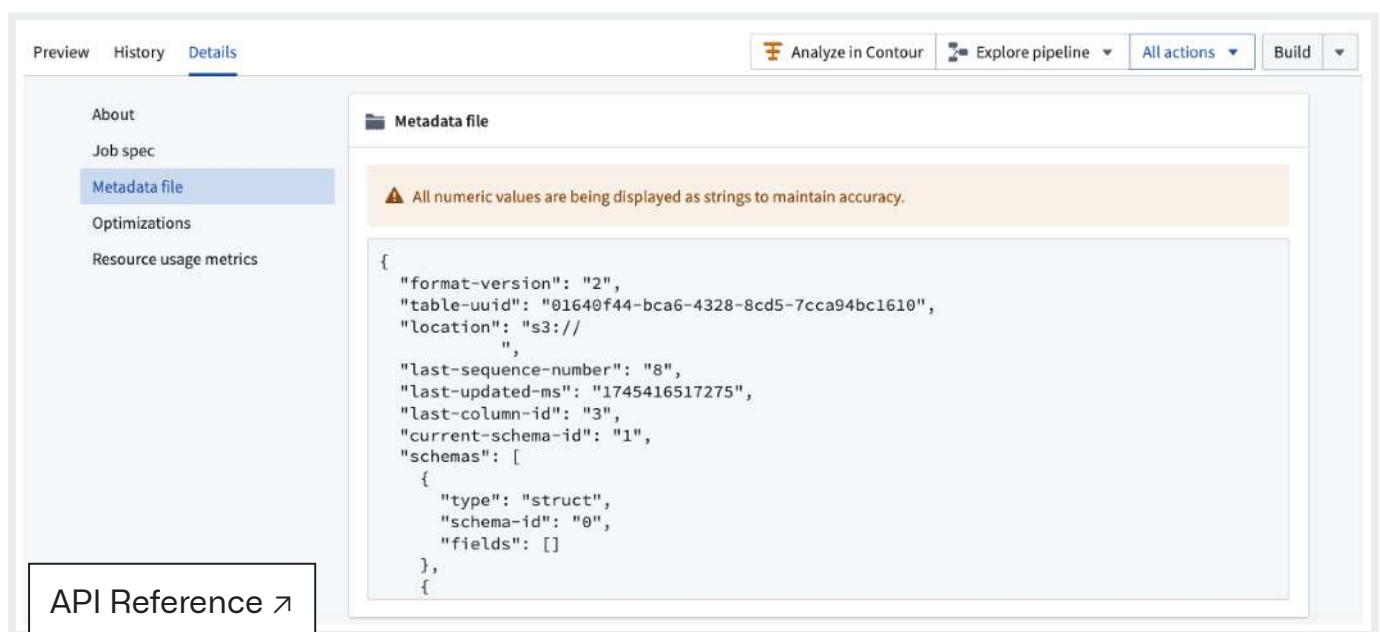
Apache Iceberg defines an [Iceberg REST Catalog ↗](#) specification, which outlines a set of endpoints and behaviors that a service must implement to function as an Iceberg REST catalog.

By adhering to this specification, the service becomes compatible with a growing number of compute engines that support the Iceberg REST Catalog.

Foundry now natively implements this Iceberg REST Catalog specification. In addition, Foundry also supports connectivity to third-party Iceberg REST catalogs such as [Databricks Unity Catalog ↗](#). See [Virtual tables: Iceberg catalogs](#) for supported third-party catalogs.

Foundry exposes Iceberg catalog metadata explicitly as a JSON file via dataset application under the Details tab.

The screenshot below shows how Iceberg catalog metadata appears in the dataset application:



[Send feedback](#)

Iceberg tables vs. datasets

As Palantir expands coverage for Iceberg tables, the features and benefits available for Foundry Iceberg tables will grow over time and limitations will be removed.

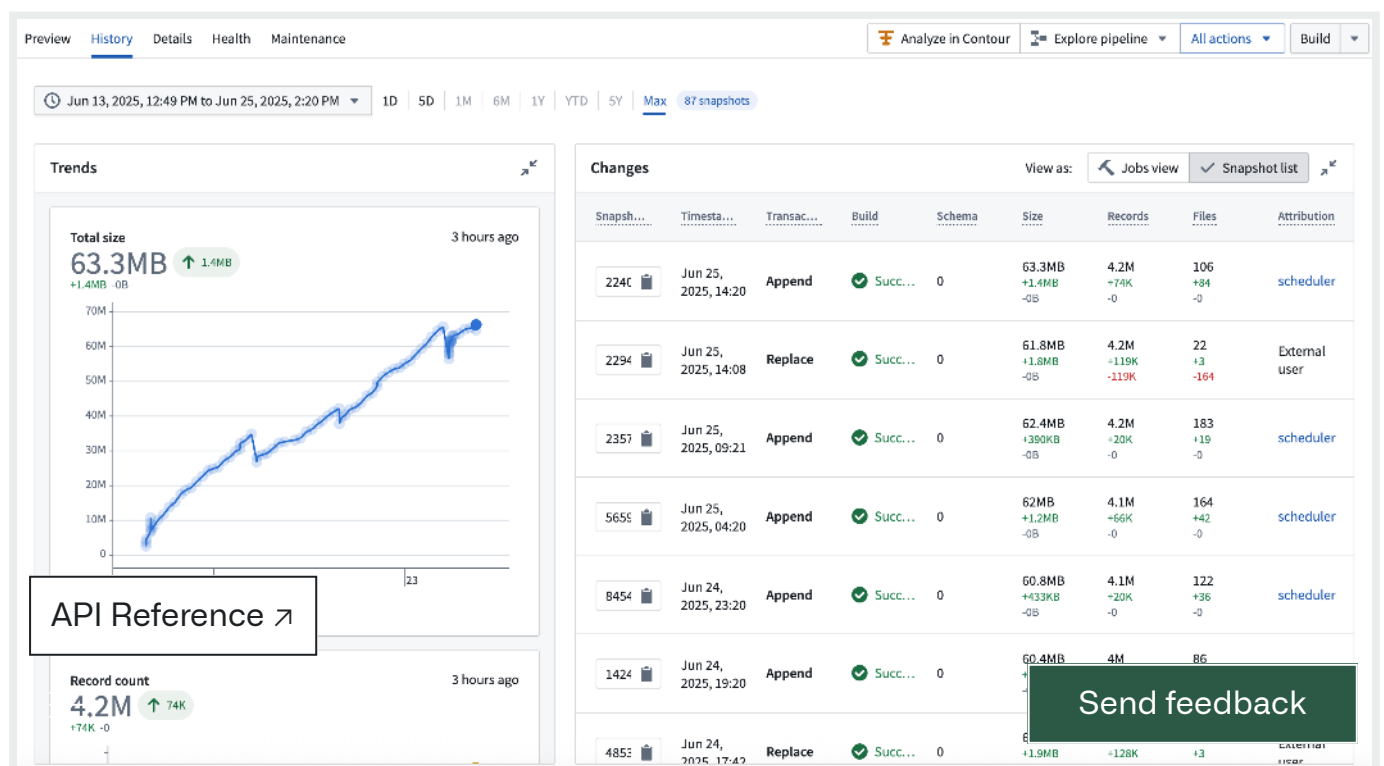
Below summarizes the current state of support as compared to [datasets](#), while Iceberg tables are in the Beta phase of development.

Benefits of Iceberg tables

The following benefits of Iceberg tables are currently available in Foundry:

- **Interoperability:** Open Iceberg format means third-party tools can more easily read and write Palantir Iceberg tables.
- **Compaction:** Support for automated compaction without affecting incremental reads.
- **Row edits:** Support for `DELETE`, `UPDATE` and `MERGE INTO` statements, which allow you to conditionally modify rows without the need to re-snapshot.
- **Changelogs:** Incrementally consume row deletions and updates.
- **Enriched table history:** Enhanced history view surfacing Iceberg [table metadata](#).

The interface for viewing Iceberg table history can be seen in the following screenshot:



Notable differences between Iceberg tables and Foundry datasets

There are a few notable differences worth calling out between the behavior of Iceberg tables and Foundry datasets today:

- **Default branches:** Iceberg's main branch is called `main`, whereas Foundry's main branch is called `master`. In Foundry's integration with Iceberg, `main` and `master` are treated as the same, which means a Foundry job running on `master` will write to an Iceberg's table's `main`.
- **Automatic schema evolution:** Iceberg table schema evolution is not always automatic. Certain workflows, such as table replaces, evolve the table's schema automatically; however, others require explicit schema evolution commands or settings, such as `ALTER TABLE`. When writing through Foundry's transforms APIs and Pipeline Builder, Foundry uses table replaces when running non-incrementally to handle this for you. Other workflows, such as incremental schema evolution, may fail and require manual resolution. See the [Apache Iceberg documentation on schema evolution ↗](#) for details.

Foundry functionality not yet available for Iceberg tables

Note that certain Foundry features are not currently supported for Iceberg tables, including:

- Views ([Foundry views](#) and [Iceberg views ↗](#))
- [Restricted views](#)
- [Streaming](#)
- [Timeseries syncs](#)
- [Projections](#)
- [Faster pipelines in Pipeline Builder](#)

Iceberg terminology disambiguation

~~Iceberg introduces~~ terms that do not always have direct equivalents in Foundry:

API Reference ↗

Send feedback

Iceberg term	Meaning	Foundry disambiguation
Table metadata ↗	Metadata describing the structure, schema, and state of an Iceberg table.	Foundry also tracks metadata for tables and datasets across all formats, but this metadata is managed internally by Foundry services. For Iceberg tables, Foundry exposes the native Iceberg metadata explicitly as a JSON file via the Iceberg catalog.
Snapshots ↗	A record of the table's state at a specific point in time. Iceberg creates a new snapshot with every data modification.	An Iceberg snapshot is roughly equivalent to a Foundry transaction . However, Foundry also uses the term " snapshot " differently: in Foundry, a "snapshot" refers to a transaction that fully replaces all data in a dataset, while an Iceberg snapshot captures all types of data operations, including incremental changes. See Iceberg snapshot types for a complete disambiguation of Foundry catalog dataset transaction types and Iceberg snapshot types.

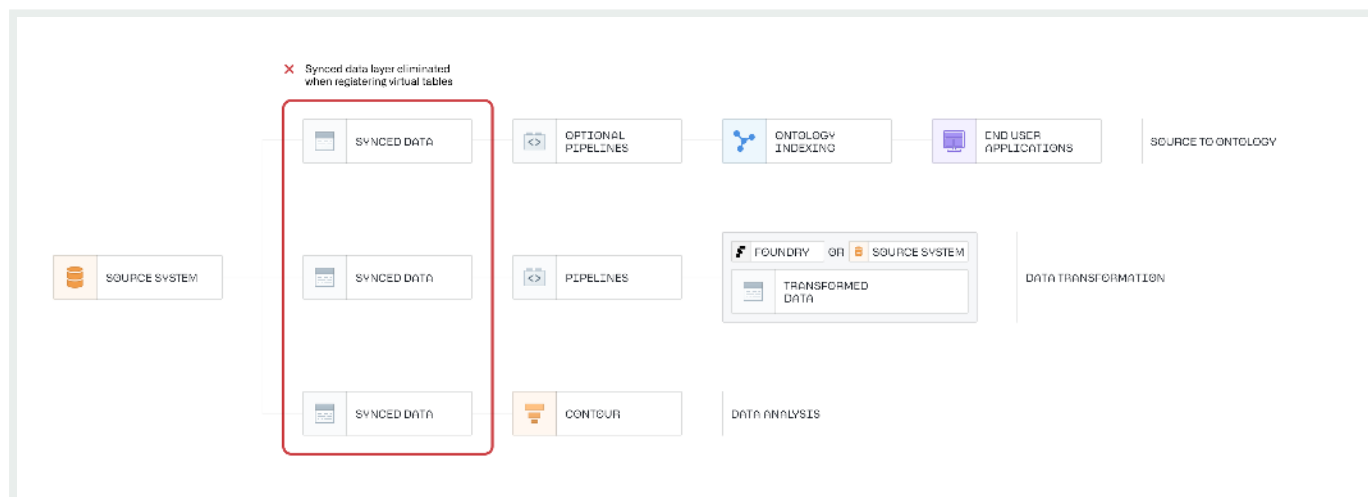
Using Iceberg in Foundry

Refer to the [Iceberg tables](#) section of the documentation for details and guides on working with Foundry Iceberg tables.

Virtual tables

Virtual tables allow you to query tables in supported data platforms without first storing the data in a Foundry [dataset](#).

A virtual table acts as a pointer to a table in a source system outside of Foundry. Virtual tables abstract away the underlying source system and storage formats, enabling you to build workflows that combine data from different source systems seamlessly. Virtual tables can also be combined with datasets stored in Foundry as part of a flexible architecture where data need not be consolidated in one place. You can also create new virtual tables as outputs from Foundry data transformations, enabling workflows where storage is fully external and Foundry handles orchestration, security, and other functions.



A virtual table is defined by:

- A *connection* to the source storage system (for example, a source URL or credentials). This connection is established by [setting up a source](#) in Foundry's data connection

API Reference ↗

which identifies the table in the source system (for example, schema and table name).

Send feedback

As with any resource in Foundry, virtual tables are governed by Foundry's security and permissions model and can be opened or used in various Foundry applications.

Supported sources

The following sources support virtual tables. Refer to the [source documentation](#) for more details on how to configure the connection as well as the supported capabilities.

Source	Status	Supported Formats	Manual Registration	Automatic Registration
Amazon S3	 Generally available	Avro ↗ , Delta ↗ , Iceberg ↗ , Parquet ↗	✓	
OneLake and Azure Data Lake Storage Gen2 (Azure Blob Storage)	 Generally available	Avro ↗ , Delta ↗ , Iceberg ↗ , Parquet ↗	✓	
BigQuery	 Generally available	Table, View, Materialized View	✓	✓
Databricks	 Generally available	Table, View, Materialized View	✓	✓
Foundry	 Beta	Only managed Iceberg	✓	
Google Cloud Storage	 Generally available	Avro ↗ , Delta ↗ , Iceberg ↗ , Parquet ↗	✓	
Snowflake	 Generally available	Table, View, Materialized View	✓	✓
API Reference ↗				

Iceberg catalogs

Send feedback

An Iceberg catalog is required to load virtual tables backed by an Apache Iceberg table. To learn more about Iceberg catalogs, see the [Apache Iceberg documentation ↗](#). Virtual tables support different catalogs depending on the source being used. The table below highlights the supported catalogs. Refer to the [source documentation](#) for more details on how to configure each catalog and use Iceberg tables from the source.

Source	AWS Glue	Horizon Catalog	Object Storage	Polaris	Unity Catalog
Amazon S3	Generally available	Not available	Generally available	Generally available	Legacy: recommended to use Databricks source.
Databricks	-	-	-	-	Generally available
Google Cloud Storage	Not available	Not available	Generally available	Not available	Not available
OneLake and Azure Data Lake Storage Gen2 (Azure Blob Storage)	Not available	Not available	Generally available	Generally available	Legacy: recommended to use Databricks source.
Snowflake	-	Generally available	-	-	-

Supported Foundry workflows

Virtual tables are supported as inputs in the below applications and workflows, and as

- o

API Reference ↗

 builder and Code Repositories.

Send feedback

Supported application	Supported workflow	Not supported
Data Connection	Configure source Register virtual tables	Agent-based connections
Contour	Analyze in Contour	Save as dataset
Ontology	Object creation via Ontology Manager Object creation via Pipeline Builder	
Data Lineage	View Foundry lineage	
Pipeline Builder	Pipeline input Pipeline output Snapshot builds Incremental builds (append-only) External pipeline (compute pushdown)	Streaming builds Faster pipelines
Code Repositories	Python Transforms Java Transforms SQL Transforms Snapshot builds Incremental builds (append-only) Compute pushdown	

API Reference ↗



Send feedback

Supported application	Supported workflow	Not supported
Observability	Health checks supported for virtual tables Data expectations in Pipeline Builder Data expectations in Code Repositories	In single-node transforms, setting <code>on_error='FAIL'</code> to abort jobs on failed expectations is not supported for virtual table outputs. However, it is supported for virtual table inputs, Foundry-managed Iceberg tables, and generally in Spark transforms.
Code Workspaces	JupyterLab® VS Code	



Note that some source types may not support all these capabilities. Refer to the [source-specific documentation](#) for more details. [Learn more about how to configure a source when using virtual tables in Code Repositories.](#)

In general, virtual tables can be used to back most common Foundry workflows by either:

- Directly interacting with the virtual table as described above, or
- Creating a transformation pipeline backed by a virtual table that outputs Foundry datasets or objects. These outputs can be used as normal in the platform.

Virtual table compatibility matrix by source & table type

The matrix below provides an overview of the key capabilities available for virtual tables, broken down by data source and table type. For full details, including any source-specific limitations and supported features, refer to the source-specific documentation.

API Reference ↗

Send feedback

Source	Table type	Capability	Foundry compute	Compute pushdown*
BigQuery	Tables, Views, Materialized Views, Other	✓ Read ✓ Write	✓ Python: Single-node ✓ Python: Spark ✗ Pipeline Builder: Single-node ✓ Pipeline Builder: Spark	✓ Python (Ibis) ✗ Pipeline Builder
Databricks	External Delta, Managed Iceberg	✓ Read ✓ Write	✓ Python: Single-node ✓ Python: Spark ✗ Pipeline Builder: Single-node ✓ Pipeline Builder: Spark	✓ Python (PySpark) ✓ Pipeline Builder
Databricks	Managed Delta	✓ Read ✗ Write	✓ Python: Single-node ✓ Python: Spark ✗ Pipeline Builder: Single-node ✓ Pipeline Builder: Spark	✓ Python (PySpark) ✓ Pipeline Builder
Databricks API Reference ↗ ☰	Views, Materialized Views, Other	✓ Read ✗ Write	✓ Python: Single-node ✓ Python: Spark ✗ Pipeline Builder: Single-node	✓ Python (PySpark) ✓ Pipeline Builder
				Send feedback

Source	Table type	Capability	Foundry compute	Compute pushdown*
Snowflake	Tables, Views, Materialized Views, Other	✓ Read ✓ Write	✓ Pipeline Builder: Spark	
			✓ Python: Single-node	
			✓ Python: Spark ✗ Pipeline Builder: Single-node ✓ Pipeline Builder: Spark	✓ Python (Snowpark) ✓ Pipeline Builder
Snowflake	Managed Iceberg	✓ Read ✗ Write	✗ Python: Single-node ✓ Python: Spark ✗ Pipeline Builder: Single-node ✓ Pipeline Builder: Spark	✓ Python (Snowpark) ✓ Pipeline Builder
AWS S3	Parquet, Avro, CSV, Delta	✓ Read ✓ Write	✗ Python ✗ Pipeline Builder	NA
Azure ADLS	Parquet, Avro, CSV, Delta	✓ Read ✓ Write	✗ Python ✗ Pipeline Builder	NA
Google Cloud Storage	Parquet, Avro, CSV, Delta	✓ Read ✓ Write	✗ Python ✗ Pipeline Builder	NA

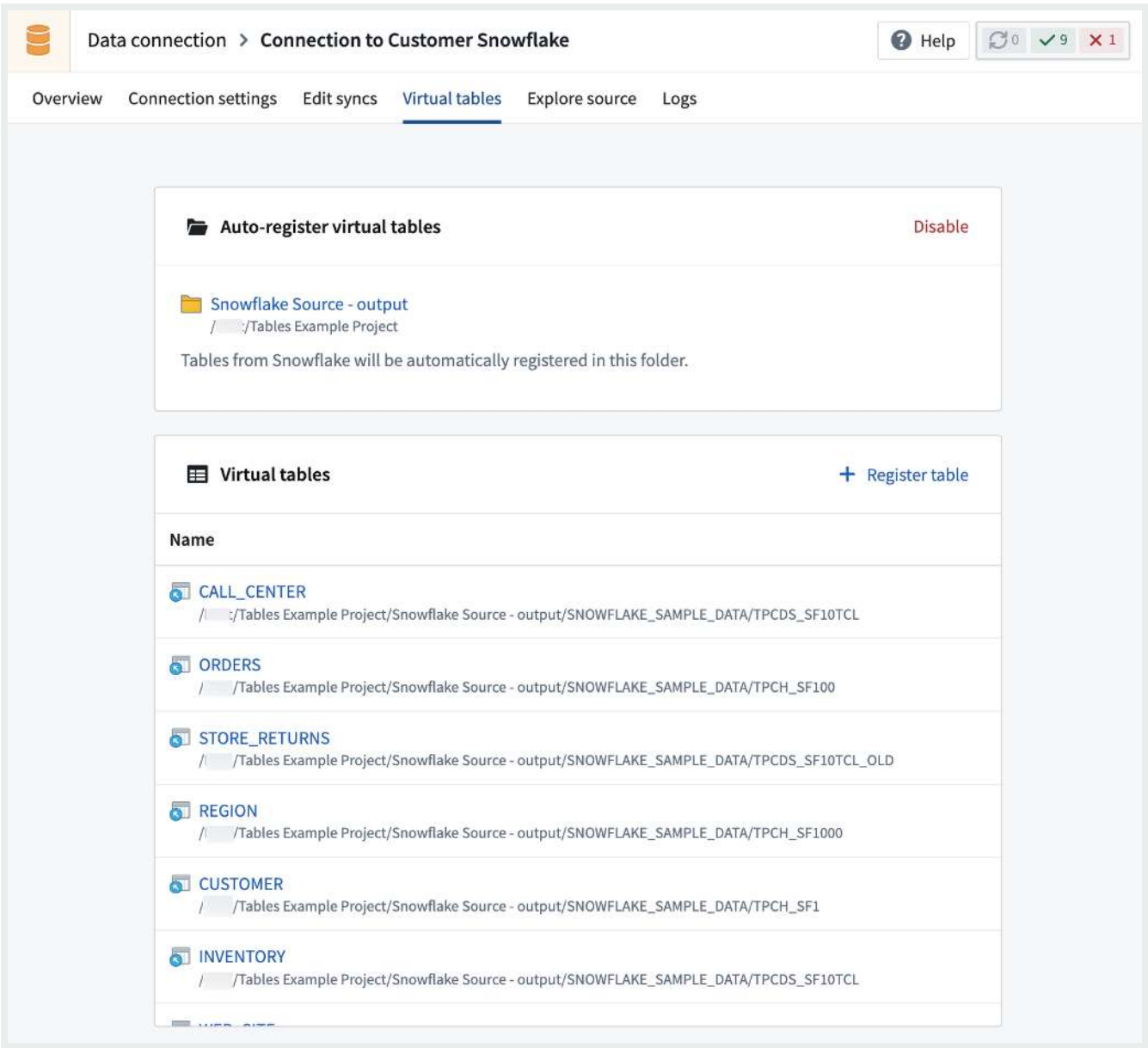
* [Compute pushdown](#) refers to using the native compute engine of the source system.

[API Reference ↗](#)

Set up a connection for a virtual table

Send feedback

Sources supporting virtual tables are set up in the [Data Connection](#) application. Select the source that you want to use, then navigate to the **Virtual tables** tab in the source configuration. Follow the [source-documentation](#) and any requirements listed there for using virtual tables.



Create virtual tables

i All supported sources allow you register individual tables from the source system in Foundry. Tabular source types also support **bulk registration** of multiple virtual tables. Some sources additionally support **automatic registration**, which will periodically register all tables in the source that are accessible with the provided credentials in a designated project.

[API Reference ↗](#)

[Send feedback](#)

To register a virtual table, select **Create virtual table** in the **Virtual tables** tab in the source. Browse available tables and select the table to register. Unless you choose a different location, the virtual table will be created in the default output folder of the source.



Data connection > Connection to Customer Snowflake > CUSTOMER

?

Help

↺0

✓9

✗1

←

CUSTOMER

Registering table from Connection to Customer Snowflake

Register table

Configuration

Database

SNOWFLAKE_SAMPLE_DATA

×

The Snowflake database containing the table.

Schema

TPCH_SF1

×

The Snowflake schema containing the table.

Table

CUSTOMER

×

The Snowflake table or view to be registered.

Confirm location of virtual table

Tables Example Project > raw > New

CUSTOMER

You can manage the default output location on the [source overview page](#). You can also choose a [different location](#) or [use default output location](#).

Bulk registration

When working with tabular source types such as [Databricks](#), [BigQuery](#), and [Snowflake](#), you will be able to **bulk register** multiple virtual tables at once. To begin, select one or more external tables from the left panel. Use the right panel to change where your new virtual tables will be saved, or update their names. Note that changing the name of a virtual table in F

API Reference ↗

Send feedback

Previewing source

Refresh

These resources are a preview of what can be synced into Foundry.

Filter by name...

catalog-test

default

call_center

table

information_schema

reason

catalogtesttwo

samples

accuweather

forecast_daily_calendar_i...

forecast_daily_calendar_...

forecast_daynight_imperial

forecast_daynight_metric

forecast_hourly_imperial

forecast_hourly_metric

historical_daily_calendar_...

historical daily calendar

table

Remove

	city_name STRING	country_code STRING	latitude DOUBLE	longitude DOUBLE	date DATE
1	singapore	sg	1.31056	103.84319	2024-07
2	singapore	sg	1.31056	103.84319	2024-07
3	singapore	sg	1.31056	103.84319	2024-07
4	singapore	sg	1.31056	103.84319	2024-07
5	singapore	sg	1.31056	103.84319	2024-07
6	singapore	sg	1.31056	103.84319	2024-07
7	singapore	sg	1.31056	103.84319	2024-07
8	singapore	sg	1.31056	103.84319	2024-07
9	singapore	sg	1.31056	103.84319	2024-07
10	singapore	sg	1.31056	103.84319	2024-07
11	singapore	sg	1.31056	103.84319	2024-07
12	singapore	sg	1.31056	103.84319	2024-07
13	singapore	sg	1.31056	103.84319	2024-07
14	kolkata	in	22.52648	88.35336	2024-07
15	kolkata	in	22.52648	88.35336	2024-07
16	kolkata	in	22.52648	88.35336	2024-07
17	kolkata	in	22.52648	88.35336	2024-07
18	kolkata	in	22.52648	88.35336	2024-07
19	kolkata	in	22.52648	88.35336	2024-07
20	kolkata	in	22.52648	88.35336	2024-07

Create virtual tables

Configure settings

All selected source tables will be created at this location

Tables Example Project > raw >

table

call_center

forecast_daily_calendar_imperial

forecast_daily_calendar_metric

Confirm creation of resources

Create 4 virtual tables

Auto-registration

When enabling **auto-registration**, you create a new Foundry [project](#) where virtual tables will be created automatically. The folder hierarchy in this project will mirror the structure of the source system, and be periodically updated as new tables are created in the source. When source tables are deleted, related virtual tables won't be auto-deleted in the project, but accessing them won't load any data.

To enable auto-registration, you must have [project creation permissions](#) in Foundry.

API Reference ↗

Send feedback

Data connection

Connection to Customer Snowflake

Table auto-registration

 Help

 0

 9

 1

←

 Table auto-registration

Enable auto-registration of tables in Connection to Customer Snowflake

Enable

 Project details

A new project will be created using the details below. Tables will be periodically registered from Snowflake into the project, after which they can be added as references in other projects.

Name

Connection to Customer Snowflake - Virtual Tables

Description

Auto-registered tables from Connection to Customer Snowflake.

 Access

Everyone from  will be able to see the existence of the project and be granted the Tables Viewer role. Markings can be applied to further restrict access to the project. Once created, access to the project can be managed as normal using the access panel in the project.

Default role

Tables Viewer ▾

 Search for markings...

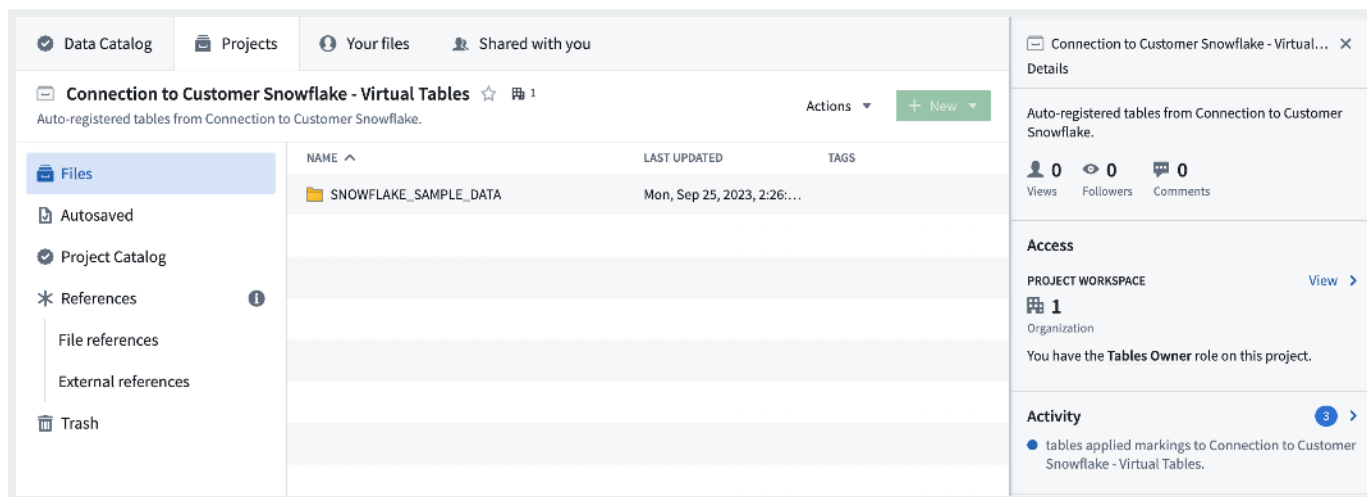
There are no markings on this project

The project is managed by Foundry, and users cannot manually create or update resources in it. Virtual tables registered in this project can be imported into other projects for use in workflow development.

Enabling auto-registration allows setting permissions and access to the project, which can later be managed by the project owner using the access sidebar.

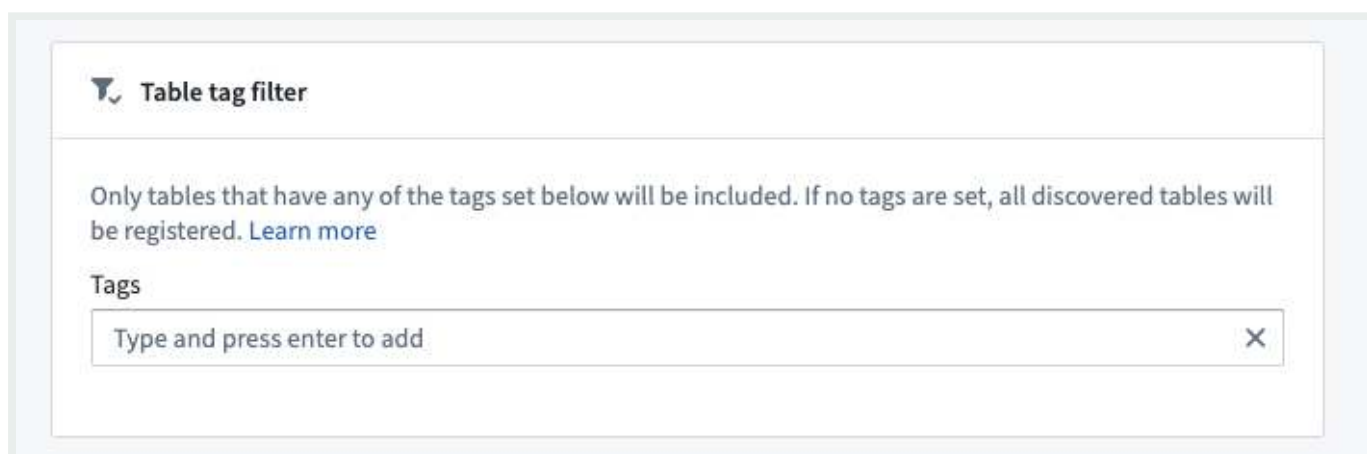
[API Reference ↗](#)

[Send feedback](#)



Tag filtering for Databricks sources

When configuring auto-registration for Databricks sources, you have the option of specifying a list of table tags to filter by. Only tables that have at least one of these tags set in the Databricks [TABLE_TAGS](#) system table will be registered.



Virtual tables in Code Repositories

When virtual tables are used in Code Repositories, the transforms consuming them will automatically obtain network egress based on the [egress policies](#) configured on the source. The credentials configured on the source will necessarily be made available to connect to the source. This is similar behavior to [External Transforms](#).

The following settings must be enabled on the source:

API Reference ↗

1. **Code imports:** This allows the source to be imported and used in [Code Repositories](#). Further details of this setting and how to enable it can be found [here](#).

Send feedback

2. **Export controls:** This controls which [security markings](#) and organizations will be allowed on inputs to a Python Transform using a virtual table. Further details of this setting and how to enable it can be found [here](#).

Once a source has been configured and imported into a code repository, virtual tables can be used as inputs to [Python Transforms](#) in the same way a dataset would be used, using `transforms.api.Input`. [Incremental computation](#) has a consistent API to that of datasets and is supported by a subset of sources. Refer to the [source-specific documentation](#) for more information.

In general, virtual tables are supported as inputs to Python, SQL, and Java Transforms. Only Python Transforms support creating a new virtual table as a transform output, while SQL and Java Transforms support writing to existing virtual tables.

[Learn more about creating new virtual tables via Python Transforms.](#)

Viewing virtual table details

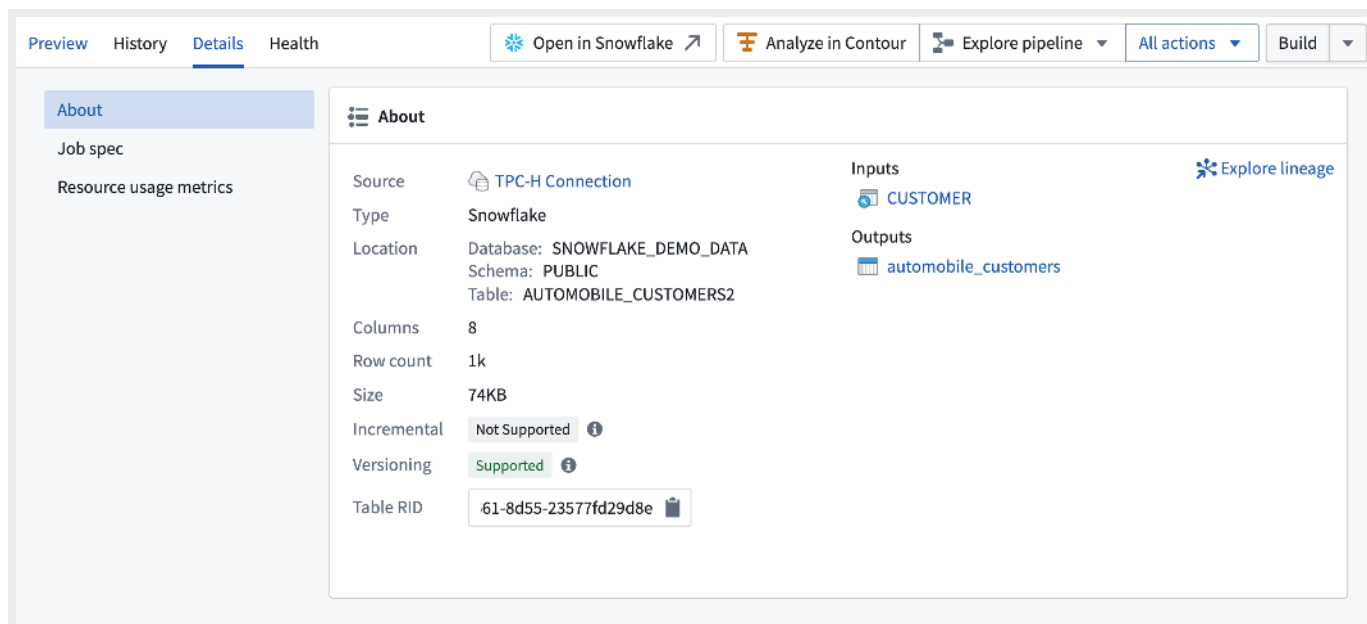
You can access key information about virtual tables in the Details panel of [Dataset preview](#).

This includes:

- **Incremental:** If supported, you can configure incremental pipelines with the table, so downstream builds process only new or changed data instead of reprocessing all rows.
- **Versioning:** If supported, the table provides versioning, allowing Foundry to detect updates and skip unnecessary downstream builds when the data has not changed.

API Reference ↗

Send feedback



Update detection for virtual table inputs

The update detection feature monitors virtual tables and detects changes to the table in the source system. This allows you to use virtual tables as build triggers for dependent resources, such as pipelines or objects.

i Update detection is only available if your virtual table input has been used to create downstream resources, such as downstream tables, datasets, or objects.

Update detection is intended for virtual table inputs, which are tables managed externally and registered in Foundry. Update detection is not necessary for *virtual table outputs* (created by Foundry pipelines), as Foundry automatically tracks updates for these using pipeline build information.

When update detection is enabled, Foundry regularly polls the source system to check for updates to the table. If the source table format supports versioning (for example, Delta or Iceberg), Foundry can detect changes and only trigger downstream builds when necessary. If versioning is not supported, every poll is treated as a potential update, which may result in unnecessary downstream builds.

[API Reference ↗](#)

You can check whether your virtual table supports versioning by [viewing virtual table details](#).

[Send feedback](#)

Update detection only controls *when* a downstream build is triggered. It does not determine whether the resulting build is a snapshot or an [incremental build](#). Sources with versioning support can also support incremental (append-only) builds, where Foundry identifies the specific rows that were added since the previous build and processes only that new data, rather than reprocessing the entire table. Incremental support for virtual tables is currently limited to append-only changes and has additional source-specific prerequisites, such as enabling Delta's [Change Data Feed ↗](#) or Iceberg's [Incremental Reads ↗](#). Refer to the [source-specific documentation](#) for details. If a source does not support versioning, downstream pipelines built on that virtual table must use snapshot builds, which reprocess the entire table on every triggered run.

Enable update detection

To enable update detection for a virtual table input, follow these instructions:

1. Open the virtual table in [Dataset preview](#).
2. Go to the update detection section in the left panel.
3. Enable **Update detection** and set the desired polling schedule (for example, hourly or daily).

Once enabled, you can use the virtual table input as a [schedule trigger](#) for downstream tables and datasets. Any objects backed by the virtual table will reindex automatically when source updates are detected.

API Reference ↗

Send feedback

[Preview](#)
[History](#)
[Details](#)
[Health](#)
Beta


example table

About

Columns

Enter description...

Updated

Jul 11, 2025, 4:02 PM by [username](#)

Created

Jul 11, 2025, 4:02 PM by [username](#)

Location

/path/example table

Type

Databricks table

RID



Source


[Databricks](#)

Source type


Databricks

Size

16 columns

Tags

[Add tags](#)

Update detection


☒

Every hour (Europe/London)

Last updated -

Next run Today at 12:00 PM

Configure objects backed by virtual tables

You can configure objects backed directly by virtual tables in [Ontology Manager](#).

If the backing virtual table is updated outside of Foundry, you should enable [update detection](#) on the virtual table to ensure the objects receive regular updates from the source

[API Reference](#)


Using virtual tables vs syncing to datasets

Send feedback

The decision to use virtual tables vs. sync to Foundry datasets depends on your architecture goals and the target workflow to be supported. We recommend considering the appropriate integration pattern on a workflow-by-workflow basis. The two approaches can be used in conjunction to complement one another.

Below are some considerations to keep in mind about the potential benefits, drawbacks, and limitations of using virtual tables vs. syncing data to datasets.

Benefits of using virtual tables

Virtual tables provide a number of benefits, including:

- Reduction of duplicate storage by not storing source data in Foundry. Note that Foundry will still store data for any downstream-created resources, such as datasets and objects that are outputs from Foundry pipelines.
- Queries can be pushed down to the source system to limit total data transfer. Note that availability of compute pushdown varies by source system and query type.
- Virtual tables may be especially beneficial for very large tables where duplicative storage costs become material.
- With virtual tables, data is queried directly upon use, without the need to synchronize data or consider potential for data staleness.
- Virtual tables provide optionality to help align Foundry implementation with target architecture patterns.

Drawbacks of using virtual tables

Virtual tables may not be the best choice in all circumstances. Some considerations include:

- Interactive performance may be slower than working with data stored in Foundry datasets.
- Compute usage may increase depending on the types of queries being run on the virtual table. For example, tables that are used as an input into a scheduled pipeline may [API Reference ↗](#) compute compared to tables that are frequently accessed interactively in Contour analyses.
- Virtual tables do not benefit from Foundry dataset capabilities such as versioning or branching.

[Send feedback](#)

Limitations of using virtual tables

Limitations of virtual tables include:

- Virtual tables are not available for all sources.
- Virtual tables require a [Foundry worker](#) source and [direct egress policies](#).
- Connections using [self-service private link egress policies](#) are generally not supported. However, [private links](#) set up by a Palantir representative are supported.
- Connections using [agent proxy policies](#) are not supported.
- Connections using [bucket endpoint egress policies](#) are not supported.
- Connections using an [agent worker](#) are not supported.
- Not all Foundry applications and features support using virtual tables as inputs. However, any materialized resources created downstream of virtual tables, such as datasets and object outputs from pipelines, are fully supported across the Foundry ecosystem as usual.
- Transforms that use the `use_external_systems` decorator are currently not compatible with Virtual Tables. Switch to [source-based external transforms](#) or split your transform into multiple transforms, one that uses Virtual Tables as input and one that uses the `use_external_systems` decorator.

Compute for queries on virtual tables

For queries run directly on virtual tables, compute may be split between Foundry and the source system. The specific behavior depends on the query and the degree of pushdown computation supported by the source system. Refer to the [source-specific documentation](#) for more information.

PREVIOUS

← [Iceberg tables](#)

NEXT

[Change data capture \(CDC\)](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[API Reference ↗](#)

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Send feedback](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Change data capture (CDC)

[Change data capture](#) (CDC) is an enterprise data integration pattern often used to stream real-time updates from a relational database to other consumers.

Foundry supports syncing, processing, and storing data from systems that produce change data capture feeds. Components throughout the platform, including [data connectors](#), [streams](#), [pipelines](#), and the [Foundry Ontology](#) can all work natively with this changelog data using the metadata configured on the [schema](#) of the underlying data.

Change data capture support in Foundry

Foundry offers the following support for change data capture updates:

Application	CDC support
Data Connection	CDC support available for syncs (source-system dependent)
Ontology	CDC indexing for batch- and stream-backed objects
Pipeline Builder	Full CDC stream processing and partial CDC streams with backfill
Streams	Full CDC support for live and archive views
Transforms	Append-only incremental for datasets, full changelog incremental available for Iceberg tables
Workshop	Auto-refresh available for frequently updating objects
API Reference ↗	

Changelog metadata requires the following attributes on the data:

- One or more primary key columns
 - Changelog data will contain many entries with identical primary key columns, as changelogs are meant to convey every individual change that occurred on the record with that primary key. Change data capture is particularly useful for data that is being edited, rather than immutable or append-only data feeds.
- One or more ordering columns
 - The ordering column(s) must be numeric and are used to determine the relative order of changes for records with a given set of primary keys. Ordering columns are frequently timestamps represented as longs, however this is not always the case. The largest value is understood to be the most recent, even if the order columns are not timestamps.
- A deletion column
 - The deletion column is used to determine if a given update deleted the record with the given set of primary keys. This must be a boolean that is true if the record was deleted, else it should be false.

This changelog metadata is used to specify a resolution strategy to resolve the changelog down to a current view of the data in the source system. The resolution strategy for change data capture works as follows:

1. Group the data based on the primary key column(s).
2. Find the entry with the largest value in the ordering column(s).
3. If this entry has the value `true` in the deletion column, remove it.

After these steps, the changelog should now be collapsed down to the same view as you would see in the upstream source system generating the changelog. The changelog metadata and resolution strategy is used in different ways by various Palantir components, as described below.

Change data capture in Data Connection

Source system Data Connection may support syncing data from systems as a stream with API Reference ↗ data applied. The ability for a source to implement the capability to sync changelog data is dependent on the ability of the source system

Send feedback

with the required attributes (primary key column(s), ordering column(s), and a deletion column).

Many commonly used databases including MySQL, PostgreSQL, Oracle, Db2, and SQL Server can produce CDC changelog data. Data connectors in Foundry for these systems may or may not support directly syncing these changelogs. Even if a connector does not currently support CDC syncs, you can still sync data via Kafka or other streaming middleware and use as a changelog once the data arrives in Foundry.

The following source types in Data Connection currently support CDC syncs:

- Db2
- Microsoft SQL Server
- Oracle
- PostgreSQL

If you have changelog-shaped data available from other sources, such as Kafka, Kinesis, Amazon SQS, or push-based integration with streams, read how to manually configure changelog metadata using the key-by board in [Pipeline Builder](#).

Enable change data capture on an external database

To sync changelog feeds from a supported source type, you must have the correct settings enabled for the relevant tables and, in some cases, for the entire database.

For example, to enable change data capture for Microsoft SQL Server, you must run a command to enable CDC on the database:

```
USE <database>
GO
EXEC sys.sp_cdc_enable_db
GO
```

For more information, see the [API Reference](#) for the `sp_cdc_enable_db` command on each table that should be recording changelogs:

[Send feedback](#)

```
EXEC sys.sp_cdc_enable_table
    @source_schema = N'<schema>'
    , @source_name = N'<table_name>'
    , @role_name = NULL
    , @capture_instance = NULL
    , @supports_net_changes = 0
    , @filegroup_name = N'PRIMARY';
GO
```

The above examples provide a high level explanation of what may be required to enable a source system to produce changelog data. Specific information on how to enable change data capture logs for a given system can be found in documentation provided by that system.

Set up a source connection for change data capture

Now, you can configure a source in Data Connection that connects to the system from which you want to capture changelogs. Continuing with our example, set up a connection using the Microsoft SQL Server connector, as shown below:

The screenshot shows the 'Data connection > CDC Demo Source' interface. The 'Connection settings' tab is active, displaying the following configuration:

- Select host type:** Hostname (selected), IPv4, IPv6
- Hostname:** myhostname
- Port:** 1433
- Database name:** Example
- Authentication:**
 - Select credentials method:** Username/Password (selected), Active Directory Msi, Active Directory Password, Active Directory Service Principal

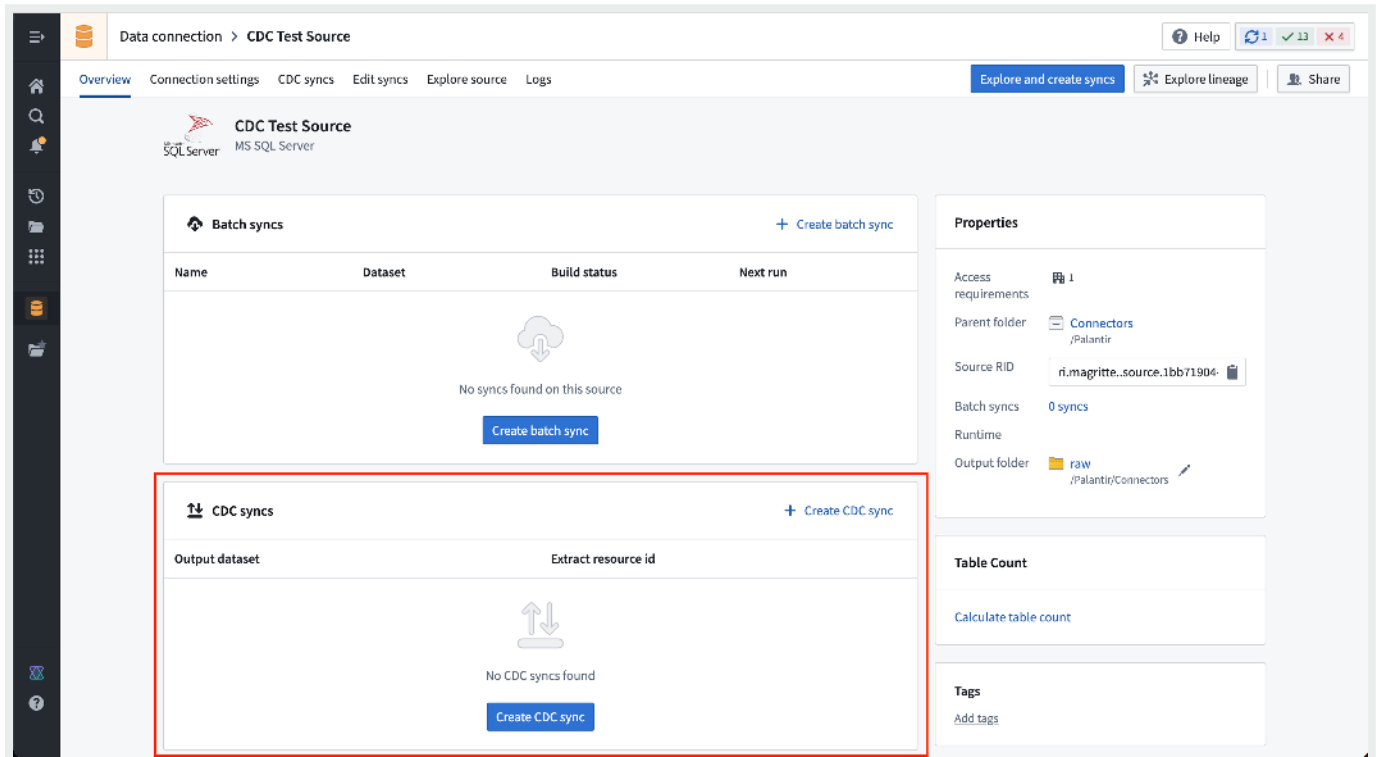
Buttons for 'Back' and 'Save' are visible at the bottom of the settings panel.

[API Reference ↗](#)

[Send feedback](#)

Create a change data capture sync

On the source overview page, you will find an empty table of **CDC Syncs**.



Select **Create CDC sync** to add a new change data capture sync. Specify the table you wish to sync, and the following information will be automatically derived by the connector:

- The schema for the output streaming dataset.
- Changelog metadata, including primary key columns, ordering columns, and a deletion column.

[API Reference ↗](#)

[Send feedback](#)

The screenshot shows the configuration interface for a CDC Test Source. The breadcrumb path is 'Data connection > CDC Test Source > dbo.Customers'. The main configuration area is divided into three sections:

- Table to sync:** A dropdown menu is set to 'dbo.Customers'.
- Schema:** A table lists columns with their names, types, and nullability. To the right, a 'Schema JSON' is displayed.
- Primary keys:** A text input field contains 'CustomerId'.

Name	Type	Nullable
CustomerId	INTEGER	☐
CustomerName	String	☐
CustomerCount	String	☐
Add a column...	String	☐

```

{
  "name": "CustomerId",
  "type": "INTEGER",
  "precision": null,
  "scale": null
},
{
  "name": "CustomerName",
  "type": "STRING",
  "precision": null,
  "scale": null
}

```

As with other streams, you must specify the expected [throughput](#) at creation. The throughput cannot be changed after sync creation; ensure that your stream is configured to support the expected volume of changelog data. The default volume is 5MB/s, which is typically more than required for most change data capture workflows. Since changes in relational databases are often produced at “human scale”, the volume and frequency of changes is much smaller than what is possible with “machine scale” sensor or equipment data.

Save the sync configuration to create a new stream in the specified output location.

i Currently, the CDC job must be manually (re)started after any changes. All CDC syncs run as a single multi-output extract job, meaning that any existing CDC streams from the same source must be briefly stopped whenever a feed is added or removed. The streams will gracefully catch up with any data that was changed while the streaming sync job was stopped.

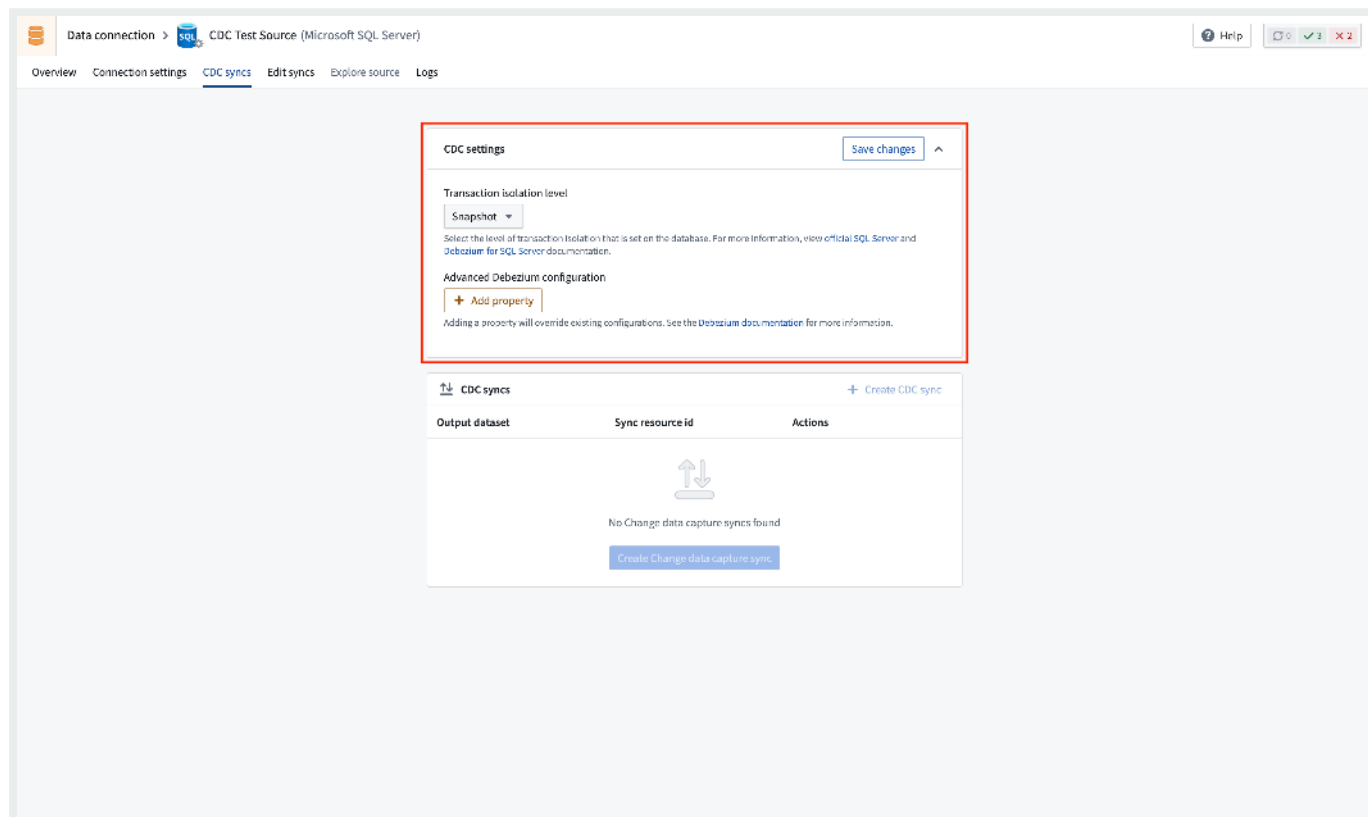
After starting the output stream, changelog data should begin flowing and appear in the live view of the output streaming dataset.

[API Reference ↗](#)

Configure custom CDC settings per source

[Send feedback](#)

Foundry's CDC implementation leverages the open-source platform [Debezium](#) to read data source logs. You can configure the Debezium connection and engine properties for an individual source directly from the Data Connection interface. First, navigate to the created source that you want to configure. Then select **CDC syncs** from the top right. Select the **CDC settings** section to expand the configuration options.



Foundry's default CDC configuration is designed to provide a frictionless setup and minimize disruptions on your source systems. However, we strongly recommend reviewing the [specific default behaviors](#) for your connector, particularly its [snapshot mode](#), to ensure these defaults align with your data requirements.

Each data source type has recommended configurable settings. For example, the image above shows a **Transaction isolation level** for the Microsoft SQL Server connector. However, you can also configure arbitrary Debezium properties using the **Advanced Debezium configuration** option. This flexibility allows you to customize settings to the specific needs of each data source.

Customized CDC settings for data connectors can be useful for a variety of situations, in the following examples:

[API Reference](#)

1. **High transaction volume:** For data connectors experiencing rapid growth, the default `max.batch.size` setting may lead to excessive small batch processing,

[Send feedback](#)

increasing overhead and delaying data propagation. By raising the `max.batch.size`, you can consolidate changes into larger batches, enhancing efficiency and data flow.

2. **Network instability:** For data connectors prone to transient network issues or temporary errors, increasing `error.max.retries` provides the connector with more opportunities to recover autonomously, reducing the risk of missed data.
3. **Real-time updates:** In high-transaction environments where near-real-time updates are crucial, lowering `offset.flush.interval.ms` ensures more frequent offset flushing, minimizing recovery windows and improving change propagation timeliness.

Review [Debezium's documentation ↗](#) to learn which properties can be configured for each data connector type.



When configuring arbitrary Debezium properties, it is your responsibility to ensure compatibility and functionality. Improper configurations may lead to unexpected behavior or data loss. Proceed with caution and consult the Debezium documentation for guidance.

Change data capture in streams

Streams with changelog metadata will display two views into the data:

- A **live view** showing the fully expanded list of changelog entries.
- An **archive view** where the data is resolved according to the resolution strategy and collapsed down to the current latest view of the data.

[API Reference ↗](#)

[Send feedback](#)

 CDC Demo > [Demo] Customer ☆
File ▾ Help ▾ | 1 | master ▾

 Live ▾

PreviewStreamDetailsMet

 Live

This is a live view of the stream and shows a snippet of the latest incoming data. ✓

 Archive

This is the archived dataset of the live stream. It was last updated Oct 4, 2023, 5:37 PM.

The schema will display changelog metadata as a primary key resolution strategy in the details view.

PreviewStreamDetailsMetrics

Stopped since Oct 16, 2023, 4:37 PMExplore pipeline ▾

AboutLatest run detailsStream settingsSchemaFilesJob specCustom metadataCurrent transaction viewResource usage metricsRetention policiesLineage-aware retention policies

Currently viewing details for live streamSwitch to archive dataset

SchemaShow versioned schema

```
"primaryKey": {
  "columns": [
    "CustomerId"
  ],
  "resolution": {
    "type": "duplicate",
    "duplicate": {
      "resolutionStrategy": {
        "type": "latestWins",
        "latestWins": {
          "orderingColumn": "__timestamp"
        }
      },
      "deletionColumn": "__is_deleted"
    }
  }
}
```

Edit

[API Reference ↗](#)

 Streams currently use the ordering column to perform resolution. data will be resolved in the archive according to the provided ordering column, even

[Send feedback](#)

if the data was received out of order. This behavior differs from the behavior of changelog data in the Foundry Ontology, which indexes data based on the order of arrival in the stream used to back the object type.

Change data capture in Pipeline Builder

Pipeline Builder offers powerful capabilities for processing and configuring change data capture (CDC) streams. This section covers two scenarios: working with full CDC streams and handling partial CDC streams that require backfilling.

Process full CDC streams

When working with streaming transformations in Pipeline Builder, you can process changelog data while preserving its metadata. As long as the metadata columns (primary key, ordering, and deletion) remain unmodified during transformations, the changelog metadata will automatically propagate to any outputs.

Configure partial CDC streams with a backfill dataset

In some cases, a CDC stream may not contain all historical data. Instead, it could only contain changes from a specific point in time. To create a complete dataset, you must combine a backfill dataset with the ongoing CDC stream. Use the steps below to achieve this:

1. Prepare the backfill dataset:

- Ingest a snapshot of the data using the same Data Connection source used for the CDC stream.
- Ensure the backfill dataset includes a timestamp column. If not present in the source data, generate one using an SQL function (`CURRENT_TIMESTAMP()` , for example).

2. Combine the datasets in Pipeline Builder:

- Use a **Union** transform to combine the backfill dataset with the ongoing CDC stream.

[API Reference](#) ↗

Metadata:

- Use the **Key By** transform board to prepare the combined data in the Foundry Ontology.

[Send feedback](#)

- Set the primary key column(s) to match those in your source system.
- Choose the timestamp column as the ordering column.
- If applicable, specify the deletion column.

4. Handle deletions and ordering:

- Ensure the **Key By** board is configured to order the data by the timestamp column.
- If the source data does not include explicit deletion information, you may need to implement logic to infer deletions (for example, by comparing consecutive snapshots).

The **Key By** transform board will append the necessary CDC metadata to the stream, including the following:

- A primary key resolution strategy
- Ordering column(s)
- A deletion column

This configuration prepares the data for proper resolution when synced to the Ontology, ensuring that the most recent state of each record is maintained based on the timestamp ordering.

Apply CDC metadata to non-CDC data

If your input stream lacks changelog metadata or if the metadata columns were modified during transformations, you can still use the **Key By** board to apply changelog metadata to your output(s). This allows you to treat any dataset as a CDC stream, enabling powerful data integration and real-time update capabilities.

KEY BY

Cancel Preview

Primary key is deleted column ?

Select a column

Key by columns * ?

Search for columns...

Primary key ordering columns ?

Search for columns...

API Reference ↗

Apply Cancel

Send feedback

Change data capture in the Ontology

The Ontology uses changelog metadata to index data into object types in [Object Storage v2](#) that are backed by a streaming data source. The data arriving in the stream is resolved and indexed into a latest current view that is available when querying the Ontology (to display in a Workshop module, for example).

If [retention](#) is configured on a data source with changelog metadata, any records that do not receive updates within the retention window time will disappear from the Ontology.

 The Ontology currently ignores the ordering column(s) specified in the changelog metadata. Instead, Object Storage v2 indexes data based on the order it arrives in the backing data source. Concretely, this means that for a given primary key, if a log entry with an ordering value of `2` arrives at `t1` and `data_column=foo`, followed by another log entry with an ordering value of `1` that arrives at `t2` with `data_column=bar`, the record will appear with `data_column=bar` even though in the source system the most recent value is `data_column=foo`. This can cause the Ontology to incorrectly reflect the data in the source system if data arrives out of order.

Since connectors used with Palantir are guaranteed to deliver data in order, and Foundry streams maintain ordering, this Ontology behavior will likely only affect custom setups or older streaming changelogs that are manually backfilled and not re-ordered before syncing. If you encounter this situation, we recommend applying a transformation to reorder the data in Pipeline Builder before syncing to the Ontology.

Change data capture in Workshop

Workshop supports [auto-refresh](#) to display the frequently updating data as soon as it is available in the Ontology. Auto-refresh is compatible with CDC and may be used to ensure that any data streamed in with change data capture is promptly available in Workshop applications.

[API Reference ↗](#)

 Auto-refresh is available for any data that is expected to update in the Workshop module is open. Data is not required to have changelog metadata on the

[Send feedback](#)

backing data source to use auto-refresh.

Considerations when using change data capture

We recommend reviewing the following information on backfills, outages, and other known limitations before using CDC workflows.

Backfill

All changelog syncs are handled on an exclusively "going forward" basis; no automatic backfill is performed.

Often, a full backfill of changelogs is not possible, since most systems do not enable CDC by default. Even if changelogs are enabled, most systems include a retention period, after which changelogs are permanently deleted and no longer recoverable.

If a full backfill is required, we recommend the following:

1. Set up a CDC stream on a "going forward" basis.
2. Perform a batch sync to extract the desired historical data.
3. Convert the historical batch data into a stream of "create" records for each primary key, then merge that stream into the CDC stream.

Backfills may result in data that is out of order, and you may need to manually reorder or replay streams to properly prepare data for syncing to the Ontology.

Outages

You may encounter the following outages when using a CDC workflow:

- Network connectivity between the source database and the Foundry agent

API Reference ↗
• Network connectivity between the Foundry agent host and Foundry

- Foundry outage
- Database outage

Send feedback

- Agent outage

Outages are handled gracefully if the retention window on replication logs and changelogs in the database are configured to be longer than the maximum expected outage.

For example, if the connection to Foundry goes down for several hours, and the log retention window on Microsoft SQL Server is set to one day, the database will continue recording changelog entries. Foundry will gracefully catch up once it is back online and reconnected to Microsoft SQL Server. Since no new data will flow until the queue of changelog entries is cleared, there may be some lag before changes are again flowing at near real time into Palantir.

Use changelog-shaped data from non-changelog sources

Data does not need to be ingested as a changelog to use CDC workflows. Any streaming data in Palantir may be “keyed” with changelog metadata and then used as CDC data in workflows after syncing to the Ontology.

This means, for example, that push-based ingestion using stream proxy may be used to manually push changelog-shaped records into a stream.

Similarly, if changelog data is available in a Kafka topic, it may be ingested using a standard (non-CDC) sync. Then, it can be “keyed” using Pipeline Builder and used in the Ontology and beyond.

Remove changelog metadata

Sometimes, it may be useful to remove changelog metadata. For example, you might want to remove metadata to analyze the process flow captured by the changelog. To remove changelog metadata, use one of the following methods:

- Perform any transformation on the metadata columns
- Manually remove the resolution strategy from the schema of the streaming dataset

[API Reference ↗](#)

Enabling access to your CDC tables

[Send feedback](#)

When connecting to your source system, you must provide user credentials within **Connection Details**. These credentials are tied to permissions set at the source. For basic data ingestion, this user will need `SELECT` permissions for the datasets you plan to ingest. However, CDC connections may require the user to have both `SELECT` and `EXECUTE` permissions on the table you wish to ingest from as well as its schema.

Default CDC behavior

Below are some universal default settings for CDC connections:

Property	Default Value	Notes
tombstones.on.delete	false	Tombstone records are not produced when rows are deleted.
decimal.handling.mode	string	Decimal columns are converted to string.
include.schema.changes	false	DDL events (schema changes) are not forwarded into output topics.
offset.flush.interval.ms	60000 (1 minute)	Debezium attempts to flush offsets every minute.
error.max.retries	5	The engine will attempt to recover from transient errors five times before failing.

Connector-specific defaults

Most database connectors also have default settings for snapshot mode and other parameters. The tables below contain summaries of the most important settings. You can override these configurations from the **CDC syncs** page. Select **Edit** in the **CDC settings** dropdown section, then select **Add property** under **Advanced Debezium configuration**.

Oracle

API Reference ↗	Default value
snapshot.mode	Send feedback
schema.history.internal.store.only.captured.tables.ddl	true

Property	Default value
internal.log.mining.log.query.max.retries	10
log.mining.strategy	online_catalog
log.mining.query.filter.mode	in
log.mining.batch.size.default	50000
database.connection.adapter	logminer

PostgreSQL

Property	Default value
snapshot.mode	never
plugin.name	pgoutput

Db2

Property	Default value
snapshot.mode	no_data
snapshot.isolation.mode	none

Microsoft SQL Server

Property	Default value
snapshot.mode	schema_only
offset.flush.interval.ms	1000

Snapshot mode considerations

By default, Foundry configures `snapshot.mode` to `schema_only` (Oracle, SQL Server), `no_data` (Db2) or `never` (PostgreSQL) to avoid impacting the database's availability; these modes avoid locking the tables during the snapshot process, ensuring minimal disruption to ongoing database operations. Although these modes avoid locking, they still pose a data loss risk if archived logs are purged before Debezium can capture them.

[API Reference ↗](#)

[Send feedback](#)

example, Oracle may delete older redo logs over time; if Debezium tries to read a deleted log, it cannot recover the missing data without a full snapshot.

Alternative configurations

If you want to protect against log purging or ensure a base dataset, consider a more complete snapshot configuration instead:

- **recovery**
 - **Behavior:** Recovers a connector that is already capturing changes if the schema history topic is lost or corrupted; resets and rebuilds the database schema history on restart if no schema changes happened since the last shutdown.
 - **Limitation:** Requires infinite retention on the schema history topic.
 - **always**
 - **Behavior:** Takes a complete snapshot each time the connector restarts, guaranteeing a fully up-to-date copy of the data.
 - **Limitation:** May induce table locking and added overhead on every start.
3. **when_needed**
- **Behavior:** Snapshots only if Debezium cannot detect valid offsets; for example, if the offset does not exist or is expired.
 - **Limitation:** Still may lock tables when a snapshot is triggered.

PREVIOUS
← [Virtual tables](#)

NEXT
[Views](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

[API Reference ↗](#)

[Send feedback](#)

Views

A **View** behaves similarly to a Foundry [dataset view](#), but it does not hold any files containing data. Instead, it is composed of the union of other datasets (known as backing datasets) when it is read. Views can be thought of as "pointing" to their backing datasets.

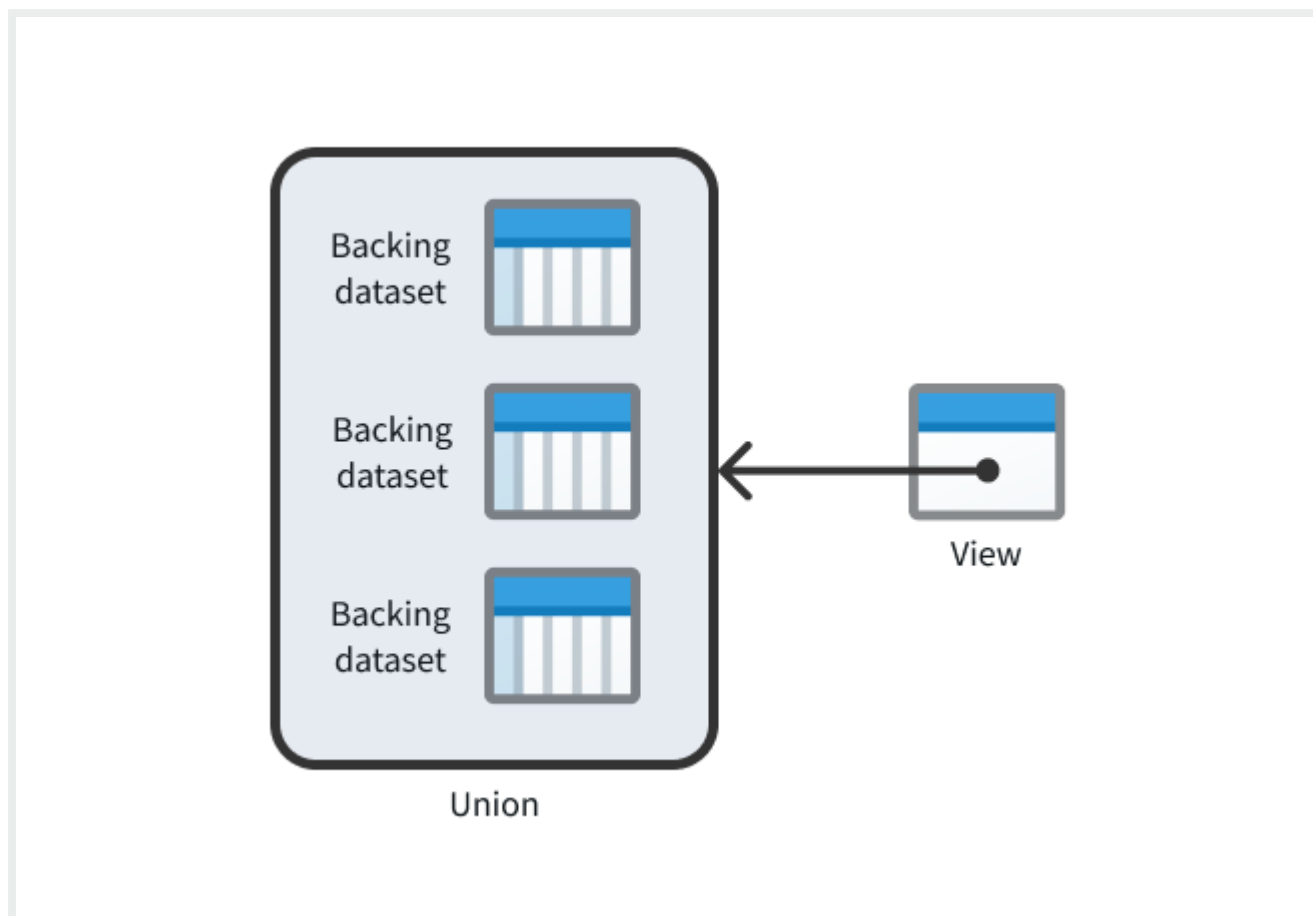
Views can also automatically perform deduplication of data with the addition of a primary key. If any of a View's backing datasets have new data, a build will be automatically triggered to ensure that the latest data is available after deduplication. Views build like regular datasets but complete almost instantly since no data is actually read or written as part of the build.

Generally, you can use Views like regular datasets. However, Views cannot be specified as valid transform *outputs*; instead, they can be specified as valid transform *inputs*.

Additionally, Views can only be used with datasets that have a schema, since Views operates with rows.

⚠ Views built over incremental datasets should not be used as incremental inputs in downstream transforms. When Views are read incrementally, the deduplication is done within the incrementally read transaction range and not across the entire View.

[API Reference ↗](#)[Send feedback](#)



Some primary use cases for views include the following:

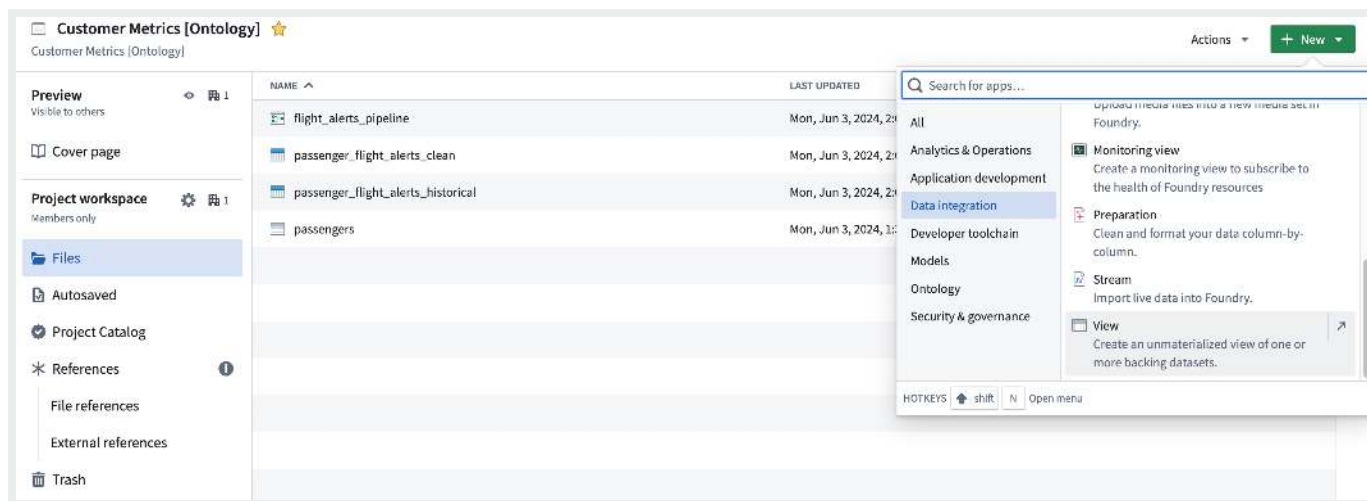
- **Automatic updates:** Create a union dataset that is always up-to-date without having to perform a transform.
- **Folder organization:** Replicate a dataset in a different location without incurring additional compute and storage costs.
- **Data uniqueness:** Automatically guarantee data with primary key deduplication without transforms.

Create a View

To create a new View, open **Files** from the left workspace navigation panel, then find your desired Project or folder. Once there, select **New > View** in the top right to create a new View within the current Project or folder:

[API Reference ↗](#)

[Send feedback](#)



Choose a name and location for the View, then proceed with configuration. When the View is created, a build schedule will be automatically created in the background that will rebuild the View any time the backing datasets update. If you want to immediately read the View after creating or modifying it, you must build the View manually.

Select backing datasets

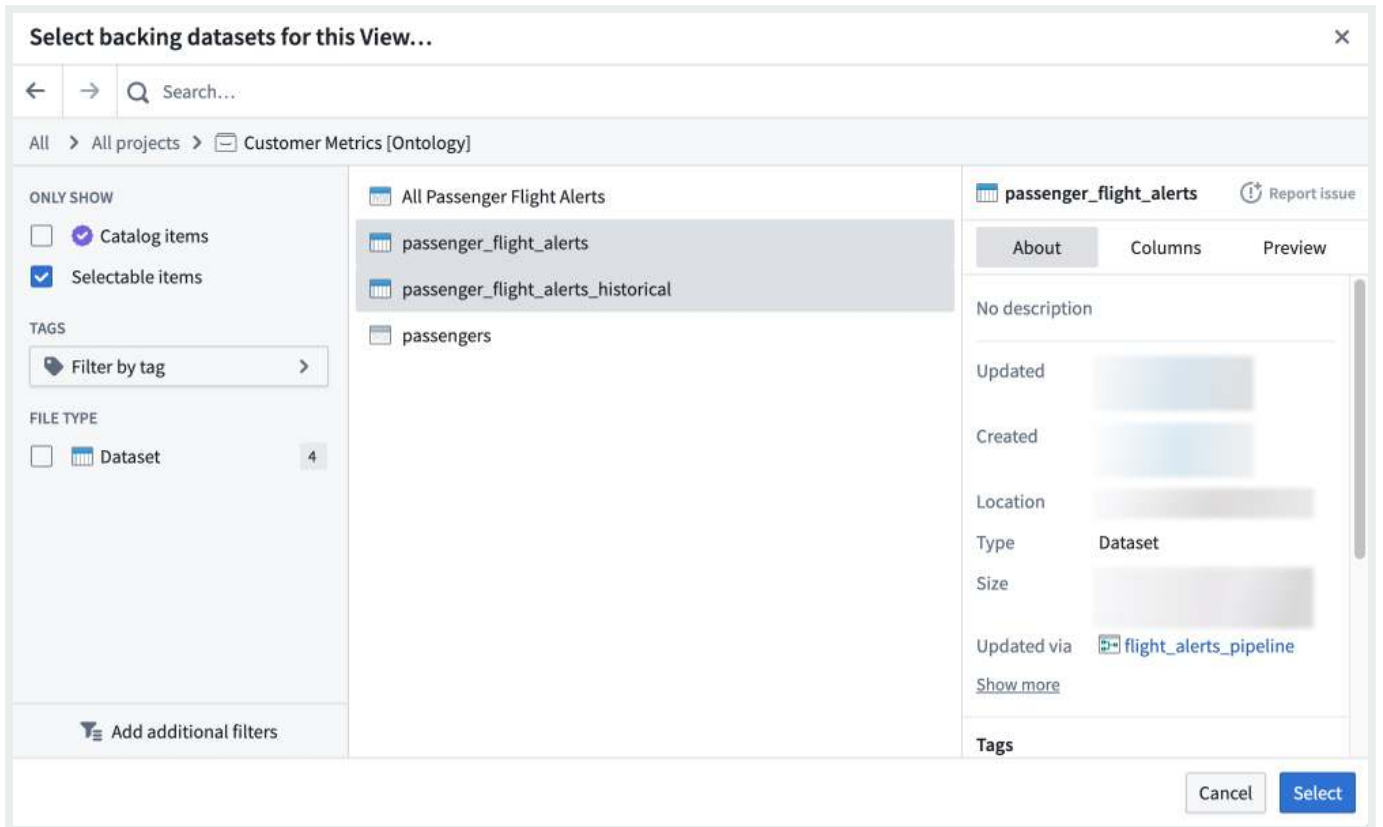
⚠ Because a View is composed of the union of its backing datasets, every backing dataset for a given View must possess the same set of column names and types (though the column order can be different).

i If you add or remove columns from a backing dataset, the View may fail with a `View:MismatchedNumberOfColumns` error. To resolve this, navigate to **Details > Dataset view settings** and enable **Schema evolution**. This allows the View to automatically adapt to schema changes in its backing datasets without needing to create a new View.

One or more backing datasets are required to construct a View. After initial configuration, a View's backing datasets can be modified by navigating to **Details > Dataset view settings**.

[API Reference ↗](#)

[Send feedback](#)



Add a primary key

Data deduplication is a key feature of Views that you can enable by adding a primary key. Once the primary key is configured, the View is guaranteed to have primary key uniqueness; each primary key present in the View appears in only one row. Add a primary key to ensure that only the most up-to-date data for each primary key is returned when the View is read. This automatic deduplication can be especially useful in cases when the View's backing dataset(s) may contain unwanted duplicate rows.

⚠ Once the primary key is set, it cannot be modified without creating a new View.

To add a primary key, select one or more columns which, when combined, will form the primary key. The column(s) must not contain null values.

[API Reference ↗](#)

[Send feedback](#)

Add primary key

Primary key columns

passenger_id

flight_number

Requires the columns to not have null values.

Duplicate keys resolution

alert_date

Rows with the same primary key will be resolved by taking the row with the highest value of this column.

Deletion column (optional)

Search for or enter a column name...

When this column is true or null in the most recent row for a given primary key, that primary key will be removed from the final resolved view. Must be a boolean column.

!

Once saved, the primary key cannot be changed.

Set primary key

If you do not add a primary key during initial configuration, you can do so later by navigating to **Details > Dataset view settings**.

Resolve duplicate keys

To add a primary key, you must also specify a column (or combination of columns) for resolving duplicate primary keys. Deduplication of rows with the same primary key is performed by keeping the row with the highest values of these columns in order and discarding the others. This ensures that every primary key appearing in the View is unique.

API Reference ↗

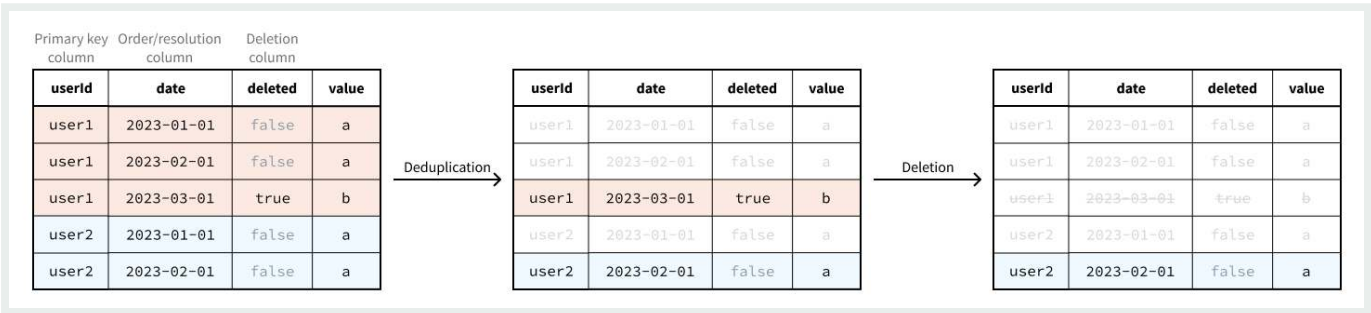
ion column

Send feedback

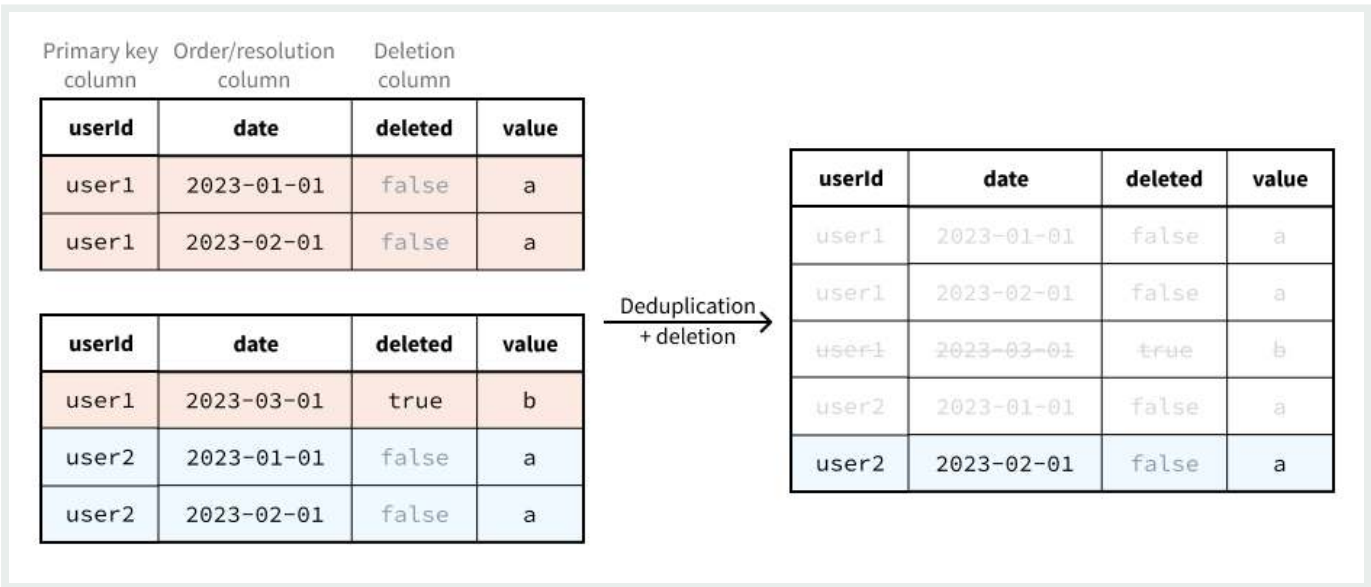
You can also optionally specify a deletion column, which must contain Boolean values. After deduplication, if the final remaining row for a given primary key has `true` in the deletion column, that row will be excluded from the View.

Deduplication and deletion in practice

Duplicate primary keys are resolved *before* the deletion column is used to exclude rows. Therefore, only the value of the deletion column in the “latest” row for a given primary key matters.



Deduplication and deletion is performed across all of the backing datasets:



Deduplication and deletion is implemented with a combination of the following:

- An appropriately configured, automatically created [projection](#) on the View.
- [API Reference](#) ↗ -based filter logic that is evaluated on every read.

Send feedback

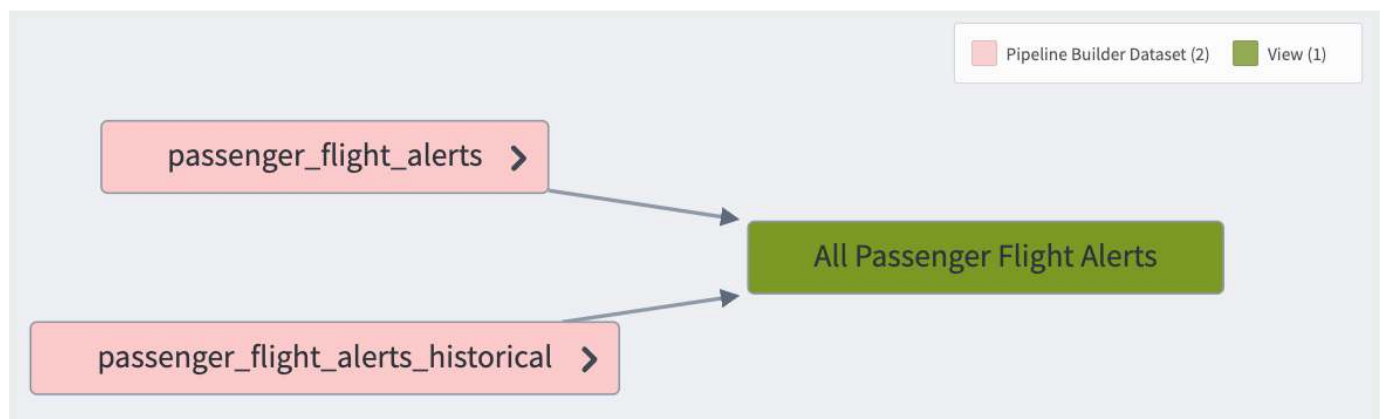
Although the filter logic is evaluated on every read, the evaluation of this logic has a negligible impact on runtime and compute cost provided the projection is up to date and the backing datasets of the View have only had `APPEND` or `UPDATE` transactions since the last projection build.

Considering this architecture, we recommend only creating a View with primary key configuration in the following cases.

- All of the backing datasets are updated with `APPEND` or strictly additive `UPDATE` transactions.
- Some of the backing datasets are updated with `APPEND` or strictly additive `UPDATE` transactions and some are updated with `SNAPSHOT` transactions, but the ones that are updated with `SNAPSHOT` transactions are built infrequently.

If updates to the backing datasets require a full rebuild of the projection, considerable compute costs will be incurred since readers ([Contour](#), transform jobs, and so on) must apply the filter logic without the benefit of the projection until the projection rebuild is complete.

Use Views



Views are visible in Foundry like regular Foundry datasets and can be used anywhere that regular datasets can be used. The only exception to this is that Views are not valid transform outputs (though they are valid transform inputs). An example use case for Views is to create a union of multiple datasets efficiently, using a primary key to ensure the resulting View has

d [API Reference ↗](#)

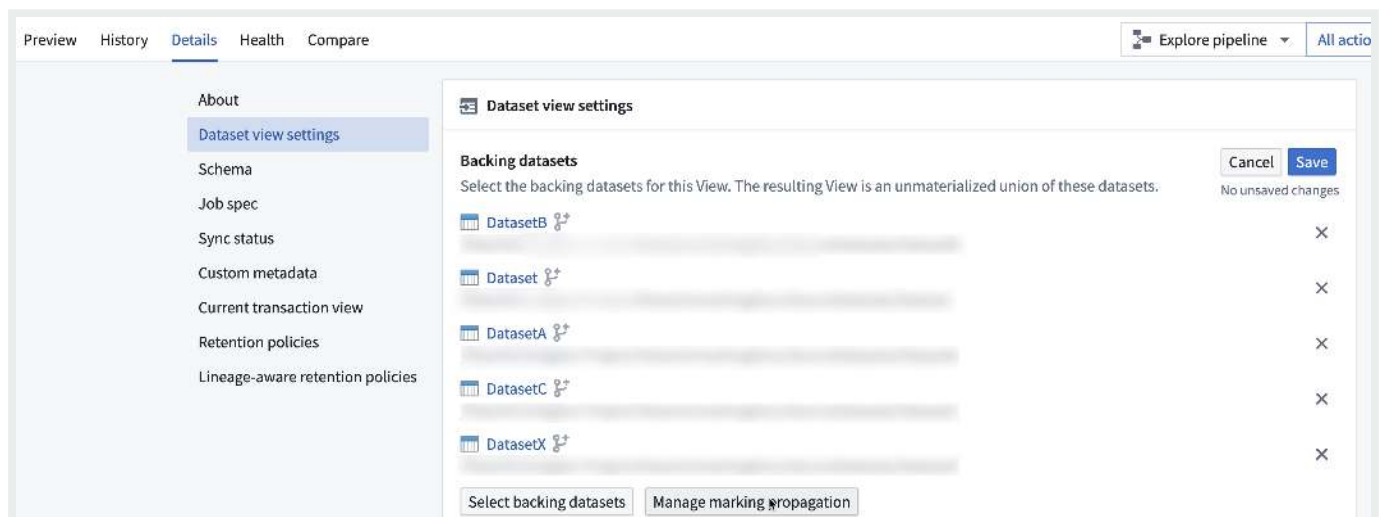
[Send feedback](#)

Remove Markings with Views

i Views do not support the removal of [Organizations](#) from datasets, only [mandatory control Markings](#).

i Removing Markings with Views may not be enabled on your Foundry enrollment. Contact Palantir Support to enable this feature.

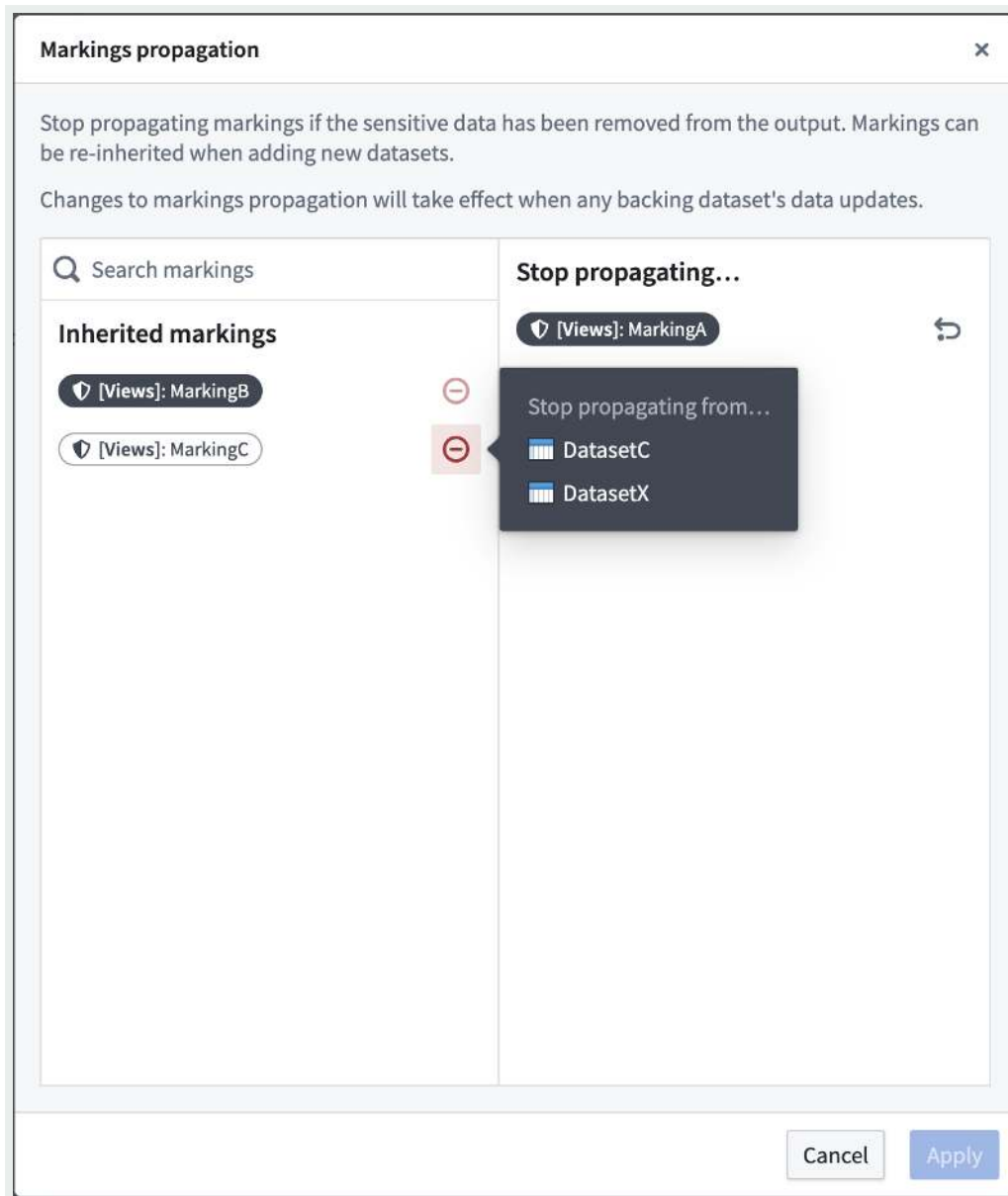
You can use Views to stop propagating Markings on datasets backing the View. You can do this by navigating to **Dataset view settings** in the **Details** tab of the View, then selecting **Edit > Manage marking propagation**.



Only users with **Remove Marking** permissions will be able to stop propagation of the Marking. Changes to Marking propagation will take effect when any updates to the backing dataset occurs.

[API Reference ↗](#)

[Send feedback](#)



Review our [guidance on removing Markings](#) for more information.

PREVIOUS
← [Change data capture \(CDC\)](#)

NEXT
[Connectivity / Data Connection / Overview](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[API Reference](#) ↗

[Terms of Use](#) ↗

[Send feedback](#)

Data connectivity & integration / Overview

Connectors

The [Data Connection](#) application is intended to provide connectivity between Foundry and your systems. This page provides an overview of the available connectors.

Filesystems and blob stores

Including:

- [Amazon S3](#)
- [OneLake and Azure Blob Filesystem \(ABFS\)](#)
- [Directory \(Agent-level filesystem\)](#)
- [FTP/FTPS](#)
- [Google Cloud Storage](#)
- [HDFS](#)
- [Server Message Block \(SMB\)](#)
- [SFTP](#)
- [SharePoint Online](#)

JDBC sources

Databases

API Reference ↗

-  Athena
- Hive

Send feedback

- IBM Db2
- Informix
- MariaDB
- [Microsoft SQL Server](#)
- MySQL
- [Oracle Database](#)
- [PostgreSQL](#)
- SQLite
- Sybase
- Vertica

Data warehouses

Including:

- [AWS Redshift](#)
- Azure Synapse
- [BigQuery](#)
- [Databricks](#)
- Netezza
- [Snowflake](#)
- Teradata

ERP and CRM

Including:

- [SAP](#)
- [Salesforce](#)
- [Oracle NetSuite](#)
- [HubSpot](#)

API Reference ↗

Streaming sources

Send feedback

Including:

- [Apache Kafka](#)
- ActiveMQ
- [Amazon Kinesis](#)
- Amazon SNS
- Amazon SQS
- Google Pub/Sub
- MQTT
- RabbitMQ
- Solace

NoSQL stores

Including:

- Amazon DynamoDB
- Apache HBase
- Azure Cosmos DB
- Cassandra
- Cockroach DB
- CouchDB
- Elasticsearch
- InfluxDB
- MarkLogic
- MongoDB
- Neo4j
- OrientDB
- Redis

REST APIs

[API Reference ↗](#)

Other source types

[Send feedback](#)

IOT / IIOT

Including:

- Amazon IoT Core
- Azure Event Hub
- Google IoT Core
- OPC-UA
- OSI PI

Geospatial systems

Including:

- ESRI / ArcGIS
- PostGIS
- Oracle Spatial DBs
- [WFS](#)

Productivity tools

Including:

- Artifactory
- [Asana](#)
- GitHub
- JIRA
- PagerDuty
- ServiceNOW
- [Slack](#)

API Reference ↗

← Connectivity / Private links (VPC connectivity) / Customer-owned domain with Private Link

Available co

Send feedback

NEXT →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Data connectivity & integration / Palantir-provided drivers for JDBC sources

Palantir-provided drivers for JDBC sources

You can connect your Foundry enrollment to various external sources using a JDBC driver that appears as Foundry sources in Data Connection. These sources are wrappers around JDBC drivers that allow for customization, and they come with recommended and required properties and links to official documentation.

If you want to upload your own JDBC driver to Foundry, review the documentation on configuring [a custom JDBC driver](#)

Setup

1. Open the Data Connection application and select **+New Source** in the upper right corner of the screen.
2. Find your specific source from the listed options. [View a complete list of Foundry-provided drivers.](#)
3. Choose to run the source capabilities on a [Foundry worker](#) or on an [agent worker](#).
4. Select **Documentation** ↗ to review official documentation for the driver source.



API Reference ↗

onal configuration prompts to continue the setup of your connector



information in the sections below.

Send feedback

Configuration options

Parameter	Required?	Description
URL	Yes	The JDBC URL that is used by the driver. Comes pre-populated with a template that may need to be modified to ensure correct behavior. Refer to the source system's documentation for the JDBC URL format, and review the Java documentation ↗ for additional information.
JDBC properties	Yes	Lists out all required and recommended properties that the driver needs. Hovering over a required or recommended property will allow you to navigate to the official documentation. You can add any additional properties by choosing the + Add property button.

JDBC properties

You can add [properties ↗](#) to your JDBC connection to configure behavior. Certain properties are mandatory for a particular driver. These mandatory properties are populated by default and must be set before you can save your source. You can also view recommended properties that you can add by selecting **+Add property** and viewing the **Recommended** section.

Hover over the name of a **Required** or **Recommended** property to visit the official documentation page for the selected driver.



[API Reference ↗](#) Country-provided driver syncs

Send feedback

SQL queries

A single SQL query can be executed per sync. This query should produce a table of data as an output, which will be saved to the output dataset in Foundry.

Exceptionally, this query can invoke stored procedures that produce data as a result. [Read below](#) for more details.



Configuration options for CData-provided drivers

Many of the Foundry-provided drivers are developed by [CData](#). CData provides full documentation for each driver including, in-depth instructions for generating credentials on the source system. You can navigate to these instructions from the documentation page for any CData driver.

The sections below contain information about CData-specific configuration options that can help you successfully connect to external systems.

Automatically perform test connection on source exploration

By default, CData drivers defer performing the connection until actual queries are made. This can result in mistaken assumptions around source exploration, since the display of static metadata tables stored in the driver may lead you to think that exploration was successful, when in reality the connection to the underlying system was not successful due to [missing credentials](#), missing egress policies, or other issues.

[API Reference ↗](#)

You can force the driver to perform a no-operation test connection, even when only exploring the source, by setting the **ConnectOnOpen** JDBC property

[Send feedback](#)

recommended to ensure that all connection issues are uncovered when exploring the source.

⚠ `ConnectOnOpen: true` can not be used alongside [OAuth 2.0 authentication](#).

`ConnectOnOpen: true` can be the reason the connection fail when the credentials used to connect are very scoped down and are not allowed perform the no-operation command used for the test connection.

Source Setup

Compute Foundry worker

Connection settings

**Azure Cosmos DB**
Driver version: 24.0.9175.0

Documentation

The underlying driver version will automatically upgrade. For many drivers provided by cdata, new driver versions frequently bring in new columns. For these drivers, we recommend [allowing schema changes in sync configuration](#), or writing batch sync queries to select specific columns.

URL

jdbc:cosmosdb:

Database connection URL

Driver class

cdata.jdbc.cosmosdb.CosmosDBDriver

Fully-qualified driver class to use for connecting to the database

JDBC properties

AccountEndpoint

Required

:

jdbcEncryptedProperty0c89f18c575e4c6fac5ca0a764198205

AccountKey

Recommended

:

Edit

ConnectOnOpen

:

true

+ Add property

Properties map passed to the JDBC driver. See [properties documentation](#)

i OAuth 2.0 Authentication is supported for this driver. See [documentation](#) for more information.

API Reference ↗

Certificate authentication

Send feedback

Many CData drivers support certificate authentication, in particular for Azure-based systems.

To connect using a certificate, you must define the following JDBC properties in addition to the connection-specific configuration requirements:

- **AuthScheme:** `AzureServicePrincipalCert`
- **OAuthJWTCertType:** `PFXBLOB`
- **OAuthJWTCert:** `base64-encoded_cert/pfx/pem_file_content`
- (Optional) **OAuthJWTCertPassword:** `password_for_the_cert/pfx/pem_file`

To transform the certificate file into Base64 format on a Windows machine, use the following command:

```
[Convert]::ToBase64String([IO.File]::ReadAllBytes("\path\to\file.pfx"))
```

Source Setup

Connection settings

**Microsoft Exchange**
Driver version: 24.0.9175.0

Documentation ↗

The underlying driver version will automatically upgrade. For many drivers provided by cdata, new driver versions frequently bring in new columns. For these drivers, we recommend [allowing schema changes in sync configuration](#), or writing batch sync queries to select specific columns.

URL

jdbc:exchange:

Database connection URL

Driver class

cdata.jdbc.exchange.ExchangeDriver

Fully-qualified driver class to use for connecting to the database

JDBC properties

Server	Required	:	https://outlook.office365.com/EWS	🔒
AuthScheme	Recommended	:	AzureServicePrincipalCert	⊖
AzureTenant	Recommended	:	YOUR_TENANT	⊖
OAuthJWTCert		:	LS0tLS1CRUdJTtBDRVJUSUZJQ0FURS0tL	⊖
OAuthJWTCertType		:	PFXBLOB	⊖

API Reference ↗

JDBC driver. See [properties documentation](#)

 OAuth 2.0 Authentication is supported for this driver. See [documentation](#) for more information.

Send feedback

Stored procedures

Some CData drivers connecting to file-based source systems like [Amazon Marketplace](#) or [Microsoft OneDrive](#) rely on the ability to invoke stored procedures to ingest data.

Configure  Sync for "OneDrive"."Files"

[View sync details](#) 

[Save](#)

☒ Append

Purely additive, yields data from previous transactions plus newly added files.

☐ Update

Replaces existing files from previous transactions based on file name.

 SQL Query 

```
1 EXECUTE DownloadFile ResourceId = 'BUT 2021 1201 0000 1000 1000 1000 1000', Encoding = 'NONE'
```

[Learn how to optimize this sync](#)

 Incremental

[Enable](#)

Incremental syncs are stateful syncs enabling append-style transactions from the same table without duplication of data. You must provide a monotonically increasing column such as a timestamp or id and an initial value for this column. An incremental sync will import rows where the value is greater than the largest already imported.

 Preview of inbound data  

	FileData STRING	Success STRING
1	              	true

Running the stored procedure will produce a table where the file content is stored as a Base64 encoded string. You can decode it in a downstream data transformation, for example in Pipeline Builder using a [Base64 decode](#) board.

API Reference 7 authentication

Some CData drivers support [OAuth 2.0](#) authorization code grant flow connections to external systems by allowing users to authenticate with their own credentials

Send feedback

and perform actions on their behalf. The [list of available drivers](#) details which ones support OAuth 2.0 authentication.

 OAuth 2.0 authentication is only supported when running the source on a [Foundry worker](#). Sources running on an [agent worker](#) do not support OAuth 2.0 authentication.

To use OAuth 2.0 authentication:

1. Add the **CallbackUrl** JDBC property to the source configuration. The property value will auto-fill with a URL of the form `https://<YOUR_FOUNDRY_URL>/workspace/oauth2-clients/callback`.
2. In the external system, register a custom OAuth application and provide the callback URL.
 - The specifics of how to register a custom OAuth application will depend on each external system. For [GitHub ↗](#), for example, navigate to **Settings > Developer Settings > OAuth Apps**. Then select **New OAuth App**.
3. Copy the generated **OAuth client id** and **OAuth client secret** and paste them in your source configuration JDBC properties.

[API Reference ↗](#)

[Send feedback](#)



GitHub

Driver version: 24.0.9329.0

Documentation ↗

The underlying driver version will automatically upgrade. For many drivers provided by cdata, new driver versions frequently bring in new columns. For these drivers, we recommend [allowing schema changes in sync configuration](#), or writing batch sync queries to select specific columns.

URL

jdbc:github:

Database connection URL

Driver class

cdata.jdbc.github.GitHubDriver

Fully-qualified driver class to use for connecting to the database

JDBC properties

CallbackUrl

Recommended

:

https://<YOUR_FOUNDRY>/oauth2-cli

⊖

InitiateOAuth

Recommended

:

REFRESH

⊖

OAuthClientId

Recommended

:

⊖

OAuthClientSecret

Recommended

:

Edit

⊖

+ Add property

⌵

Properties map passed to the JDBC driver. See [properties documentation](#)

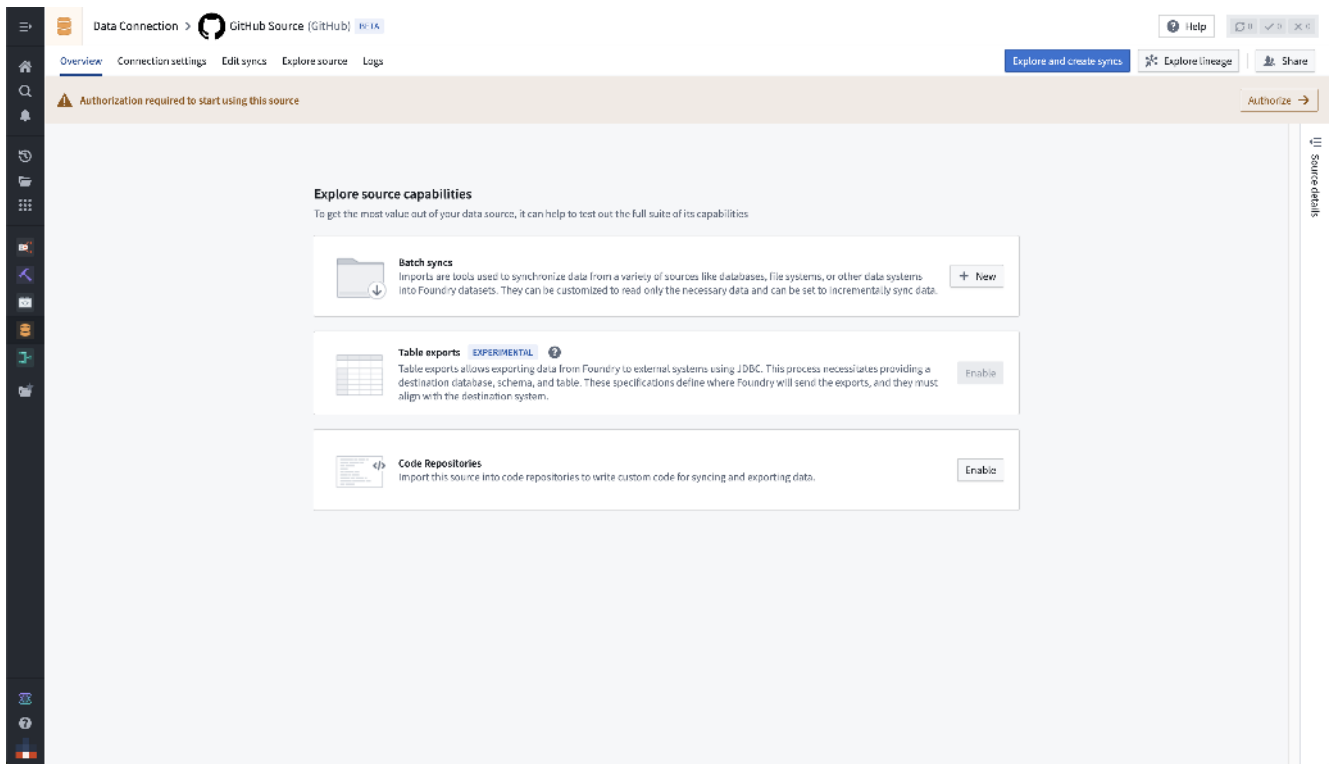
ⓘ

OAuth 2.0 Authentication is supported for this driver. See [documentation](#) for more information.

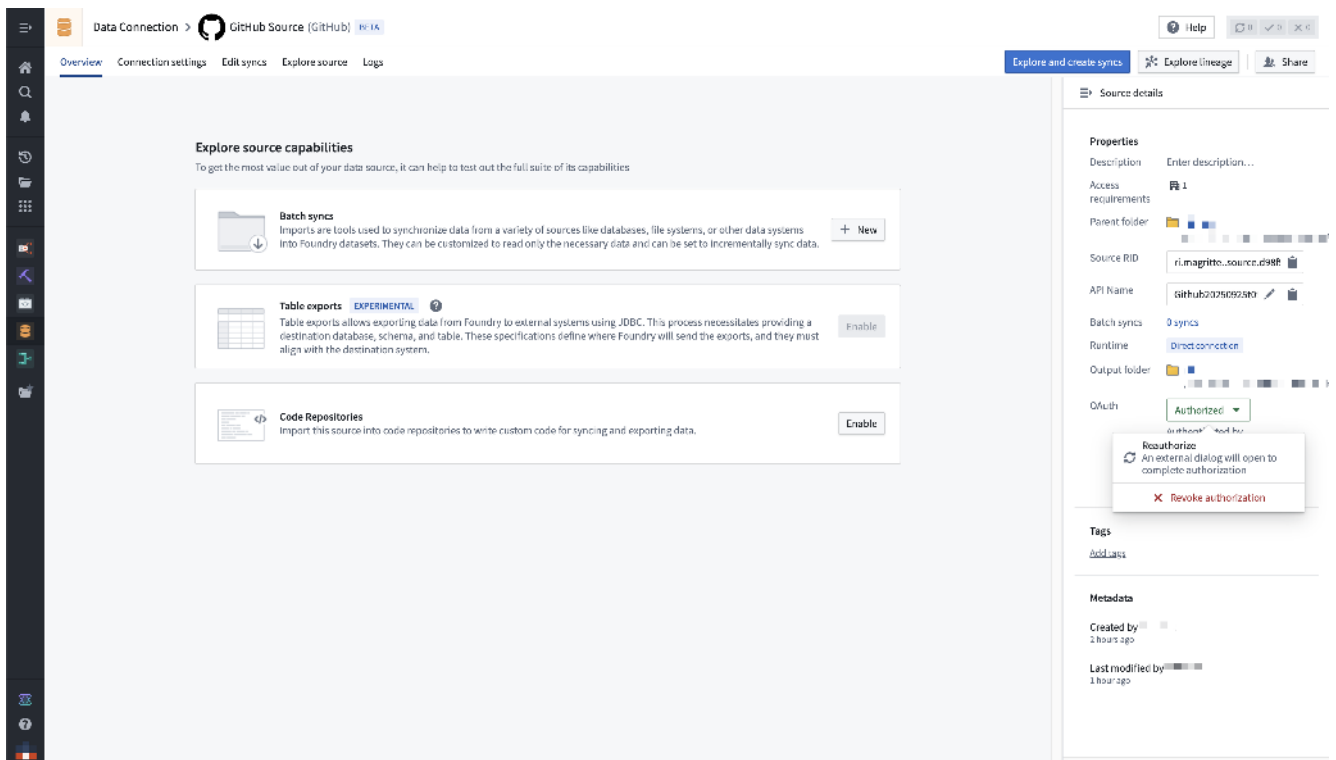
- Once the configuration is saved, navigate to the source overview page. Select **Authorize** in the top banner labeled **Authorization required to start using this source** to start the OAuth flow.

API Reference ↗

Send feedback



You can renew or revoke authorization from the right-side panel on the source overview page.



API Reference ↗

i Starting, renewing, or revoking the OAuth flow requires the **Owner** role by default. Users with only **Viewer** or **Editor** will see a permission error when selecting **Authorize**. If you need to grant these operations without granting full

Send feedback

Owner permissions, define a custom role that includes the source administration operations.

Available drivers

Driver	Supports OAuth 2.0 Authentication
Act! CRM	FALSE
Act-On	TRUE
ActiveCampaign	FALSE
Acumatica	FALSE
Adobe Analytics	TRUE
Adobe Commerce	FALSE
ADP	TRUE
Airtable	TRUE
AlloyDB	FALSE
Amazon DynamoDB	FALSE
Amazon Marketplace	TRUE
Apache CouchDB	FALSE
Apache HBase	FALSE
Apache Hive	FALSE
Apache Phoenix	FALSE
Authorize.Net	FALSE
Avalara	FALSE
Azure Active Directory	FALSE
Azure Cosmos DB	TRUE
Azure Data Catalog	TRUE
API Reference ↗	TRUE
Azure Synapse	TRUE

Send feedback

Driver	Supports OAuth 2.0 Authentication
Azure Table Storage	TRUE
Basecamp	TRUE
BigCommerce	TRUE
Blackbaud Raisers Edge NXT	TRUE
Bugzilla	FALSE
Bullhorn CRM	TRUE
Cassandra	FALSE
Certinia	TRUE
Cloudant	TRUE
CockroachDB	FALSE
Confluence	TRUE
Couchbase	FALSE
Databricks	FALSE
DocuSign	TRUE
Domino	TRUE
eBay	TRUE
eBay Analytics	TRUE
EnterpriseDB	FALSE
Epicor Kinetic	TRUE
Exact Online	TRUE
Facebook	TRUE
Facebook Ads	TRUE
FreshBooks	TRUE
Freshdesk	FALSE
GitHub	TRUE
API Reference ↗	TRUE
Google Ad Manager	FALSE
Google Ads	FALSE

Send feedback

Driver	Supports OAuth 2.0 Authentication
Google Analytics	FALSE
Google Calendar	FALSE
Google Campaign Manager	TRUE
Google Contacts	TRUE
Google Data Catalog	TRUE
Google Directory	TRUE
Google Drive	TRUE
Google Search	FALSE
Google Spanner	TRUE
GraphQL	TRUE
Greenplum	FALSE
Highrise	TRUE
IBM Cloud Data Engine	TRUE
IBM Cloud Object Storage	TRUE
Instagram	TRUE
Jira Service Management	TRUE
Kintone	FALSE
LDAP	FALSE
LinkedIn	TRUE
LinkedIn Marketing Solutions	TRUE
Mailchimp	TRUE
Marketo	TRUE
MarkLogic	FALSE
Microsoft Access	FALSE
Microsoft Ads	TRUE
API Reference ↗	FALSE
Microsoft Dataverse	FALSE
Microsoft Dynamics 365	TRUE

Send feedback

Driver	Supports OAuth 2.0 Authentication
Microsoft Dynamics 365 Business Central	TRUE
Microsoft Dynamics CRM	TRUE
Microsoft Dynamics GP	FALSE
Microsoft Dynamics NAV	FALSE
Microsoft Excel	TRUE
Microsoft Excel Online	TRUE
Microsoft Exchange	TRUE
Microsoft Office 365	TRUE
Microsoft OneDrive	TRUE
Microsoft OneNote	TRUE
Microsoft Planner	TRUE
Microsoft Power BI® XMLA	TRUE
Microsoft Project	TRUE
Microsoft SharePoint	TRUE
Microsoft SharePoint Excel	FALSE
Microsoft SQL Server	FALSE
Microsoft SQL Server Analysis Services	FALSE
Microsoft Teams	TRUE
Monday	TRUE
MYOB	TRUE
OData	TRUE
Odoo	FALSE
Oracle	FALSE
Oracle Eloqua	TRUE
Oracle Fusion Cloud Financials	FALSE
API Reference > Oracle Fusion Cloud HCM	FALSE
Oracle Fusion Cloud SCM	FALSE
Oracle Sales	FALSE

Send feedback

Driver	Supports OAuth 2.0 Authentication
Oracle Service Cloud	FALSE
Outreach	TRUE
Paylocity	TRUE
PayPal	TRUE
Pinterest	TRUE
Pipedrive	TRUE
Presto	FALSE
Quickbase	FALSE
QuickBooks Desktop	FALSE
QuickBooks Online	TRUE
QuickBooks POS	FALSE
Raisers Edge NXT	TRUE
Reckon	FALSE
Reckon Accounts Hosted	TRUE
Redis	FALSE
Redshift	FALSE
RSS	FALSE
Sage 200	TRUE
Sage 300	FALSE
Sage 50 UK	FALSE
Sage Business Cloud Accounting	TRUE
Salesforce Marketing Cloud	TRUE
Salesforce Marketing Cloud Account Engagement	TRUE
Salesloft	TRUE
API Reference  ment	FALSE
SAP Business One	FALSE
SAP BusinessObjects BI	FALSE

Send feedback

Driver	Supports OAuth 2.0 Authentication
SAP ByDesign	FALSE
SAP Cloud for Customer	TRUE
SAP Concur	TRUE
SAP Fieldglass	TRUE
SAP HANA XSA	TRUE
SAP SuccessFactors	TRUE
SAS Data Sets	TRUE
SAS Xpt	TRUE
SendGrid	FALSE
ServiceNow	FALSE
ShipStation	FALSE
Shopify	TRUE
SingleStore	TRUE
Slack	FALSE
Smartsheet	TRUE
Snapchat Ads	TRUE
Snowflake	FALSE
Spark SQL	FALSE
Splunk	FALSE
Square	TRUE
Streak	FALSE
Stripe	TRUE
SugarCRM	TRUE
SuiteCRM	TRUE
SurveyMonkey	TRUE
API Reference ↗	FALSE
Tableau CRM Analytics	TRUE
Tally	FALSE

Send feedback

Driver	Supports OAuth 2.0 Authentication
TaxJar	FALSE
Trello	TRUE
TSheets	TRUE
Twilio	FALSE
Twitter Ads	TRUE
Veeva Vault	TRUE
Wave Financial	TRUE
WooCommerce	TRUE
WordPress	TRUE
Workday	FALSE
xBase	FALSE
Xero	TRUE
Xero WorkflowMax	TRUE
YouTube Analytics	TRUE
Zendesk	TRUE
Zoho Books	TRUE
Zoho Creator	TRUE
Zoho CRM	TRUE
Zoho Inventory	TRUE
Zoho Projects	TRUE
Zuora	TRUE

[← PREVIOUS](#)
 Available connectors / Other source types

NEXT [→](#)
 SAP / Overview

[API Reference ↗](#)

© 2020 Parantur technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

Send feedback

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Data connectivity & integration / Resource guides / Streaming

Streaming resource guide

This page lists the resources you may need to reference when implementing an end-to-end streaming workflow.

Data Connection supports syncing data from a wide variety of streaming platforms into Foundry streaming datasets, which can then be used in [streaming pipelines](#). Streaming syncs enable data to flow into Foundry with low latency and high throughput to support real time decision-making processes.

There are two ways to sync data from streams into Foundry:

- Data Connection supports pulling records from streaming platforms into Foundry. As with batch syncs, data is read from a stream and synced to Foundry using only unidirectional connections using the [agent architecture](#).
- If desired, Foundry enables pushing records from a stream directly into a Foundry stream via the *stream proxy*.

Foundry can connect to many sources of streaming data. Sources with dedicated connectors include:

- [Apache Kafka](#)
- [Amazon Kinesis](#)
- [Amazon SQS](#)
- [Aveva PI](#)
- [Google Pub/Sub](#)

For sources without a dedicated connector, you can connect to them using [external transforms](#). This includes sources such as:

- [ActiveMQ](#)

Send feedback

- **Amazon SNS**
- **IBM MQ**
- **RabbitMQ**
- **MQTT [Beta]**
- **Solace**

This page lists the resources you may need to reference when implementing an end-to-end streaming workflow.

1. Core concepts

We recommend reviewing the following introductory concept page to understand what streams are, how they are stored, and how they are processed.

- [Streams](#)
- [Flink in Foundry streaming](#)

2. Overview

These pages will offer a broader scope of the various points to consider when determining if streaming is right for your use case or when deploying production streams.

- [Streaming pipelines overview](#)
- [Comparison: Streaming vs. batch](#)
- [Performance considerations](#)
- [Streaming compute usage](#)
- [Streaming profiles](#)
- [Stream monitoring](#)
- [Streaming keys](#)
- [Streaming stateful transforms](#)
- [Stream debugging](#)

API Reference ↗

3. Connect to data sources

Send feedback

You will need to complete one of the following workflows to connect your external data sources to Foundry for streaming. We recommend reviewing both options to understand possible benefits and limitations for your use case.

- [Set up a streaming sync](#)
- [Push data into a stream](#)

4. Transform your streaming data

You can use [Pipeline Builder](#) to transform your live data. Outputs of your Pipeline Builder transforms will still be streaming datasets that you can [use in real time throughout Foundry](#).

- [Create a streaming pipeline with Pipeline Builder](#)
- [Integrate your stream with the Ontology](#)

5. Monitor streaming pipelines [Beta]

Set up alerting around your pipeline's health.

- [Stream monitoring](#)

6. Development tools

Here, you can find tools to improve development of streaming pipelines.

- [Reset stream](#)

← PREVIOUS
Connector type reference / SAP / FAQ

NEXT →
Flink fundamentals

API Reference ↗

© 2020 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

Send feedback

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Data connectivity & integration / Resource guides / Flink fundamentals

Flink fundamentals

Apache Flink is a distributed computation engine capable of handling unbounded datasets with low latency, allowing it to handle common streaming workflows. Foundry streaming uses Flink as the underlying engine to execute user code and other in-platform streaming applications such as hydrating the Ontology in real time and streaming time series ingestion.

In order to understand whether you need additional job configuration for your streaming use case, it is helpful to have some understanding of how Flink works.

More detailed documentation about Flink can be found at the [Flink homepage ↗](#).

Flink jobs

All streaming jobs are described as a series of operations acting on a set of data sources, and which write results to a data sink. These operations include things like aggregations, joins, and other row-level actions like string parsing or arithmetic. Each operation is represented by Flink as the **Operator** abstraction. In Flink, sources and sinks are also described by operators.

Flink jobs are represented internally in terms of “job graphs” or “logical graphs.” Job graphs are directed graphs with nodes made up of operators and where the edges define the relationships between operators. When a job is submitted to Flink, it creates the job graph. A preview of your Flink job’s job graph is rendered in the **Details** section of jobs in the Foundry Job Tracker.

When submitting jobs, Flink translates the logical graph into a **physical graph**, a representation of how operators will be executed on the compute runtime. Physical graphs are made up of **tasks**, which are the basic units of work in a Flink job. Each task represents either one instance of an operator or many operators chained together.

API Reference ↗

Send feedback

Job Managers and Task Managers

Like Spark, Flink's runtime architecture includes different types of workers. Where Spark uses the driver to coordinate and manage jobs and executors to perform job tasks, Flink uses **Job Managers** and **Task Managers**, which fulfill roles roughly analogous to Spark drivers and executors.

The Flink Job Manager is responsible for scheduling tasks and allocating resources for tasks, handling finished or failed tasks, coordinating [job checkpoints](#) and failure recovery, as well as providing programmatic access to job information. Typically there is only one active Job Manager – the leader – at any given time, with backup(s) kept warm in the case of an unrecoverable error.

The Flink Task Manager is responsible for the execution of tasks as well as buffering and exchanging data between streams. There is always at least one Task Manager, but there may be more in order to parallelise stream processing. When Task Managers need to handle very large records they may require additional resources. If your stream has extremely high throughput, you may need to increase your job's parallelism, which results in increasing the number of Task Managers.

Job state

While some Flink operations only need to look at single events in isolation, others need to remember information across multiple events. These are [stateful operations](#). Some examples of stateful operations are:

- **Aggregations:** For example, counting the total number of events over a rolling five minute window, or calculating the running average of all known events.
- **Joins:** The execution engine needs to know about previously-seen events in order to join them with events that are currently being ingested.

The information required for stateful operations is known as *job state*, and is stored by Flink using a state backend. State is managed and stored by Task Managers and coordinated by the [API Reference](#)  the form of [checkpoints](#). When you have a larger state (such as when an aggregation operation has a very large window), your Job Managers and Task Managers may require additional resources.

Send feedback

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Reset stream

Resetting an ingest stream clears the existing records from a stream and provides an opportunity to change the stream's schema, throughput, and configuration values. This can be helpful during stream pipeline development since you can clear and update an ingest stream without replacing existing references of the stream within your Pipeline Builder pipelines.

Note that resets are only available for ingest streams. Downstream consuming pipelines of the ingest stream must be replayed after a reset.

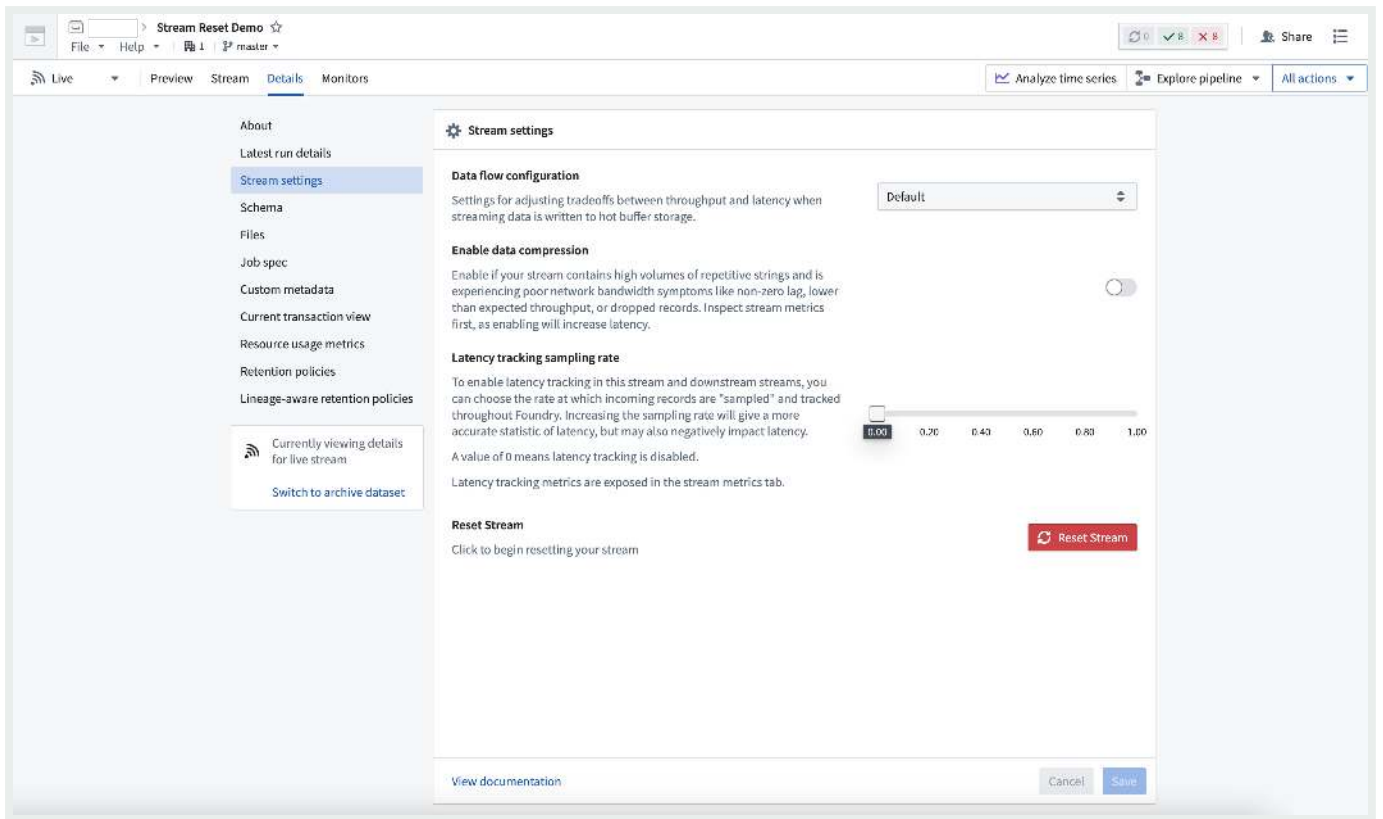
- [Push-based ingestion](#) requires updating the POST URL to reference the new stream `viewRid` value.
- [Magritte-based ingestion](#) requires an agent rebuild.

 Resetting streams can have irreversible effects on data. We do not recommend resetting production ingest streams because existing records will be lost.

To reset a stream, follow the instructions below:

1. Open an ingest stream.
2. Select the **Details** tab.
3. In the **Stream Settings** section, select **Reset stream**.

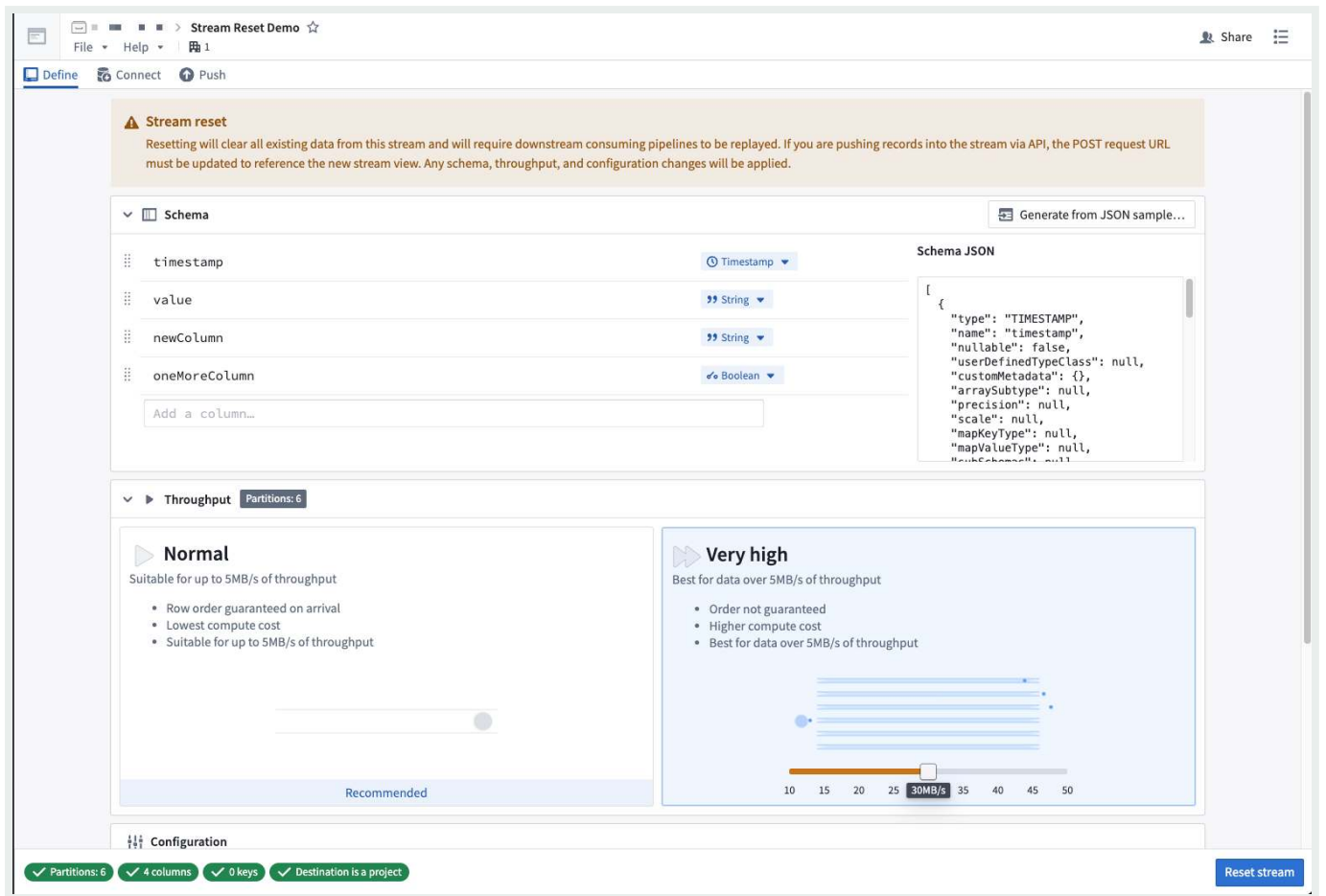
[API Reference ↗](#)[Send feedback](#)



4. You will be redirected to the stream reset page. On this page, you can optionally update the schema, throughput, or configuration values. If you only want to clear data from the stream, leave those sections unchanged.

[API Reference ↗](#)

[Send feedback](#)



5. Select **Reset stream** to initiate the stream reset.

PREVIOUS
← [Flink fundamentals](#)

NEXT
[Stream monitoring](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#)

[Privacy Statement](#)

[Terms of Use](#)

[API Reference](#)

[Send feedback](#)

Data connectivity & integration / Resource guides / Stream monitoring

Stream monitoring

Stream monitoring enables alerting around your stream's health.

A stream is considered "healthy" when it is:

- Ingesting new records from source systems.
- Processing and writing records into user-facing Foundry applications.
- Regularly persisting state, regardless of the presence of incoming records. This is *only* applicable for pipelines.

You can monitor the health of a pipeline by setting ingest and output alerts that fire when the number of records ingested or output, over a time period, falls below a user-defined threshold.

For example, an ingest monitor could alert if your stream has ingested zero records over the last five minutes, while an output monitor could alert if your stream has output less than 1000 records over the last 30 minutes.

Creating monitors

1. Open the Data Health application.
2. Select the **Monitoring Views** tab.
3. Create or select an existing Monitoring View.
4. Navigate to the **Manage Monitors** tab.
5. Select **Add new alert**.

API Reference ↗ Monitoring rules.



Send feedback

Configuring ingest monitors

 You should only apply monitors to streams with a deterministic flow of records.

Various fixed time periods, such as five and thirty minutes, are supported.

Ingest monitors ensure that:

- Your source system is generating and sending records to Foundry.
- Foundry is writing records into the ingest stream's live view.

To configure this:

1. Follow the [creating monitors](#) instructions.
2. Select **Streaming dataset** as the resource type.
3. Add the ingest streams to monitor and select **Next**.
4. Select **+ Add** in the **Ingest stream monitors** card.
5. Select **Records ingested** monitor with a 5 or 30 minute duration.
6. Set your threshold and select **Next**.
7. Review your monitor and select **Save monitoring rules**.

Examples:

- **Records ingested** with a five-minute duration and threshold of zero: Alerts when your stream has written zero records to the live view over the last five minutes.
- **Records ingested** the thirty-minute duration and threshold of 1000: Alerts when your stream has written less than or equal to 1000 records to the live view over the last 30 minutes.

Configuring pipeline monitors

You can use the same workflow used to [configure ingest monitors](#) for pipeline monitors, but there are some notable differences in the monitors available for streaming pipelines outlined below.

[API Reference ↗](#)

Send feedback

Checkpoint metrics

- **Checkpoint Liveness** : Alerts if a stream has not checkpointed in the configured amount of time. This monitor is *highly recommended* for production streams, as it is a high-signal indicator of a degraded performance. [Learn more about checkpointing](#).
- **Last Checkpoint Duration** : Alerts if checkpoint duration has increased beyond a configured threshold.
- **Checkpoint Trigger Failures** : Alerts if checkpoints fail to trigger consecutively.
- **Consecutive Checkpoint Failures** : Alerts if checkpoints fail consecutively.

Performance metrics

- **Total Lag** : Alerts if a job falls behind on the input by the configured amount of records, which signals degraded performance.
- **Total Throughput** : Alerts if the amount of records per checkpoint is under or over a configured threshold, which indicates changes in the upstream input.
- **Utilization** : Alerts if the percentage of the stream's utilized capacity is above a set threshold, which you can configure using [streaming profiles](#).

Configuring output monitors

Output monitors ensure that your pipeline is:

- Processing ingested records.
- Writing those processed records into user-facing Foundry applications.

i While stream monitoring is in the beta phase, [time series](#) and [geotemporal series](#) are the only data formats able to be monitored.

Time series

API Reference ↗

To monitor records written to [time series](#), you will set alerts on the tim

Send feedback

1. Follow the [creating monitors](#) instructions.
2. Select **Time series sync** as the resource type.
3. Add the time series syncs to monitor and select **Next**.
4. Select **+ Add** in the **monitors** card.
5. Select **Points written to Epoch** with a 5 or 30 minute duration.
6. Set your threshold and select **Next**.
7. Review your monitor and select **Save monitoring rules**.

Examples:

- **Points written to Time Series DB** with a five minute duration and threshold of zero: Alerts when your time series sync has written zero records over the last five minutes.
- **Points written to Time Series DB** with a 30 minute duration and threshold of 1000: Alerts when your time series sync has written less than or equal to 1000 records over the last 30 minutes.

Geotemporal observations

To monitor [geotemporal observations](#), you will set alerts on the backing observation dataset.

1. Follow the [creating monitors](#) instructions.
2. Select **Geotemporal observations** as the resource type.
3. Add the observation datasets to monitor and select **Next**.
4. Select **+ Add** in the **monitors** card.
5. Select **Geotemporal observations sent** with a 5 or 30 minute duration.
6. Set your threshold and select **Next**.
7. Review your monitor and select **Save monitoring rules**.

Examples:

- **Geotemporal observations sent** with a five minute duration and threshold of zero: Alerts when your geotemporal sync has sent zero observations to geotime over the last five minutes.

API Reference ↗ **Geotemporal observations sent** the 30 minute duration and threshold of 1000: Alerts when your geotemporal sync has sent less than or equal to 1000 observations to geotime over the last 30 minutes.

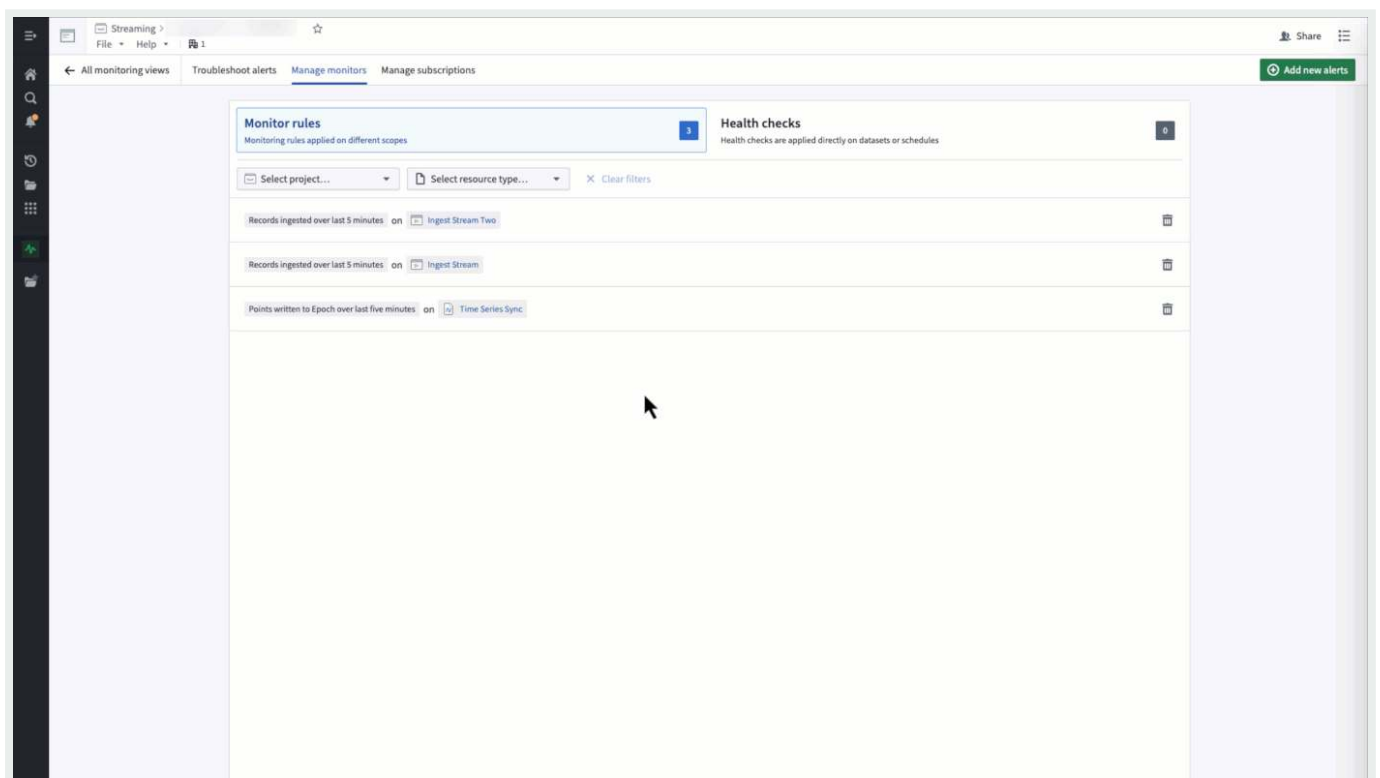
Send feedback

- Ensure you set the monitor on the backing observation dataset instead of the errors dataset.
- `Geotemporal observations sent` only ensures the records were sent from geotime ingest. This does not guarantee that the geotime service has processed the record after ingestion.

Viewing metrics

To view the metrics underlying a monitor, select the monitor rule in row in the monitoring view's **Manage monitors** tab.

Metrics are only available for streaming or time series monitors with a single target scope.



Monitor notifications

By [API Reference ↗](#) notifications through the monitoring view's **Manage subscriptions** tab.

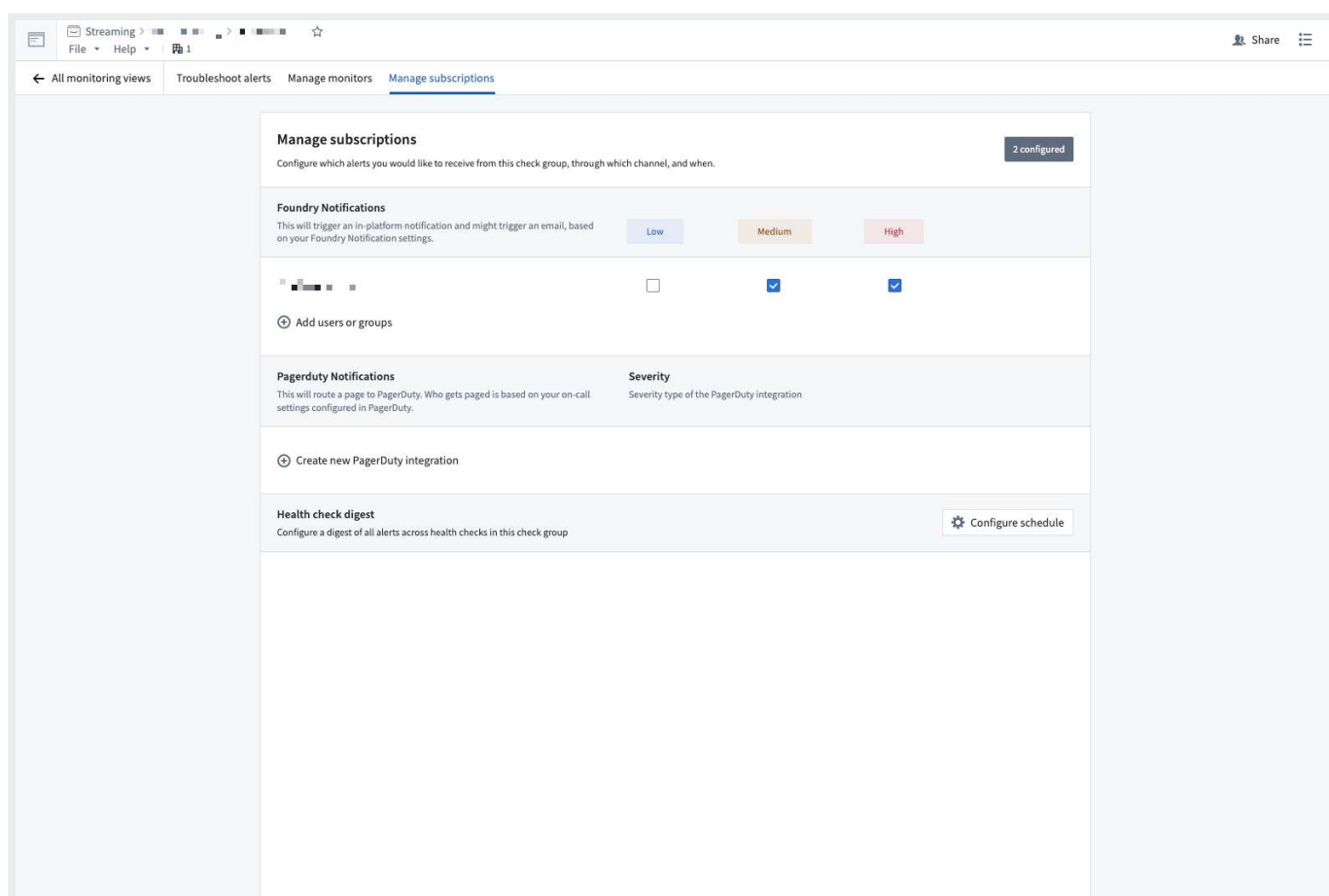
To set email alerts:

Send feedback

1. Select **Add users to group**.
2. Search and select a user.
3. Select the severities to get notified for.

To set PagerDuty alerts:

1. Select **Create new PagerDuty integration**.
2. Set the **Integration Name**.
3. Create an integration in PagerDuty.
4. Copy and paste the integration key into the **Integration Key**.
5. Select the **Severity** of alerts.



Firing alerts

You can view firing alerts in the **Troubleshoot alerts** tab of your monitoring view.

API Reference ↗

Send feedback

Streaming

FileHelp

1

Share

All monitoring views

Troubleshoot alerts

Manage monitors

Manage subscriptions

Alert summary

200

View options

Grouping options

Filter by type (for checks only)

ALERT	RESOURCE	FAILURE REASON	REPORTED
Points written to Epoch over last five minutes	Monitor	Time Series Sync	Alerts if the number of points written to Epoch over the last five minute ...
Records ingested over last 5 minutes	Monitor	Ingest Stream Two	Alerts if the number of records ingested into the raw stream's live view o...

PREVIOUS

← Reset stream

Streaming profiles

NEXT →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

[Cookies Settings](#)

[API Reference ↗](#)

Send feedback

Streaming profiles

When streaming with the Palantir platform, all aspects of configuring Flink runtime environments are managed for you, including reasonable and cost-effective defaults for Flink job configurations. In most cases, you do not need to configure anything yourself to start running a job on the Flink computation engine in a performant manner.

However, sometimes your job will require additional resources than what Foundry streaming provides by default. Such resources might include additional parallelism (if your stream has very high throughput, for example) or additional resources for JobManagers or TaskManagers that might be required if you have a very large state or very large records.

For these cases where your job requires additional resources, Palantir offers a set of job profiles that can be used to adjust specific configuration options. Note that we provide access to a limited set of the total available Flink configurations since the Palantir streaming platform can manage most things for you.

Similar to [Spark profiles in Code Repositories](#), each streaming job profile manages a specific component of the streaming job's resource requirements, and streaming job profiles can be composed with each other to meet your use case requirements.

When to use streaming profiles

Typically, you should never need to use streaming profiles in the platform since our default streaming configurations are designed to be performant and cost-effective for a majority of use cases. However, we do recommend using streaming profiles only if you are encountering s

API Reference ↗

- ⋮

🔍
- throughput
- Very large records

Send feedback

- Very large state requirements

You can decide which additional profiles are needed by considering your job requirements, logs and error messages available within the JobTracker interface, and a basic understanding of Flink.

Set up a streaming profile

The streaming profile set up interface will differ depending on where you are using the Foundry streaming platform.

In most cases, when setting up a streaming use case, you will notice a selection box that lists all streaming profiles available to you. For example, in [Pipeline Builder](#), you can see the list of configurable streaming profiles by selecting the build settings next to the **Deploy** button:

Compute profile

☐

Extra Small
Extra small profile suitable for the cost sensitive streaming jobs. This profile won't run with an extra task manager, which can lead to daily downtime on the order of minutes.

▼

☐

Small
Small profile suitable for the majority of streaming jobs.

▼

☐

Medium
Medium profile suitable for jobs with state or data volumes too large for the small profile.

▼

☐

Large
Large profile suitable for jobs with a large number of partitions. Large

▼

1

Builds with larger profiles may complete faster but may incur a higher compute cost.

[API Reference ↗](#)

[Send feedback](#)

Additionally, Pipeline Builder allows you to combine different aspects of these profiles together with the **Advanced** profile option:

Compute profile

☐ XLarge
Extra-Large profile suitable for jobs with a large number of partitions,
large state, or high data volumes.

☒ Advanced
A profile where you can configure your own settings

<i>i</i> Task Manager Memory	1 GB ▾
<i>i</i> Extra Task managers	Yes ▾
<i>i</i> Job Parallelism	1 ▾
<i>i</i> Job Manager Memory	1 GB ▾

These specifications may change between builds.

i Builds with larger profiles may complete faster but may incur a higher compute cost.

Share

Apply

ing pipelines

Advanced ▾

>

▾

🔍

📅

⚙️

▶

Note that the selection of streaming profiles available to you may differ based on which application you are using and any other security or visibility requirements applied to your working environment.

Manage streaming profiles

Because streaming profiles can be used to determine the total number of resources allocated to perpetually running streaming jobs, several administrative controls are placed on these profiles to manage [streaming costs](#).

Project references

API Reference ↗

For example, Projects define the conceptual boundary around related work and a security boundary for applying and managing access. The default application and administrative controls is at the Project level. Typically, using data or resources requires

[Send feedback](#)

that they are present in or imported into your current Project, granting them a [Project Reference](#). You can see all references in a Project in the **References** section of the Project workspace side panel.

The primary administrative control applied to streaming profiles is the requirement that they are added as Project references to the same Project as the related streaming pipeline or application. If you try to use a streaming profile that is not imported to your Project, your job will fail with an error indicating this missing requirement.

By using Project references as the administrative control, more advanced users can be granted the ability to import profiles into Projects on behalf of other users. This allows for administrators to control where streaming profiles are used at a granular level while allowing more operational users to use the profiles to which they have been granted access.

Import profiles as Project references

Typically, profiles will be imported automatically into Projects within the relevant application by using the profile selector component. For example, the profile selector in Pipeline Builder will import the selected profiles automatically.

All users are able to import most profiles as Project references into their Projects, provided they have sufficient permission to import resources. This generally means that the user has been assigned a Role which grants them the `compass:import-resource-to` permission on the Project. You can find Role configurations by navigating to the **Roles** tab in your user **Settings**. Search for the permission using the `Filter operations...` search tool on the page.

To view a list of all available streaming profiles, visit the “Streaming profiles” tab in the “Enrollment Settings” section of Control Panel. From here you can pick any specific profile and see all Projects (to which you have access) where that profile has been added as a Project reference, as well as import it into new Projects. You can also remove references to a profile from an Project that it has been imported into; but note **this will break any streaming jobs in that Project that rely on that profile**, because we require that profiles be imported into the same Project as a streaming job in order to be used.

API Reference ↗

Use large streaming profiles

Send feedback

For profiles that grant a large number of resources, Project references must be created using the **Streaming profiles** tab in the **Enrollment Settings** section of [Control Panel](#). This setting is enabled only for users who are designated as `Enrollment Resource Administrators`. These administrators can be assigned in the **Enrollment permissions** tab and can import restricted profiles into any Project to which they have access.

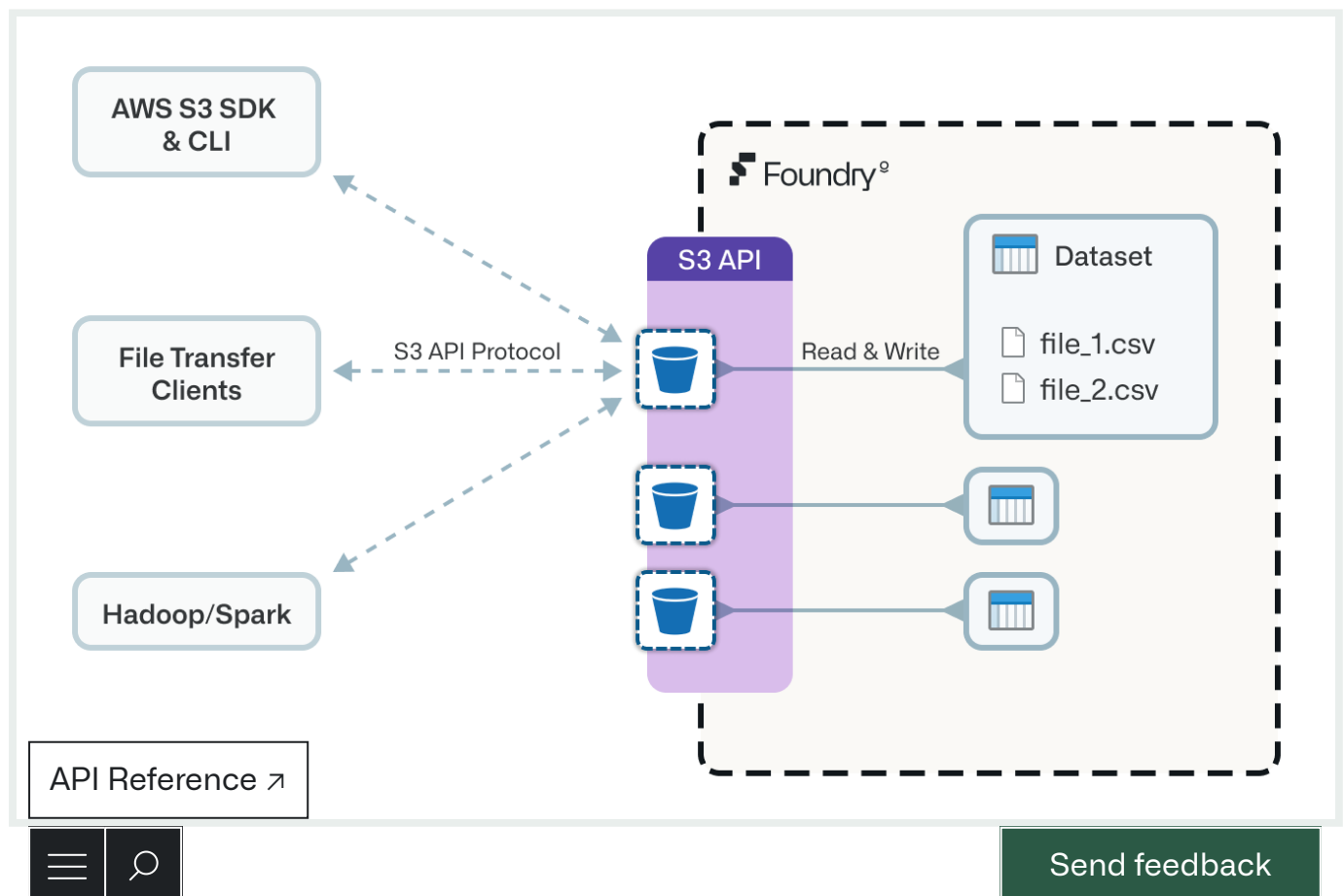
Once an administrator imports a profile into a Project, any user who has access to that Project may then use that profile.

Data connectivity & integration / S3-compatible API for Foundry datasets

S3-compatible API for Foundry datasets

The S3-compatible API for Foundry datasets allows you to interact with Foundry datasets as though they are S3 buckets. Learn how datasets behave when accessed through the API, and view the [setup guide](#) and [examples](#).

Foundry exposes a subset of the [Amazon Simple Storage Service \(S3\) API](#), allowing you to interact with Foundry datasets using clients that know how to speak to S3 storage services. Examples include the AWS CLI, AWS SDKs, Hadoop S3 filesystem, and Cyberduck.



The S3 API is not fully implemented as not all S3 concepts map naturally to concepts in Foundry. For example, creation and deletion of buckets (which represent Foundry datasets) is not currently supported; datasets should be created in Foundry ahead of using the API. However, the majority of file read/write/delete workflows are supported, including multipart uploads. See [Supported actions](#) for a list of which S3 actions are supported.

Concepts

This section outlines how S3 concepts map to Foundry dataset concepts.

S3 buckets correspond to Foundry datasets

S3 buckets correspond to **Foundry datasets**, with the bucket name being the Foundry dataset's unique identifier (for example, `ri.foundry.main.dataset.bafb7e96-84f1-4d23-a4a5-40b17c6912e7`).

S3 object keys correspond to the **logical paths of files within a Foundry dataset** (for example, `top-level-file.csv` or `subfolder/nested-file.csv`).

 Only Foundry datasets can be used as S3 buckets. Compass folders and projects are organizational containers, not datasets, and cannot be used as a bucket name. To keep uploaded files organized separately, such as separating application logs from other outputs, write to a nested object key path within an existing dataset (for example, `logs/nested-file.csv`). Alternatively, create a new dataset for that purpose. Avoid creating a new Compass folder for this.

Branches

The API supports accessing dataset branches with alphanumeric names (containing only `a-z`, `A-Z`, or `0-9`) or the special characters `-` and `_` . To specify a branch, modify the `<dataset-rid>` .
ending the branch name, separated by a period: `<dataset-rid>` .
branch is specified, the default branch is used.

API Reference ↗

Send feedback

For example, to access the `mybranch` branch of the dataset with RID `ri.foundry.main.dataset.bafb7e96-84f1-4d23-a4a5-40b17c6912e7`, use the bucket name `ri.foundry.main.dataset.bafb7e96-84f1-4d23-a4a5-40b17c6912e7.mybranch`.

S3 bucket name validation imposes the character restrictions previously described. If your branch contains characters that are not allowed such as `/`, you can encode the branch name using Base64-encoding. You should omit any trailing `=` characters. For example, `feature/my-branch` is Base64 encoded as `ZmVhdHVyZS9teS1icmFuY2g=`. `ZmVhdHVyZS9teS1icmFuY2g` (without the `=` character) can be used to reference this branch in the bucket name.

The API does not support branch creation; specified branches must already exist on the dataset.

Transactions

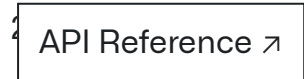
The API supports accessing dataset transactions in a similar way to branches. This allows users to access historical versions of a dataset. For example, to access the `ri.foundry.main.transaction.0cdf8c9-f595-4859-a194-7daecff9d6fe` transaction of the dataset with RID `ri.foundry.main.dataset.bafb7e96-84f1-4d23-a4a5-40b17c6912e7`, use the bucket name `ri.foundry.main.dataset.bafb7e96-84f1-4d23-a4a5-40b17c6912e7.ri.foundry.main.transaction.0cdf8c9-f595-4859-a194-7daecff9d6fe`.

Only committed transactions may be accessed in this way. As a result, the bucket will be read-only; it is not possible to put or delete objects when using a bucket name that includes a transaction identifier.

Transaction management

S3 does not have the notion of transactions, so [Foundry dataset transactions](#) are automatically handled with the following behavior:

1. Transactions are lazily opened when users write or delete files.

 writing multiple files concurrently or deleting multiple files concurrently. Files will be modified in the same open transaction, with writes using `UPDATE` transactions and deletes using `DELETE` transactions.

[Send feedback](#)

3. The API does not support concurrent writes *and* deletes, given this would require both an `UPDATE` and `DELETE` transaction to be active at the same time. This is a consequence of the platform-wide guarantee that a branch can have at most one open transaction. Whichever write or delete request reaches the platform first will open the corresponding transaction; any request of the other type will not be able to open its own transaction until the first one is committed (following the auto-commit behavior described below). This does not put the dataset at risk of an inconsistent state: writes and deletes are never applied concurrently or interleaved, only the order in which the two request types are serviced is timing-dependent.
4. Transactions are automatically committed after a short period of inactivity following a write or delete. This allows you to upload multiple files in the same transaction, avoiding many transactions with very few files. To avoid transactions being kept open indefinitely, for example if files are continuously uploaded such that there is no period of inactivity, a transaction will be committed after a certain amount of time as soon as there are no active uploads. If files are continuously uploaded in parallel, then the transaction will remain open until all uploads are complete.
5. If a read request is received within a period of inactivity while the transaction is open, the `UPDATE` and `DELETE` transactions will be immediately committed where possible. This aims to preserve read-after-write semantics that are generally expected with S3. However, if reads are attempted while there are still active write requests to the open transaction, the read will happen from the latest committed view. To guarantee a transaction commits after all writes or deletes are complete, you can issue a subsequent read request prior to any new write or delete requests.

Given the above behaviors, read-after-write semantics are not guaranteed. However, every effort is made to provide them where possible.

Authentication

Connections through the API are authenticated using access key ID and secret access key credentials.

Static credentials

Static credentials are similar to standard AWS access key ID and secret access key credentials. Static credentials are long-lived and, in the Foundry case, are used by the service user of a third-party application registered in Foundry's Cloud

Send feedback

When using a static access key ID and secret access key to connect to datasets through the S3-compatible API, the access level is determined by the access granted to the third-party application's service user. Static credentials must be restricted to individual [projects](#). The project restrictions are specified when generating a new set of credentials. Only datasets within the specified projects will be accessible using the generated credentials.

See the [setup guide](#) below for guidance on using the `/io/s3/api/v2/credentials` API endpoint to generate these credentials.

Temporary credentials

We recommend using [static credentials](#) in any workflow that requires long-lived credentials or where it is beneficial to tie access to a service user. If you prefer authenticating to the API as your regular Foundry user, we support exchanging a [user authentication token](#) for temporary S3 credentials. This token could also be obtained through one of the [OAuth2](#) grants for a [third-party application](#).

Temporary credentials are obtained using the standard [AssumeRoleWithWebIdentity](#) [Security Token Service \(STS\) API](#). We only require the `WebIdentityToken` request parameter be present and configured with a regular Foundry token as described above. The temporary credentials returned will have an identical scope to that of the provided token. The `DurationSeconds` parameter can be provided to specify the lifetime of the credentials. The credentials will have a maximum lifetime of one hour and will never exceed the lifetime of the Foundry token used to obtain temporary credentials.

The STS API can be accessed at `https://<FOUNDRY_URL>/io/s3`. If you wish to obtain STS credentials programmatically, this URL should be configured in the endpoint configuration of the standard STS clients or credential providers. For example:

```
1 import boto3
2
3 endpoint = "https://<FOUNDRY_URL>/io/s3"
4
5 session = boto3.session.Session()
6 client = session.client(service_name='sts', endpoint_url=endpoint)
```

API Reference ↗

```
7 RoleArn and RoleSessionName are required parameters in boto3 despite be
8
9 credentials = client.assume_role_with_web_identity(
10     RoleArn="xxxxxxxxxxxxxxxxxxxxxx",
11     RoleSessionName="xxxxxx",
```

Send feedback

```
12     WebIdentityToken=token
13 ) ["Credentials"]
```

Alternatively, you can access the API directly using cURL or equivalent such as in the example below. `<TOKEN>` should be replaced with a valid Foundry token.

```
1 curl -X POST \
2     "https://<FOUNDRY_URL>/io/s3?Action=AssumeRoleWithWebIdentity&WebIdentityToken=<TOKEN>"
```

You will receive session credentials in the XML response, as shown below. These credentials should be securely stored.

```
1 <?xml version='1.0' encoding='UTF-8'?>
2 <AssumeRoleWithWebIdentityResponse
3     xmlns="https://sts.amazonaws.com/doc/2011-06-15/">
4     <AssumeRoleWithWebIdentityResult>
5         <Credentials>
6             <AccessKeyId>PLTRLZZJE0...</AccessKeyId>
7             <SecretAccessKey>2j3hKX4EDP...</SecretAccessKey>
8             <SessionToken>eyJwbG50ci...</SessionToken>
9             <Expiration>2023-08-30T10:55:08.841403951Z</Expiration>
10        </Credentials>
11    </AssumeRoleWithWebIdentityResult>
12 </AssumeRoleWithWebIdentityResponse>
```

Once you have temporary credentials, navigate to step four in the [setup guide](#) below for guidance on configuring S3 clients. You do not need to follow any steps regarding third-party applications. When configuring S3 clients, you must provide the session token, the access key ID, and secret access key.

i To read or write data via the S3-compatible API, users need `s3-proxy:datasets-read` and `s3-proxy:datasets-write` permissions which, by default, are granted to `Editor` role respectively. When using static credentials, the service account connecting to the third-party app will need to be granted the relevant role.

API Reference ↗

Send feedback

When using temporary credentials, the user obtaining credentials will need the relevant role.

Path-style URL access

The API supports only [path-style](#) bucket access. Your bucket URLs will take the format `https://<FOUNDRY_URL>/io/s3/<bucket-name>/<key-name>`.

In Foundry terms, this means `https://<FOUNDRY_URL>/io/s3/<dataset-rid>/<logical-filepath>`.

Virtual-hosted-style bucket access (where the bucket name is included in the subdomain of the URL) is currently not supported.

Presigned URLs

Presigned URLs are supported for the `putObject` operation.

Setup guide

Follow these steps to set up a connection to the API from your S3 client.

1. [Register a third-party application in Foundry](#)
2. [Grant permissions to your third-party application's service user](#)
3. [Generate S3 access key ID and secret access key](#)
4. [Configure the S3 client](#)

[View example configuration settings for specific S3 clients.](#)

Step 1: Register a third-party application in Foundry

API Reference ↗

To obtain credentials for the S3-compatible API you will first need to obtain the ID and secret of a [third-party application](#) that has been created in Control Panel.

Send feedback

1. Open **Control Panel** in Foundry.
2. Select **Third party applications** in the sidebar.
3. Select **New application**, then complete the setup wizard with the following parameters:
 - **Client type:** Choose **Confidential client**.
 - **Authorization grant types:** Enable the **Client credentials grant**.
4. Select **Register application** in the lower right of the **Summary** page.
5. On the completion screen, record the **Client ID** as this will be needed in a future step.
Then select **Enable application use** and use the toggle switch to **Enable** it.

 Review [Concepts: Authentication](#) to understand the requirements and behavior for project restrictions, and the scope of generated credentials.

Step 2: Grant permissions to your third-party application's service user

 Users should use [Developer Console](#) to manage their application configuration. The **Control Panel** view only applies if **Developer Console** has not been enabled for the user.

When you created the third-party application in the previous step, Foundry created a service user automatically. To access datasets via the S3 API, this service user must have sufficient permissions on the relevant projects and Markings.

To set up permissions for the service user:

1. Find the name and ID of the service user. You can find these details on your application's "Manage application" page, under **Authorization grant types > Client credentials grant > Service user**.
2. Grant permissions to the service user, either by adding the user to projects and Markings directly, or by adding the user to a group that has been granted those permissions.

[API Reference ↗](#)

Step 3: Generate S3 access key ID and secret

[Send feedback](#)

i To generate credentials you will need to have the **User experience administrator** role on the Organization in [Control Panel](#).

Run the terminal command below (using either curl or PowerShell) to receive an access key ID and secret access key. Replace the following placeholders in the command:

- **<TOKEN>** : An active token for your user account.
- **<CLIENT_ID>** : The client ID of the third-party application generated in the previous step.
- **<PROJECT_RID>** : The resource identifier (RID) of a [project](#) that the credentials should be able to access. See [Find the project RID](#) below for guidance on how to locate this value.

The **projectRestrictions** value can take multiple projects, allowing you to list more project RIDs if necessary. At least one project must be specified. The generated credentials will only be able to access datasets that live within the listed projects.

Option 1: curl

```
1 curl -X POST \  
2   -H "Authorization: Bearer <TOKEN>" \  
3   -H "Content-type: application/json" \  
4   --data '{"clientId":"<CLIENT_ID>","projectRestrictions":["<PROJECT_RID  
5   https://<FOUNDRY_URL>/io/s3/api/v2/credentials
```

Option 2: Powershell (Windows)

```
1 $headers = @{  
2   "Authorization" = "Bearer <TOKEN>"  
3   "Content-type" = "application/json"  
4 }  
5 $body = @{  
6   "clientId" = "<CLIENT_ID>"  
7   "projectRestrictions" = @("<PROJECT_RID>")  
8 }  
9 $body | ConvertTo-Json
```

API Reference ↗

```
10 Invoke-WebRequest -Uri "https://<FOUNDRY_URL>/io/s3/ap
```

Send feedback

Securely store the access key and secret key you receive in the response of this request. You must configure clients with these credentials, *not* the third-party application's client ID and secret.

Find the project RID

The `<PROJECT_RID>` value must be the RID of a top-level [project](#), not the RID of a folder or other resource nested inside a project. Both projects and folders use the RID format `ri.compass.main.folder.{RID_VALUE}`, so the format alone does not distinguish them.

⚠ Supplying the RID of a nested folder, a dataset, or any resource that is not itself a project results in a permissions error when you use the generated credentials. This occurs even though the RID has the expected `ri.compass.main.folder.{RID_VALUE}` format. If you encounter a permissions error, confirm that the value in `projectRestrictions` is the RID of the project itself rather than a resource within it.

To find the RID of a project:

1. Navigate to the project in Foundry and open it at its root level.
2. Right-click the project and copy its RID, or open the [Project details panel](#) to view the project's RID.
3. Confirm that the copied value is the RID of the project itself and not a folder or resource within it.

The `projectRestrictions` value determines which datasets the credentials can reach; it does not control where files are written within a dataset. Because [S3 buckets correspond to Foundry datasets](#), you organize files inside a dataset using object keys (logical file paths) rather than by targeting a Foundry folder. To send files to a specific location, choose the destination dataset as the bucket and use a key prefix such as `logs/` to group files within it.

Specifying an organization

By default, the `/v2/credentials` endpoint assumes the authenticating user is generating credentials for a third-party application in their own [Organization](#). If the third-party application exists in a different Organization, specify the Organization ID as a query parameter in the URL: `https://<FOUNDRY_URL>/io/s3/api/v2/credentials?organizationRid=<ORGANIZATION_ID>`.

API Reference ↗

Send feedback

Revoking credentials

If you need to revoke an access key and secret access key, call the following endpoint and replace `<ACCESS_KEY_ID>` with the access key ID that you wish to revoke:

```
1 curl -X DELETE \
2   -H "Authorization: Bearer <TOKEN>" \
3   https://<FOUNDRY_URL>/io/s3/api/v2/credentials/<ACCESS_KEY_ID>
```

Listing credentials

You can use the following endpoint to retrieve a list of active (non-revoked) access keys, including their client ID and project restrictions.

```
1 curl -X GET \
2   -H "Authorization: Bearer <TOKEN>" \
3   https://<FOUNDRY_URL>/io/s3/api/v2/credentials
```

Step 4: Configure the S3 client

To configure an S3 client you must set the following configuration parameters. See the [examples](#) below for details on how these should be configured in common S3 clients.

Name	Value	Description
Hostname / Endpoint	https://<FOUNDRY_URL>/io/s3	The hostname to which clients should connect (rather than s3.amazonaws.com for native S3 buckets hosted in AWS).
API Reference ↗	foundry	The region must be set to foundry as it is used as part of the V4 signature verification ↗ process. If clients can only use standard AWS regions, us-east-1

Send feedback

Name	Value	Description
Credentials	Access key ID and secret access key, and optionally, session token	Static or temporary credentials generated as described above.
Path-style access	true	The API only supports path-style ↗ bucket access rather than virtual-hosted-style bucket access.
Bucket Name (Optional)	ri.foundry.main.dataset.<uuid>	Each Foundry dataset is accessible as a separate S3 bucket, with the bucket name being the dataset's RID.

Supported actions

The following [S3 actions ↗](#) are supported:

- [AbortMultipartUpload ↗](#)
- [CompleteMultipartUpload ↗](#)
- [CopyObject ↗](#)
- [CreateMultipartUpload ↗](#)
- [DeleteObject ↗](#)
- [DeleteObjects ↗](#)
- [GetObject ↗](#)
- [HeadBucket ↗](#)
- [HeadObject ↗](#)
- [ListMultipartUploads ↗](#)
- [ListObjects ↗](#)
- [ListObjectsV2 ↗](#)
- [ListParts ↗](#)
- [PutObject ↗](#)
- [UploadPart ↗](#)

[API Reference ↗](#)

Client setup examples

[Send feedback](#)

As discussed above, you must ensure the client/SDK/connector is configured to use path-style bucket access. If the client does not support path-style bucket access, it is currently not compatible with this API. For example, with the [S3A Hadoop client ↗](#) this can be configured using the `fs.s3a.path.style.access` flag.

If you are using [temporary credentials](#), be sure to also configure the AWS session access token. Consult the relevant AWS client documentation for details. For example, for the [AWS CLI ↗](#) you should set the `AWS_SESSION_TOKEN` environment variable.

AWS CLI

Once you have an access key ID and secret access key, you are ready to configure the [AWS CLI ↗](#). Run the following command, entering the access key ID, secret access key, and region.

```
$ aws configure --profile foundry
AWS Access Key ID [None]: <ACCESS_KEY_ID>
AWS Secret Access Key [None]: <SECRET_ACCESS_KEY>
Default region name [None]: foundry
Default output format [None]:
```

You should now be able to run commands for the `foundry` profile. For example:

```
1 aws --profile foundry --endpoint-url https://<FOUNDRY_URL>/io/s3 s3 ls s3:
```

As of a [recent release ↗](#) of the AWS CLI, it is now possible to configure the `endpoint-url` as part of the `profile` configuration. A sample `foundry` profile as it would be configured in `~/.aws/config` is shown below. When configuring a profile with the `endpoint_url` property, you no longer need to include the `--endpoint-url` argument when using the `aws` command. Instead, `--profile` is sufficient.

API Reference ↗

```
[foundry]
2 region = foundry
3 endpoint_url = https://<FOUNDRY_URL>/io/s3
```



Send feedback

To use [temporary credentials](#) with the AWS CLI, follow the [AWS documentation](#) that explains how to configure the CLI to make the AWS STS `AssumeRoleWithWebIdentity` call for you. A sample `foundry` profile as it would be configured in `~/.aws/config` is shown below. When using this configuration you do not need to have configured credentials in environment variables (`AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `AWS_SESSION_TOKEN`) or `~/.aws/credentials`.

```
1 [profile foundry]
2 region = foundry
3 endpoint_url = https://<FOUNDRY_URL>/io/s3
4 web_identity_token_file = ~/.foundry/web-identity.token
5 role_arn=xxxxxxxxxxxxxxxxxxxxxx
```

The example above assumes a valid Foundry token is stored in a file at `~/.foundry/web-identity.token`. We only recommend this approach if this file is properly secure and not at risk of being leaked. The `role_arn` property is not used but must still be provided and be at least 20 characters long due to AWS CLI validations. We use `xxxxxxxxxxxxxxxxxxxxxx` as a placeholder in the example. To use this configuration, you must configure the `endpoint_url` in `~/.aws/config` rather than using `--endpoint-url`, as discussed above.

AWS SDK for Python (Boto3)

```
1 import boto3
2 import pandas as pd
3
4 s3 = boto3.client(
5     's3',
6     aws_access_key_id="<ACCESS_KEY_ID>",
7     aws_secret_access_key="<SECRET_ACCESS_KEY>",
8     # aws_session_token="<SESSION_TOKEN>", only needed when using temporary credentials
9     endpoint_url="https://<FOUNDRY_URL>/io/s3",
10    region_name="foundry"
11 )
12
```

API Reference ↗

```
    .foundry.main.dataset.<uuid>'  
    csv'
```

```
15  
16 obj = s3.get_object(Bucket=bucket, Key=key)
```

Send feedback


```
17 df = pd.read_csv(obj['Body'])
18 print(df)
```

Review the [S3 section of Boto3's documentation](#) for more information on connecting to S3-compatible sources using Boto3.

Spark

```
1 from pyspark.sql import SparkSession
2
3 hostname = "https://<FOUNDRY_URL>/io/s3"
4 access_key_id = "<ACCESS_KEY_ID>"
5 secret_access_key = "<SECRET_ACCESS_KEY>"
6
7 # ensure dataset RID can be parsed as a valid hostname
8 dataset_rid = "ri.foundry.main.dataset.<uuid>".replace('.', '-')
9
10 spark_session = (
11     SparkSession.builder
12         .config("fs.s3a.access.key", access_key_id)
13         .config("fs.s3a.secret.key", secret_access_key)
14         # .config("fs.s3a.session.token", session_token) only needed when
15         .config("fs.s3a.endpoint", hostname)
16         .config("fs.s3a.endpoint.region", "foundry")
17         .config("fs.s3a.path.style.access", "true")
18         .getOrCreate()
19 )
20
21 df = spark_session.read.parquet(f"s3a://{dataset_rid}/*")
22 df.show()
```

Review the [Spark documentation](#) for more information on using Spark with S3.

 There is a known issue using the Hadoop AWS client with bucket names that contain '!'. You may encounter an error message such as *"bucket is null/empty"*. If dataset RID cannot be parsed as a valid hostname. As a workaround, you can substitute '!' in the dataset RID with '-!'. [API Reference](#)

[Send feedback](#)

Cyberduck

1. Download the following profile: [Foundry S3.cyberduckprofile ↗](#)
2. Double-click the profile file to open and register the profile in Cyberduck.
3. Set the following connection properties on the default bookmark that Cyberduck created for you:
 - **Server:** `https://<FOUNDRY_URL>`
 - **Access Key ID:** `<ACCESS_KEY_ID>`
 - **Secret Access Key:** `<SECRET_ACCESS_KEY>`
 - **Path:** (under **More Options**) `ri.foundry.main.dataset.<uuid>`
4. Close the bookmark settings window.
5. Double-click the bookmark to open a connection.

Review the [Cyberduck documentation ↗](#) for more information on connecting to S3-compatible sources.

Google Storage Transfer Service

Google Cloud's [Storage Transfer Service ↗](#) can treat Foundry as an [S3-compatible source ↗](#). You can transfer data from a Foundry dataset to a Cloud Storage bucket by following the below steps.

1. Set up an agent pool and transfer agents by following [Google Cloud's instructions ↗](#).
2. Create a transfer job, and select **S3-compatible object storage** as the **Source type**. Then select the agent pool you created in step (1) and ensure the following configuration is set:
 - **Bucket:** `ri.foundry.main.dataset.<uuid>`.
 - **Endpoint:** `https://<FOUNDRY_URL>/io/s3`
 - **Signing region:** `foundry`
 - **Signing process:** Signature Version 4 (SigV4)
 - **Addressing-style:** Path-style requests
 - **Network protocol:** HTTPS

API Reference ↗

Version: `ListObjectsV2` Complete the setup to configure the Cloud Storage bucket as the destination, schedule, and settings of your transfer job.

Send feedback

Apache NiFi

You can use Apache NiFi to read and write files inside a Foundry dataset. The following example shows how to configure the `PutS3Object` processor type for writing:

- **Object Key:** Logical file path in dataset of the form `path/to/file.csv`
- **Bucket:** Dataset RID, such as `ri.foundry.main.dataset.<uuid>`
- **Access Key ID:** `<ACCESS_KEY_ID>`
- **Secret Access Key:** `<SECRET_ACCESS_KEY>`
- **Region:** "US East (N. Virginia)" which corresponds to `us-east-1`
- **Use Path Style Access:** `true`
- **Endpoint Override URL:** `https://<FOUNDRY_URL>/io/s3`

Refer to the [Apache NiFi documentation ↗](#) for more information on the `PutS3Object` processor and other processors that support S3-compatible sources.

Airbyte

[Airbyte ↗](#)'s support for S3 destinations can be used to write files to Foundry datasets. Set the following destination settings:

- **Destination type:** S3
- **S3 Key ID:** `<ACCESS_KEY_ID>`
- **S3 Access Key:** `<SECRET_ACCESS_KEY>`
- **S3 Bucket Name:** `ri.foundry.main.dataset.<uuid>`
- **S3 Bucket Path:** Any valid subdirectory path. Airbyte will write files into this subdirectory within the Foundry dataset.
- **S3 Bucket Region:** `us-east-1`
- **Output Format:** All of Airbyte's output formats are compatible with Foundry datasets.
- **Endpoint:** `https://<FOUNDRY_URL>/io/s3`

Refer to Airbyte's documentation for [S3 destinations ↗](#) for more information and [configuration options](#).

API Reference ↗

DuckDB

Send feedback

[DuckDB](#)'s support for S3 can be used to query Foundry datasets. You can manage credentials using DuckDB secrets and query datasets using the `s3://` prefixed URLs.

```
CREATE SECRET foundry_secret (  
  TYPE S3,  
  KEY_ID '<ACCESS_KEY_ID>',  
  SECRET '<SECRET_ACCESS_KEY>',  
  REGION 'foundry',  
  ENDPOINT '<FOUNDRY_URL>/io/s3',  
  URL_STYLE 'path'  
);  
  
CREATE TABLE new_tbl AS SELECT * FROM 's3://ri.foundry.main.dataset.<uuid>/spark'
```

Refer to the [DuckDB documentation](#) for more information.

- i** In the secret configuration above, the `ENDPOINT` configuration parameter should not include the `https://` scheme. The URL scheme is handled automatically by the `USE_SSL` parameter, which defaults to `true`.

Polars

[Polars](#)' support for S3 can be used to query Foundry datasets.

```
1 import polars as pl  
2  
3 hostname = "https://<FOUNDRY_URL>/io/s3"  
4 access_key_id = "<ACCESS_KEY_ID>"  
5 secret_access_key = "<SECRET_ACCESS_KEY>"  
6 dataset_id = "ri.foundry.main.dataset.<uuid>"  
7  
8 storage_options = {  
9     "access_key_id": access_key_id,  
10    "secret_access_key": secret_access_key,  
11    "aws_region": "foundry",  
12    "endpoint_url": hostname  
13 }
```

API Reference [↗](#)

Send feedback

```
14
15 df = pl.scan_parquet(f"s3://{dataset_id}/spark/*.parquet", storage_optio
```

Refer to the [Polars documentation ↗](#) for more information.

PREVIOUS

← [Microsoft Excel / Transforms Excel Parser](#)

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)